UTL-X

Format-Agnostic Data Transformation

The Complete Guide

Ir. Marcel A. Grauwen

Creator of UTL-X

Version 1.0 — 2026

Table of Contents

Preface ... 18
- 0.1 Why This Book ... 18
- 0.2 Who This Book Is For ... 18
- 0.3 How to Read This Book ... 18
- 0.4 Conventions Used ... 18
- 0.5 Online Resources ... 18

1 Introduction ... 20
- 1.1 The Data Transformation Problem ... 20
- 1.2 What Is UTL-X? ... 20
- 1.3 The Swiss Army Knife Analogy ... 20
- 1.4 UTL-X vs The Alternatives ... 21
- 1.5 Architecture Overview ... 21
- 1.6 The UTL-X Ecosystem ... 22
- 1.7 A First Example ... 22
- 1.8 What's Ahead ... 23

2 Licensing: Open Source and Commercial ... 25
- 2.1 UTL-X Is Open Source (AGPL-3.0) ... 25
- 2.2 What You Can Do (Without Paying Anything) ... 25
- 2.3 The Key AGPL Condition ... 25
- 2.4 Your Transformations Are Yours ... 25
- 2.5 Usage Scenarios ... 26
- 2.6 The Subprocess Exception ... 26
- 2.7 The Commercial License ... 27
  - 2.7.1 Why It Exists ... 27
  - 2.7.2 Cloud Marketplace (Simplest Path) ... 27
  - 2.7.3 Direct Commercial License ... 27
- 2.8 Dual Licensing: How It Works ... 27
- 2.9 Frequently Asked Questions ... 27

3 Transformation: The Cornerstone of Integration ... 29
- 3.1 The Three Pillars of Integration ... 29
- 3.2 Why Transformation Is the Hardest Pillar ... 29
- 3.3 Transformation in the Enterprise Service Bus ... 30
- 3.4 The JSON-to-XML-to-XSLT Anti-Pattern ... 30
- 3.5 The UTL-X Approach: No Intermediate Conversion ... 31
- 3.6 Why UTLXe Needs Messaging Capabilities ... 31
- 3.7 The Canonical Data Model ... 31
- 3.8 What This Means for You ... 32

4 Getting Started ... 33
- 4.1 Installation ... 33
  - 4.1.1 macOS (Homebrew) — Recommended ... 33
  - 4.1.2 Windows (Chocolatey) ... 33
  - 4.1.3 Linux / macOS / Windows — Direct Download ... 33
  - 4.1.4 Building from Source ... 34
  - 4.1.5 Native Binary vs JVM JAR ... 34
- 4.2 Your First Transformation ... 35

4.3 The .utlx File Structure ... 35
4.3.1 Input Format Options ... 36
4.3.2 Output Format Options ... 36
4.3.3 Multi-Input ... 36
4.4 Trying It Out: Five Quick Examples ... 36
4.5 What's Next ... 37
5 UTL-X on the Command Line ... 38
5.1 Three Ways to Use the CLI ... 38
5.2 Expression Mode: The jq Experience ... 38
5.2.1 Raw Output (-r) ... 38
5.2.2 Expressions with Any Format ... 38
5.2.3 Complex Expressions ... 39
5.3 Identity Mode: The Flip ... 39
5.4 Shell Scripting Patterns ... 40
5.4.1 Batch File Conversion ... 40
5.4.2 Pipeline Integration ... 40
5.4.3 Using UTL-X in CI/CD ... 41
5.4.4 Combining with jq ... 41
5.5 The UTL-X jq Cheat Sheet ... 41
5.6 When to Use Which Mode ... 42
5.7 Performance Notes ... 42
6 The Three Executables: utlx, utlxd, and utlxe ... 43
6.1 One Language, Three Runtimes ... 43
6.2 utlx — The CLI ... 43
6.3 utlxd — The Daemon ... 44
6.4 utlxe — The Engine ... 44
6.5 Comparison at a Glance ... 45
6.6 The Wrapper Pattern ... 46
6.7 When to Use Which ... 46
7 The UTL-X IDE ... 48
7.1 Installing the VS Code Extension ... 48
7.2 The Three-Panel Layout ... 48
7.3 Design-Time vs Runtime ... 48
7.4 Schema-Driven Development ... 49
7.4.1 The Professional Approach ... 49
7.4.2 Scaffolding ... 49
7.5 The UDM Tree Browser ... 49
7.6 The Function Library ... 50
7.7 Real-Time Diagnostics ... 50
7.8 Autocompletion ... 50
7.9 The Header Editor ... 51
7.10 Keyboard Shortcuts ... 51
7.11 Tips for Productive IDE Usage ... 51
8 Language Fundamentals ... 52
8.1 Scalar Types ... 52
8.2 Objects ... 52

8.2.1 Object Literals ... 52
8.2.2 Property Access ... 53
8.2.3 Nested Access ... 53
8.2.4 XML Attribute Access ... 53
8.2.5 Computed Property Names ... 53
8.2.6 The Spread Operator ... 53
8.3 Arrays ... 54
8.3.1 Array Literals ... 54
8.3.2 Index Access ... 54
8.3.3 Array Operations ... 54
8.4 The $input Variable ... 54
8.4.1 Safe Navigation ... 55
8.4.2 Recursive Descent ... 55
8.5 Variable Binding (let) ... 55
8.5.1 Top-Level: No Separator Needed ... 56
8.5.2 Inside Object Literals: Commas Required ... 56
8.5.3 Inside Lambdas Returning Arrays: Semicolons Required ... 56
8.5.4 Summary of Let Separators ... 57
8.6 Conditional Expressions ... 57
8.6.1 if / else ... 57
8.6.2 Ternary Operator ... 57
8.6.3 Nullish Coalescing (??) ... 58
8.6.4 Combining Conditionals ... 58
8.7 Comments ... 58
8.8 Type Annotations (Optional) ... 58
8.9 Putting It Together ... 58
9 The Universal Data Model (UDM) ... 60
9.1 Why UDM Exists ... 60
9.2 UDM Types ... 60
9.3 How Formats Map to UDM ... 61
9.3.1 JSON → UDM ... 61
9.3.2 XML → UDM ... 61
9.3.3 CSV → UDM ... 62
9.3.4 YAML → UDM ... 62
9.3.5 OData → UDM ... 62
9.4 UDM Navigation ... 62
9.4.1 Dot Notation ... 62
9.4.2 Safe Navigation ... 62
9.4.3 Recursive Descent ... 63
9.5 UDM and Type Coercion ... 63
9.5.1 Automatic Coercion ... 63
9.5.2 Explicit Coercion ... 63
9.5.3 XML Text Node Unwrapping ... 63
9.6 UDM is Format-Agnostic, Transformations are Format-Agnostic ... 64
9.7 UDM and Flat/Relational Data ... 64
9.7.1 The Flat Data Challenge ... 64

9.7.2 How to Handle It in UTL-X . . . 64
9.7.3 Hierarchical vs Relational: A Design Trade-Off . . . 65
9.8 When UDM Matters . . . 66
10 Schema-to-Schema Mapping . . . 67
10.1 Why Schema Mapping Matters . . . 67
10.2 How Traditional Tools Do It . . . 67
10.2.1 The XSLT Approach . . . 67
10.2.2 The JSON Problem . . . 67
10.3 The UTL-X Approach . . . 68
10.4 Tier 1: Instance Data Mapping . . . 68
10.4.1 Every Data Format Has an Accompanying Schema Format . . . 68
10.4.2 Common Instance Mapping Scenarios . . . 68
10.5 Tier 2: Metadata Mapping (Schema-to-Schema) . . . 69
10.5.1 Why Metadata Mapping Is Powerful . . . 69
10.5.2 The Full Schema-to-Schema Matrix . . . 69
10.5.3 Practical Use Cases for Schema Mapping . . . 69
10.6 Nested Schemas: Include and Import . . . 70
10.7 JSON Schema to JSON Schema Mapping . . . 70
10.8 Schema-Driven Validation in UTL-X . . . 70
10.8.1 Pre-Validation . . . 70
10.8.2 Post-Validation . . . 71
10.8.3 The Validation Orchestrator . . . 71
10.9 The Bridge: USDL . . . 71
11 Format Support Overview . . . 73
11.1 Tier 1: Data Formats . . . 73
11.1.1 JSON . . . 73
11.1.2 XML . . . 73
11.1.3 CSV . . . 73
11.1.4 YAML . . . 74
11.1.5 OData . . . 74
11.2 Tier 2: Schema Formats . . . 74
11.2.1 XSD (XML Schema Definition) . . . 74
11.2.2 JSCH (JSON Schema) . . . 74
11.2.3 Avro . . . 74
11.2.4 Protobuf (Protocol Buffers) . . . 74
11.2.5 OSCH (OData Schema / EDMX) . . . 75
11.2.6 TSCH (Table Schema) . . . 75
11.3 Format Detection . . . 75
11.4 Format Conversion Matrix . . . 75
11.5 Identity Mode Defaults . . . 75
11.6 Format Options Summary . . . 76
11.7 What's Next . . . 76
12 Expressions and Operators . . . 78
12.1 Arithmetic Operators . . . 78
12.2 Comparison Operators . . . 78
12.3 Logical Operators . . . 79

12.3.1 Truthy and Falsy Values ... 79
12.4 String Concatenation ... 79
12.5 Object Construction ... 79
12.5.1 Conditional Properties ... 80
12.6 Array Construction ... 80
12.7 The Pipe Operator (|>) ... 81
12.8 Nullish Coalescing (??) ... 81
12.9 Safe Navigation (?.) ... 81
12.10 Operator Precedence ... 82
12.11 Practical Expression Patterns ... 82
12.11.1 Price Calculation ... 82
12.11.2 Conditional Formatting ... 83
12.11.3 Data Normalization ... 83
13 Functions and Lambdas ... 84
13.1 Lambda Expressions ... 84
13.1.1 Basic Syntax ... 84
13.1.2 Multiple Parameters ... 84
13.1.3 No Parameters ... 84
13.1.4 Lambdas in Action ... 84
13.2 User-Defined Functions ... 85
13.2.1 Function Definitions Are Expressions ... 85
13.2.2 Recursive Functions ... 85
13.3 Higher-Order Functions ... 86
13.3.1 map — Transform Each Element ... 86
13.3.2 filter — Select Matching Elements ... 86
13.3.3 reduce — Accumulate a Result ... 86
13.3.4 find — First Match ... 87
13.3.5 sortBy — Order by Computed Value ... 87
13.3.6 groupBy — Group by Key ... 87
13.4 Function Composition ... 87
13.4.1 Pipe Chains ... 87
13.4.2 Nested Functions ... 88
13.4.3 Combining map and filter ... 88
13.5 Closures ... 88
13.6 Practical Patterns ... 89
13.6.1 Flatten and Restructure ... 89
13.6.2 Aggregate with Multiple Results ... 89
13.6.3 Lookup Table Pattern ... 89
13.6.4 Conditional Transformation ... 89
13.7 The 652 Standard Library Functions ... 90
14 Standard Library Overview ... 91
14.1 String Functions (83 functions) ... 91
14.1.1 Most-Used String Functions ... 91
14.1.2 Case Conversion ... 91
14.1.3 Padding and Formatting ... 91
14.2 Array Functions (67 functions) ... 92

14.2.1 Transform and Filter ... 92
14.2.2 Search and Access ... 92
14.2.3 Aggregate ... 92
14.2.4 Reshape ... 92
14.3 Math Functions (37 functions) ... 92
14.4 Date/Time Functions (68 functions) ... 93
14.4.1 Current Time ... 93
14.4.2 Parse and Format ... 93
14.4.3 Arithmetic ... 93
14.4.4 Extract Components ... 93
14.5 Type Functions (27 functions) ... 93
14.5.1 Type Checking ... 93
14.5.2 Type Conversion ... 94
14.6 Encoding Functions (30 functions) ... 94
14.7 Security Functions (16 functions) ... 94
14.8 XML Functions (60 functions) ... 94
14.9 Other Categories ... 94
14.9.1 JSON Functions (6) ... 94
14.9.2 CSV Functions (12) ... 95
14.9.3 YAML Functions (22) ... 95
14.9.4 Financial Functions (16) ... 95
14.9.5 Geospatial Functions (8) ... 95
14.9.6 Binary Functions (47) ... 95
14.9.7 Utility Functions (27) ... 95
14.9.8 Object Functions (1) ... 95
14.10 Finding the Right Function ... 96
14.10.1 In the CLI ... 96
14.10.2 In the IDE ... 96
14.10.3 In This Book ... 96
14.11 A Note on Naming ... 96
15 Security Library and Message Security ... 97
15.1 The Security Library (stdlib-security) ... 97
15.1.1 Available Functions ... 97
15.1.2 Hashing ... 97
15.1.3 HMAC (Hash-based Message Authentication) ... 97
15.1.4 UUID Generation ... 98
15.2 Transport Security: mTLS ... 98
15.3 Message-Level Encryption ... 98
15.3.1 When It's Required ... 98
15.3.2 The Encrypt-Transform-Re-encrypt Pattern ... 98
15.3.3 Why This Is a Maintenance Burden ... 99
15.3.4 Recommendation ... 99
15.4 Digital Signatures and Integrity ... 99
15.4.1 When You Need Signatures ... 99
15.4.2 Impact on Transformation ... 99
15.4.3 What UTL-X Can Do Today ... 100

15.4.4 What UTL-X Cannot Do Yet (Future) . . . 100
15.5 Large Files and Sensitive Data . . . 100
15.5.1 The Claim Check Pattern . . . 100
15.5.2 Avoid Base64 Embedding . . . 101
15.6 What NOT to Log . . . 101
16 Pattern Matching and Control Flow . . . 102
16.1 Match Expressions . . . 102
16.1.1 Basic Value Matching . . . 102
16.1.2 Matching with Guards . . . 102
16.1.3 Complex Matching . . . 102
16.1.4 Match vs If/Else Chains . . . 103
16.2 Try/Catch — Error Recovery . . . 103
16.2.1 Basic Try/Catch . . . 103
16.2.2 Using the Error Variable . . . 103
16.2.3 Try/Catch in Map . . . 104
16.2.4 When to Use Try/Catch vs Null Coalescing . . . 104
16.3 Practical Control Flow Patterns . . . 104
16.3.1 Message Type Routing . . . 104
16.3.2 API Version Handling . . . 105
16.3.3 Country-Specific Business Rules . . . 105
16.3.4 Data Cleansing with Fallbacks . . . 105
16.3.5 Polymorphic Output (Different Structure Based on Input) . . . 106
16.4 Control Flow Summary . . . 106
17 Schema Validation . . . 107
17.1 Why Validate? . . . 107
17.2 Supported Schema Formats . . . 107
17.3 The Validation Orchestrator . . . 108
17.3.1 Configuring Validation . . . 108
17.4 Validation Policies . . . 108
17.5 JSON Schema Validation . . . 109
17.5.1 What JSON Schema Checks . . . 109
17.5.2 Validation Errors . . . 109
17.6 XSD Validation . . . 110
17.7 Syntactical vs Semantic Validation . . . 110
17.7.1 What Syntactical Validation Catches . . . 110
17.7.2 What Syntactical Validation Cannot Catch . . . 110
17.7.3 The Gap . . . 110
17.8 Validation in Practice . . . 111
17.8.1 Development Workflow . . . 111
17.8.2 Production Workflow . . . 111
17.8.3 When to Skip Validation . . . 111
17.9 Validation and the Conformance Suite . . . 111
18 Pipeline Chaining and Multi-Hop Transformations . . . 112
18.1 What Is Pipeline Chaining? . . . 112
18.2 Why Pipelines? . . . 112
18.3 Single-Hop vs Multi-Hop . . . 112

18.3.1 Single-Hop (Simple Transformation) . . . 112
18.3.2 Multi-Hop (Pipeline) . . . 112
18.4 Multi-Hop with Additional Inputs per Stage . . . 112
18.4.1 The Problem . . . 112
18.4.2 The Solution: Additional Inputs per Hop . . . 112
18.4.3 When Is Multi-Hop with Additional Inputs Handy? . . . 112
18.5 Defining Pipelines . . . 112
18.5.1 Pipeline via UTLXe HTTP API . . . 112
18.5.2 Pipeline via CLI . . . 112
18.6 Multi-Input Transformations (Single Step) . . . 112
18.7 Pipeline Patterns . . . 112
18.7.1 Enrichment Pipeline . . . 112
18.7.2 Validation Sandwich . . . 112
18.7.3 Fan-Out . . . 112
18.7.4 Fan-In (Aggregation) . . . 112
18.7.5 Conditional Routing . . . 112
18.8 Performance Characteristics . . . 112
18.9 Status: Multi-Hop Additional Inputs . . . 112
19 Data Restructuring: Flat, Hierarchical, and Back . . . 113
19.1 The Six Restructuring Patterns . . . 113
19.2 Pattern 1: Flat → Hierarchical (join) . . . 113
19.2.1 The Problem . . . 113
19.2.2 The Solution: join() . . . 113
19.2.3 Today's Workaround (without join) . . . 114
19.3 Pattern 2: Record Enrichment (lookup) . . . 114
19.3.1 The Problem . . . 114
19.3.2 The Solution: lookup() . . . 115
19.3.3 Today's Workaround (without lookup) . . . 115
19.4 Pattern 3: Sequential Grouping (chunkBy) . . . 115
19.4.1 The Problem . . . 115
19.4.2 The Solution: chunkBy() . . . 115
19.4.3 Today's Workaround (without chunkBy) . . . 116
19.5 Pattern 4: Denormalization (unnest) . . . 116
19.5.1 The Problem . . . 116
19.5.2 The Solution: unnest() . . . 116
19.5.3 Today's Workaround (without unnest) . . . 116
19.6 Pattern 5: Key-Based Grouping (groupBy — exists) . . . 117
19.7 Pattern 6: Array Flattening (flatten — exists) . . . 117
19.8 Combining Patterns . . . 117
19.8.1 IDoc → Enriched Hierarchical JSON . . . 117
19.8.2 Hierarchical → Flat CSV Report . . . 118
19.9 Choosing the Right Pattern . . . 118
20 XML Transformations . . . 120
20.1 Reading XML . . . 120
20.1.1 Element Access . . . 120
20.1.2 Attribute Access . . . 120

20.1.3 Repeated Elements ... 120
20.1.4 The Single-Element Problem ... 121
20.1.5 Nested Access ... 121
20.1.6 Namespace-Prefixed Elements ... 121
20.2 Writing XML ... 122
20.2.1 Basic Element Output ... 122
20.2.2 Creating Attributes ... 122
20.2.3 Self-Closing Elements ... 123
20.2.4 Namespace Declarations ... 123
20.2.5 Encoding ... 123
20.3 XML Encoding Detection and Conversion ... 123
20.4 XML Attributes in JSON/YAML Output ... 124
20.5 Common XML Patterns ... 124
20.5.1 SOAP Envelope Parsing ... 124
20.5.2 UBL Invoice (Peppol BIS 3.0) ... 124
20.5.3 HL7 FHIR XML ... 124
20.5.4 SAP IDoc XML ... 125
20.6 XML Round-Trip ... 125
21 XML Attribute Handling: Design Decisions ... 126
21.1 The Fundamental Problem ... 126
21.2 How UTL-X Handles Attributes Internally (UDM) ... 126
21.3 The _text Convention ... 126
21.4 The writeAttributes Option ... 126
21.5 What Gets Kept vs Dropped (Default) ... 126
21.6 Why Drop by Default? (DataWeave-Compatible) ... 126
21.7 JSON vs YAML Consistency ... 126
21.8 Industry Conventions Reference ... 126
21.9 FHIR XML: The value Attribute Convention ... 126
22 JSON Transformations ... 127
22.1 JSON in UTL-X ... 127
22.2 Reading JSON ... 127
22.3 Writing JSON ... 127
22.4 JSON-Specific Functions ... 127
22.5 Common JSON Patterns ... 127
22.6 JSON Lines (JSONL) ... 127
23 CSV Transformations ... 128
23.1 CSV in UTL-X ... 128
23.2 CSV Options ... 128
23.3 Reading CSV ... 128
23.4 Writing CSV ... 128
23.5 CSV-Specific Functions ... 128
23.6 Common CSV Patterns ... 128
23.7 Regional Formats ... 128
24 YAML Transformations ... 129
24.1 YAML in UTL-X ... 129
24.2 Reading YAML ... 129

24.3 Writing YAML ... 129
24.4 YAML Output Options ... 129
24.5 YAML-Specific Functions ... 129
24.6 Common YAML Patterns ... 129
24.7 YAML ↔ JSON Conversion ... 129
25 OData Transformations ... 130
25.1 What Is OData? ... 130
25.2 OData in UTL-X ... 130
25.3 Reading OData ... 130
25.4 Writing OData ... 130
25.5 OData-Specific Functions ... 130
25.6 Common OData Patterns ... 130
25.6.1 Dynamics 365 Integration ... 130
25.6.2 SharePoint Integration ... 130
25.6.3 SAP OData Integration ... 130
25.6.4 Power Platform ... 130
25.7 OData Schema (EDMX/CSDL) ... 130
25.8 OData ↔ Other Formats ... 130
26 Schema Formats ... 131
26.1 XSD (XML Schema Definition) ... 131
26.2 JSON Schema ... 131
26.3 Avro Schema ... 131
26.4 Protobuf (proto3) ... 131
26.5 OSCH (OData Schema / EDMX) ... 131
26.6 TSCH (Table Schema) ... 131
26.7 USDL (Universal Schema Definition Language) ... 131
27 XSD Design Patterns Deep Dive ... 132
27.1 The Eight XML Schema Design Patterns ... 132
27.1.1 1. Russian Doll ... 132
27.1.2 2. Salami Slice ... 132
27.1.3 3. Venetian Blind ... 132
27.1.4 4. Garden of Eden ... 132
27.1.5 5. Swiss Army Knife (Hybrid) ... 132
27.1.6 6-8. Chameleon, Abstract, Adapter ... 132
27.2 Pattern Detection in UTL-X ... 132
27.3 Pattern Conversion in UTL-X ... 132
27.4 XSD Patterns in Cross-Format Mapping ... 132
28 Cross-Format Patterns ... 133
28.1 The Power of Format Agnosticism ... 133
28.2 XML ↔ JSON Conversion ... 133
28.3 CSV ↔ JSON Conversion ... 133
28.4 XML → CSV (Flattening) ... 133
28.5 JSON → XML (Structuring) ... 133
28.6 Multi-Format Pipelines ... 133
28.7 Data Normalization Patterns ... 133
29 Enterprise Integration ... 135

29.1 UTL-X in Integration Architecture ... 135
29.2 Use Case: European E-Invoicing (Peppol/UBL) ... 135
29.3 Use Case: Healthcare (HL7 FHIR) ... 135
29.4 Use Case: Financial Services (SWIFT/ISO 20022) ... 135
29.5 Use Case: Retail/E-Commerce ... 135
29.6 Message Broker Integration ... 135
29.7 Error Handling in Production ... 135
30 Engine Lifecycle: Design-Time, Init-Time, Runtime ... 136
30.1 The Three Phases ... 136
30.1.1 Phase 1: Design-Time ... 136
30.1.2 Phase 2: Init-Time (Load / Compile) ... 136
30.1.3 Phase 3: Runtime (Execute) ... 136
30.2 Phase Diagram ... 136
30.3 How Strategies Affect Each Phase ... 136
30.4 Worker Threading Model ... 136
31 Cloud Deployment ... 137
31.1 UTLXe: The Production Engine ... 137
31.2 Azure Deployment ... 137
31.3 GCP Deployment ... 137
31.4 AWS Deployment ... 137
31.5 Docker and Kubernetes ... 137
31.6 Health and Monitoring ... 137
31.7 Hot Reload ... 137
32 SDKs and Wrapper Libraries ... 138
32.1 The Wrapper Pattern ... 138
32.2 Available SDKs ... 138
32.2.1 C# / .NET Wrapper (Built) ... 138
32.2.2 Go Wrapper (Built) ... 138
32.2.3 Python Wrapper (Planned) ... 138
32.2.4 Java / Kotlin Wrapper (Direct) ... 138
32.2.5 Node.js / TypeScript Wrapper (Planned) ... 138
32.3 Alternative: HTTP API (No SDK Needed) ... 138
32.4 Alternative: Dapr Sidecar (Cloud-Native) ... 138
32.5 SDK Comparison ... 138
33 Migration Guides ... 139
33.1 Migrating from XSLT ... 139
33.2 Migrating from DataWeave ... 139
33.3 Migrating from jq ... 139
33.4 Migrating from Custom Code ... 139
33.5 Migrating from Tibco BW / BWCE ... 139
34 Performance and Optimization ... 140
34.1 Execution Strategies ... 140
34.2 Performance Characteristics ... 140
34.3 Memory Optimization ... 140
34.4 GraalVM Native Image ... 140
34.5 Benchmarking ... 140

34.6 Production Tuning ... 140
35 Message Parsing, Memory, and Size Boundaries ... 141
35.1 How UTL-X Parses Messages ... 141
35.1.1 DOM-Style Parsing (What UTL-X Uses) ... 141
35.1.2 SAX-Style Parsing (Not Used by UTL-X) ... 141
35.1.3 StAX-Style Parsing (Not Used by UTL-X) ... 141
35.1.4 Summary: Parsing Approaches ... 141
35.2 Memory Expansion Factor ... 141
35.2.1 The Problem: Why 100KB of XML Becomes 5-10MB in Memory ... 141
35.2.2 Expansion Factors by Format ... 141
35.2.3 Real-World Examples ... 141
35.3 Practical Message Size Boundaries ... 141
35.3.1 Sweet Spot: 1 KB - 1 MB ... 141
35.3.2 Feasible: 1 MB - 10 MB ... 141
35.3.3 Not Recommended: 10 MB - 50 MB ... 141
35.3.4 Not Feasible: 50 MB+ ... 141
35.3.5 Size Boundary Summary ... 141
35.4 Memory Sizing Guide for Production ... 141
35.4.1 Formula ... 141
35.4.2 The 75% Rule ... 141
35.5 Future: Streaming Support? ... 141
35.5.1 Could UTL-X Support Streaming? ... 141
35.5.2 Split-Before-Transform Pattern ... 141
35.6 Comparison: UTL-X vs Streaming Tools ... 141
36 Logging, Observability, and Compliance ... 142
36.1 Logging in UTL-X ... 142
36.1.1 What UTL-X Logs ... 142
36.1.2 Log Levels ... 142
36.1.3 Configuring Logging ... 142
36.2 When NOT to Log: Compliance and Data Protection ... 142
36.2.1 PCI DSS (Payment Card Industry Data Security Standard) ... 142
36.2.2 GDPR (General Data Protection Regulation) ... 142
36.2.3 HIPAA (Health Insurance Portability and Accountability Act) ... 142
36.2.4 NEN 7510 (Dutch Healthcare Information Security) ... 142
36.2.5 SOX (Sarbanes-Oxley Act — Financial) ... 142
36.3 Designing for Compliance ... 142
36.3.1 The Principle: Log Metadata, Not Data ... 142
36.3.2 Audit Logging Pattern ... 142
36.3.3 Masking in Error Messages ... 142
36.4 Prometheus Metrics (Safe by Design) ... 142
37 Quality Assurance and Testing ... 143
37.1 UTL-X's Layered Testing Strategy ... 143
37.2 Layer 1: Unit Tests (Kotlin) ... 143
37.3 Layer 2: Format Tests (Kotlin) ... 143
37.4 Layer 3: Module Tests (Kotlin) ... 143
37.5 Layer 4: Integration Tests (Kotlin) ... 143

37.6 Layer 5: Conformance Suite (Python) . . . 143
37.7 Auto-Capture: Recording Tests from CLI Usage . . . 143
37.8 UTLXe Engine Tests . . . 143
37.9 Test-Driven Transformation Development . . . 143
37.10 Continuous Integration . . . 143
38 Semantic Validation: Beyond Syntax . . . 145
38.1 Syntactical vs Semantic Validation . . . 145
38.1.1 What UTL-X Does Today: Syntactical Validation . . . 145
38.1.2 What UTL-X Does NOT Do: Semantic Validation . . . 145
38.1.3 The Gap . . . 145
38.2 Semantic Validation Languages . . . 145
38.2.1 Schematron (XML world) . . . 145
38.2.2 JSONTron . . . 145
38.2.3 FEEL (Friendly Enough Expression Language) . . . 145
38.2.4 Other Approaches . . . 145
38.3 How Semantic Validation Could Work in UTL-X . . . 145
38.3.1 Option 1: Built-in assertion syntax . . . 145
38.3.2 Option 2: Schematron support for XML output . . . 145
38.3.3 Option 3: FEEL integration for cross-format validation . . . 145
38.3.4 Option 4: Validation DSL (UTL-X native) . . . 145
38.3.5 Recommended Approach . . . 145
39 API Contracts and Data Contracts . . . 146
39.1 What Is an API Contract? . . . 146
39.2 Two Dimensions of Contracts . . . 146
39.2.1 Hierarchical Contracts (API-Centric) . . . 146
39.2.2 Relational Contracts (Data-Centric) . . . 146
39.2.3 Key Difference . . . 146
39.3 Hierarchical API Contract Standards . . . 146
39.3.1 OpenAPI (REST APIs) . . . 146
39.3.2 RAML (REST APIs — MuleSoft) . . . 146
39.3.3 SOAP / WSDL (XML Web Services) . . . 146
39.3.4 AsyncAPI (Event-Driven APIs) . . . 146
39.3.5 gRPC / Protocol Buffers . . . 146
39.4 Abstract vs Concrete API Contracts . . . 146
39.4.1 Abstract Contracts (Schema-Only) . . . 146
39.4.2 Concrete Contracts (Full API Specification) . . . 146
39.4.3 The Relationship . . . 146
39.5 Relational Data Contract Standards . . . 146
39.5.1 Data Contract (datacontract.com) . . . 146
39.5.2 ODCS (Open Data Contract Standard — PayPal) . . . 146
39.5.3 Comparison: datacontract.com vs ODCS . . . 146
39.6 How UTL-X Relates to API and Data Contracts . . . 146
39.6.1 UTL-X as the Transformation Layer . . . 146
39.6.2 Schema Formats in UTL-X . . . 146
39.6.3 Future: Data Contract Integration . . . 146
39.6.4 Future: Contract-Driven Transformation . . . 146

40 Formats Not Yet Covered (and Why) ... 147
40.1 Formats on the Horizon ... 147
40.2 TOML (Tom's Obvious Minimal Language) ... 147
40.3 JSON-LD (JSON for Linked Data) ... 147
40.4 HL7 FHIR (Fast Healthcare Interoperability Resources) ... 147
40.5 EDI / EDIFACT (Electronic Data Interchange) ... 147
40.6 HL7 v2 (Health Level Seven Version 2) ... 147
40.7 Parquet / Arrow (Columnar Data) ... 147
40.8 MessagePack / CBOR (Binary JSON) ... 147
40.9 GraphQL ... 147
40.10 Apache Thrift ... 147
40.11 FlatBuffers ... 147
40.12 Summary: Format Priority Matrix ... 147
41 Competitive Landscape: Mapping and Integration Tools ... 148
41.1 Command-Line Transformation Tools ... 148
41.1.1 jq — The JSON Swiss Knife ... 148
41.1.2 yq — YAML/JSON/XML/CSV/TOML Processor ... 148
41.1.3 JSONata — JSON Query and Transformation ... 148
41.1.4 XSLT — The XML Veteran ... 148
41.2 Enterprise Integration Platforms (iPaaS) ... 148
41.2.1 MuleSoft (Salesforce) ... 148
41.2.2 Tibco BusinessWorks (BWCE) ... 148
41.2.3 Boomi (formerly Dell Boomi) ... 148
41.2.4 Workato ... 148
41.2.5 Informatica / IICS ... 148
41.3 Hyperscaler Native Tools ... 148
41.3.1 Azure ... 148
41.3.2 AWS ... 148
41.3.3 GCP ... 148
41.3.4 Hyperscaler Comparison Table ... 148
41.4 ERP Vendors ... 148
41.4.1 SAP ... 148
41.4.2 Oracle ... 148
41.4.3 Microsoft Dynamics 365 ... 148
41.4.4 Odoo ... 148
41.4.5 Summary: ERP + UTL-X Integration Pattern ... 148
41.5 Open Source Alternatives ... 148
41.5.1 Apache Camel ... 148
41.5.2 Smooks ... 148
41.5.3 Ballerina ... 148
41.6 Positioning Matrix ... 148
42 Open-M: The Integration Controller (Teaser) ... 149
42.1 What Is Open-M? ... 149
42.2 How UTL-X Fits in Open-M ... 149
42.3 Why Open-M + UTL-X? ... 149
42.4 Open-M Architecture (Preview) ... 149

42.5 MPPM: Message Processing Pipeline Model . . . 149
42.5.1 What Is MPPM? . . . 149
42.5.2 MPPM Stages . . . 149
42.5.3 MPPM Message Envelope . . . 149
42.5.4 How MPPM Maps to UTL-X . . . 149
42.5.5 Why MPPM Matters for UTL-X Users . . . 149
42.6 What This Means for UTL-X Users . . . 149
43 AI and UTL-X: The Future of Automated Transformation . . . 150
43.1 The Current State: AI-Assisted UTL-X Development . . . 150
43.1.1 What Works Today . . . 150
43.1.2 The Problem: Token Cost and Context . . . 150
43.2 The Vision: A UTL-X Fine-Tuned LLM . . . 150
43.2.1 What Fine-Tuning Would Enable . . . 150
43.2.2 Token Savings Calculation . . . 150
43.3 Training Data: What Would Be Needed . . . 150
43.3.1 Available Training Corpus . . . 150
43.3.2 Additional Training Data Needed . . . 150
43.4 Fine-Tuning Approaches . . . 150
43.4.1 Option A: LoRA Fine-Tune on Open Model . . . 150
43.4.2 Option B: Fine-Tune on Frontier Model . . . 150
43.4.3 Option C: RAG (Retrieval-Augmented Generation) — No Fine-Tuning . . . 150
43.4.4 Recommended Path . . . 150
43.5 Investment Estimate . . . 150
43.5.1 Development Costs . . . 150
43.5.2 Ongoing Costs . . . 150
43.5.3 Revenue Potential . . . 150
43.6 What an Investor Would Want to Know . . . 150
43.6.1 The Opportunity . . . 150
43.6.2 The Moat . . . 150
43.6.3 The Ask . . . 150
43.6.4 Why NOW . . . 150
43.7 Ethical Considerations . . . 150
44 Why Kotlin and GraalVM (and Why Not Other Languages) . . . 151
44.1 Why Kotlin Was the Right Choice . . . 151
44.1.1 Kotlin's Strengths for UTL-X . . . 151
44.1.2 Kotlin vs Alternatives Considered . . . 151
44.2 Why GraalVM on Top of Kotlin . . . 151
44.2.1 GraalVM Native Image . . . 151
44.2.2 GraalVM for Cloud (UTLXe) . . . 151
44.3 Why NOT a JavaScript/TypeScript Version . . . 151
44.3.1 The Temptation . . . 151
44.3.2 Why It Would Fail . . . 151
44.4 Why NOT WebAssembly (WASM) . . . 151
44.4.1 The Promise . . . 151
44.4.2 Why It Doesn't Work (Today) . . . 151
44.5 Why Multiple Language Implementations Lead to Deviation . . . 151

44.5.1 The N-Implementation Problem ... 151
44.5.2 The Wrapper Pattern Avoids Deviation ... 151
45 Case Studies and Real-World Use Cases ... 153
45.1 Case Study 1: European E-Invoicing (Peppol / UBL) ... 153
45.2 Case Study 2: Healthcare HL7 FHIR Integration ... 153
45.3 Case Study 3: Financial Services — SWIFT ISO 20022 Migration ... 153
45.4 Case Study 4: Retail — Multi-Channel Order Processing ... 153
45.5 Case Study 5: IoT / Manufacturing — Sensor Data Normalization ... 153
45.6 Case Study 6: Government — Tax Filing Integration ... 153
45.7 Azure Marketplace: Getting Started Guide ... 153
45.8 Recipes: Common Transformation Patterns ... 153
46 Grammar Reference ... 155
46.1 UTLX 1.0 Grammar (EBNF) ... 155
46.2 Header Syntax ... 155
46.3 Output Options ... 155
46.4 Expression Syntax ... 155
46.5 Reserved Words ... 155
46.6 Identifier Rules ... 155
47 Appendices ... 156
47.1 Appendix A: Format Conversion Matrix ... 156
47.2 Appendix B: Error Messages Reference ... 156
47.3 Appendix C: Comparison Tables ... 156
47.4 Appendix D: Industry Standards Reference ... 156
47.5 Appendix E: Glossary ... 156
47.6 Appendix F: Version History ... 156
47.7 Appendix G: License ... 156
48 Standard Library Reference ... 158
48.1 How to Read This Reference ... 158
48.2 Core Functions (49) ... 158
48.3 String Functions (83) ... 158
48.4 Array Functions (67) ... 158
48.5 Math Functions (37) ... 158
48.6 Date/Time Functions (68) ... 158
48.7 Type Functions (27) ... 158
48.8 Object Functions (1) ... 158
48.9 Encoding Functions (30) ... 158
48.10 XML Functions (60) ... 158
48.11 Security Functions (16) ... 158
48.12 JSON Functions (6) ... 158
48.13 Binary Functions (47) ... 158
48.14 Utility Functions (27) ... 158
48.15 Financial Functions (16) ... 158
48.16 Geospatial Functions (8) ... 158
48.17 CSV Functions (12) ... 158
48.18 YAML Functions (22) ... 158

Preface

0.1 Why This Book

0.2 Who This Book Is For

0.3 How to Read This Book

0.4 Conventions Used

0.5 Online Resources

Part I

Foundation

Understanding the language, the data model, and the philosophy

1 Introduction

Every enterprise integration project faces the same fundamental problem: systems don't speak the same language. Your ERP sends XML. Your REST API expects JSON. Your legacy batch process reads CSV. Your DevOps pipeline is configured in YAML. And somewhere, an OData service from Microsoft Dynamics 365 is waiting for its own peculiar JSON dialect.

Traditionally, you'd learn a different tool for each format. XSLT for XML. Liquid templates for JSON. Custom Python scripts for CSV. And for YAML? Good luck — most tools pretend it doesn't exist.

UTL-X changes this. One language. All formats. Write your transformation logic once, and it works whether your input is XML, JSON, CSV, YAML, or OData.

1.1 The Data Transformation Problem

Consider a typical integration scenario. A Dutch wholesale distributor receives purchase orders from 12 European partners. Partner A sends XML via SOAP. Partner B posts JSON to a REST API. Partner C drops CSV files on an SFTP server. Partner D sends YAML webhooks from their Kubernetes-based platform.

Without a format-agnostic tool, this distributor needs:

- An XSLT stylesheet for Partner A's XML
- Custom C# code for Partner B's JSON
- A Python script for Partner C's CSV
- Yet another script for Partner D's YAML

Four languages. Four maintenance burdens. Four sets of bugs. And when a new partner appears with OData from SAP? A fifth.

With UTL-X, all four partners are handled by the same transformation language. The `.utlx` file doesn't care whether the input is XML or CSV — it operates on the *Universal Data Model* (UDM), an internal representation that normalizes all formats into one tree structure.

1.2 What Is UTL-X?

UTL-X — *Universal Transformation Language Extended* — is a format-agnostic, functional transformation language for data integration. It is:

- **Format-agnostic**: one transformation works with XML, JSON, CSV, YAML, and OData
- **Functional**: declarative, composable, testable — no mutable state, no side effects
- **Open source**: licensed under AGPL-3.0, free for any use (see Chapter 2)
- **Production-ready**: 86,000+ messages per second, schema validation, pipeline chaining
- **Batteries included**: 652 built-in functions across 18 categories

At its core, UTL-X answers a simple question: *"Given this input data, produce that output data."* Everything else — format parsing, type coercion, attribute handling, namespace resolution — is handled automatically.

1.3 The Swiss Army Knife Analogy

UTL-X is often compared to a Swiss Army knife for data. The analogy is apt:

- Each **blade** is a format: XML, JSON, CSV, YAML, OData

- The **handle** is the UDM — the common core that connects all blades
- The **tools** are the 652 standard library functions: string manipulation, date arithmetic, array operations, cryptographic hashing, and more
- You carry **one tool** instead of five — and it handles any situation

1.4 UTL-X vs The Alternatives

How does UTL-X compare to existing transformation tools?

Capability	XSLT	DataWeave	jq	JSONata	UTL-X
JSON native	No	Yes	Yes	Yes	Yes
XML native	Yes	Yes	No	No	Yes
CSV native	No	Yes	No	No	Yes
YAML native	No	Yes	No	No	Yes
OData native	No	No	No	No	Yes
Schema validation	XSD	Limited	No	No	7 formats
Open source	Standard	No	Yes	Yes	Yes (AGPL)
Production engine	Yes	Yes	No	No	Yes
Functions	50	200	50	30	**652**
Cloud Marketplace	No	MuleSoft	No	No	**Azure/GCP/AWS**

XSLT is the 25-year veteran — powerful for XML, but XML-only and verbose. DataWeave (MuleSoft) is the closest commercial competitor — format-agnostic and functional, but proprietary and expensive ($1,250+/month). jq is the beloved command-line JSON processor — fast and elegant, but JSON-only. JSONata is a compact JSON query language — clean syntax, but limited scope.

UTL-X combines the breadth of DataWeave with the openness of jq, adds production deployment capabilities, and extends to formats that no other tool covers natively.

1.5 Architecture Overview

UTL-X's architecture is built around one central idea: *all data is the same, regardless of format.*

The pipeline works in three stages:

1. **Parse**: The input — whether XML, JSON, CSV, YAML, or OData — is parsed into the Universal Data Model (UDM). The UDM is a tree of typed nodes: scalars (strings, numbers, booleans), objects (with properties and optionally XML attributes), and arrays.

2. **Transform**: The `.utlx` transformation operates on the UDM tree. It can access any property, iterate arrays, apply functions, and construct new structures. The transformation is format-agnostic — it doesn't know or care whether the data came from XML or JSON.

3. **Serialize**: The resulting UDM tree is serialized to the target format. XML gets elements and attributes. JSON gets objects and arrays. CSV gets rows and columns. Each serializer handles format-specific concerns (XML namespaces, JSON types, CSV delimiters) transparently.

This three-stage architecture is what makes format-agnosticism possible. The transformation logic never touches raw XML tags or JSON braces — it works with the *meaning* of the data, not its *syntax*.

1.6 The UTL-X Ecosystem

UTL-X is not a single tool — it's an ecosystem of three executables, each designed for a different stage of the development lifecycle:

Executable	Purpose	When to use
`utlx`	CLI — command-line tool	Development, scripting, CI/CD
`utlxd`	Daemon — IDE integration	VS Code extension, live preview
`utlxe`	Engine — production runtime	Cloud deployment, Azure Marketplace

All three share the same parser, interpreter, UDM, and standard library. A transformation written in VS Code (via `utlxd`) runs identically on the command line (via `utlx`) and in production (via `utlxe`). Chapter 5 explains each executable in detail.

1.7 A First Example

Let's see UTL-X in action. Given this XML order:

```
<Order id="ORD-001">
  <Customer>Alice Johnson</Customer>
  <Items>
    <Item sku="WIDGET-01" price="50.00" qty="2"/>
    <Item sku="GADGET-02" price="75.00" qty="1"/>
  </Items>
</Order>
```

We want to produce this JSON:

```
{
  "orderId": "ORD-001",
  "customer": "Alice Johnson",
  "total": 175.00
}
```

Here is the UTL-X transformation:

```
%utlx 1.0
input xml
output json
---
{
  orderId: $input.Order.@id,
  customer: $input.Order.Customer,
  total: sum(map($input.Order.Items.Item,
    (item) -> toNumber(item.@price) * toNumber(item.@qty)
```

```
  ))
}
```

Let's break it down:

- `%utlx 1.0` — declares this is a UTL-X version 1.0 transformation
- `input xml` — the input data is XML
- `output json` — the output should be JSON
- `---` — separates the header from the body
- `\$input.Order.\@id` — accesses the `id` attribute on the `Order` element
- `\$input.Order.Customer` — accesses the text content of `Customer`
- `map(...)` — iterates over each `Item`, computing price times quantity
- `sum(...)` — adds up all the line totals

Run it:

```
cat order.xml | utlx transform order-to-json.utlx
```

Output:

```
{
  "orderId": "ORD-001",
  "customer": "Alice Johnson",
  "total": 175.0
}
```

That's UTL-X. One transformation file. Input XML, output JSON. All the format details — XML attributes, element text, JSON types — handled automatically.

Now change `output json` to `output yaml`, and the same transformation produces:

```
orderId: ORD-001
customer: Alice Johnson
total: 175.0
```

Or `output csv`:

```
orderId,customer,total
ORD-001,Alice Johnson,175.0
```

The transformation logic didn't change. Only the output declaration. That is the power of format-agnosticism.

1.8 What's Ahead

This book is organized in eight parts:

- **Part I** (you are here) lays the foundation: installation, the three executables, the UDM, schema mapping, and format support

- **Part II** teaches the language: expressions, functions, pattern matching, validation, and pipelines
- **Part III** dives deep into each format: XML, JSON, CSV, YAML, OData, and schema formats
- **Part IV** covers real-world applications: enterprise integration, cloud deployment, SDKs, performance, and compliance
- **Part V** looks ahead: semantic validation, API contracts, AI-assisted generation, and why Kotlin was the right choice
- **Part VI** presents case studies from e-invoicing, healthcare, financial services, and more
- **Part VII** provides reference material: grammar, appendices, and glossary
- **Part VIII** is the complete standard library encyclopedia: all 652 functions with signatures and examples

Whether you're an integration developer replacing XSLT, a data engineer building multi-format pipelines, or an architect evaluating transformation technologies — this book will take you from first transformation to production deployment.

Let's get started.

2 Licensing: Open Source and Commercial

Before diving into the technical details of UTL-X, it's important to understand what you can and cannot do with it. Licensing matters — especially in enterprise environments where legal and compliance teams review every dependency.

The short version: UTL-X is free and open source for almost every use case. The only exception is embedding it inside a proprietary product you distribute — and even then, a commercial license is available.

2.1 UTL-X Is Open Source (AGPL-3.0)

UTL-X is licensed under the **GNU Affero General Public License v3.0** (AGPL-3.0). This is one of the strongest open-source licenses, designed to keep software free and open — even when deployed as a cloud service.

If you've worked with GPL-licensed software before, AGPL is similar with one important addition: the *network use* clause. Under regular GPL, sharing obligations only trigger when you *distribute* the software. Under AGPL, they also trigger when you make the software available over a *network* — for example, as a SaaS offering.

This matters for UTL-X because transformation engines are typically deployed as services, not distributed as installable software.

2.2 What You Can Do (Without Paying Anything)

Under the AGPL-3.0 license, you are free to:

- **Use UTL-X for any purpose** — personal, educational, or commercial
- **Run UTL-X internally** within your organization, on any number of servers
- **Modify the source code** to fit your needs
- **Deploy UTL-X as a service** within your organization
- **Write any number of `.utlx` transformations** — they are yours (more on this below)

There is no usage limit, no trial period, no feature restriction. The full UTL-X engine with all 652 standard library functions, all 11 format parsers, and all execution strategies is available under AGPL-3.0.

2.3 The Key AGPL Condition

If you **modify** UTL-X and make your modified version available to users over a network (as a web service, API, or cloud service), you must make your modified source code available to those users under the same AGPL-3.0 license.

This is the *quid pro quo* of AGPL: you benefit from the open-source community's work, and in return, your improvements flow back to the community.

Note the emphasis on *modified*. If you use UTL-X as-is (without changing the engine source code), there is nothing to share — the original source is already public on GitHub.

2.4 Your Transformations Are Yours

This is the most important clarification in this chapter:

The AGPL license covers the UTL-X software — the parser, interpreter, engine, and standard library. It does NOT cover the transformations you write or the data you process.

Your `.utlx` files are *input* to the program, not part of the program. They are your intellectual property. You can keep them proprietary, sell them, or do anything you want with them.

The analogy is precise: GCC (the GNU C compiler) is licensed under GPL. But the C programs you compile with GCC are *yours* — GCC's license does not "infect" your code. Similarly, UTL-X's AGPL license does not affect your `.utlx` transformation files.

This means:

- Your transformation logic is private
- Your schemas are private
- Your pipeline configurations are private
- Your test data is private
- Only the UTL-X engine itself is AGPL

2.5 Usage Scenarios

The following table covers every common scenario:

Scenario	License	Cost	Why
Personal use / learning	AGPL (free)	Free	Any use is permitted
Internal company use (unmodified)	AGPL (free)	Free	No distribution, no network clause
Internal company use (modified)	AGPL (free)	Free	Internal use is not distribution
Open-source project using UTL-X	AGPL (free)	Free	Your project must be AGPL-compatible
Consulting (writing .utlx for clients)	AGPL (free)	Free	.utlx files are data, not software
Azure Marketplace deployment	Commercial	$35–105/mo	Commercial license included
GCP / AWS Marketplace	Commercial	$44–131/mo	Commercial license included
Embedding in proprietary SaaS	Commercial	Contact	AGPL requires source sharing
OEM / white-label distribution	Commercial	Contact	Distributing modified AGPL code

The vast majority of users fall into the first five rows — all free, no obligations.

2.6 The Subprocess Exception

A technical but important detail: if your application communicates with UTL-X via **HTTP API**, **subprocess** (stdio-proto), or **Dapr sidecar**, there is no *linking* between your code and UTL-X. Your application's license is not affected by AGPL.

The C# and Go wrappers use subprocess communication (protobuf over stdin/stdout). They spawn UTL-X as a separate process. This is not linking — it's inter-process communication. Your C#, Go, or Python application remains under whatever license you choose.

This is the same principle that allows proprietary applications to call GPL-licensed command-line tools (like `grep` or `git`) without becoming GPL themselves.

2.7 The Commercial License

2.7.1 Why It Exists

Some organizations have policies that prohibit AGPL dependencies — regardless of how they're used. Others want to embed UTL-X inside a proprietary product without source-sharing obligations. The commercial license serves these cases.

2.7.2 Cloud Marketplace (Simplest Path)

When you deploy UTL-X from the Azure Marketplace (or GCP/AWS when available), a commercial license is included in the subscription. No AGPL obligations apply. The licensing is handled by Microsoft/Google/Amazon through the Marketplace billing.

- **Azure**: Starter $35/month, Professional $105/month
- **GCP**: Starter $44/month, Professional $131/month
- **AWS**: Starter $44/month, Professional $131/month

2.7.3 Direct Commercial License

For companies embedding UTL-X in proprietary products, OEM distribution, or organizations with AGPL-incompatible compliance policies, a direct commercial license is available. Pricing is per-deployment or per-organization, negotiated based on use case.

Contact: licensing@utlx-lang.org

2.8 Dual Licensing: How It Works

UTL-X uses a *dual licensing* model — the same source code is available under two licenses. You choose which one applies to your use:

License A: AGPL-3.0 (free)	License B: Commercial (paid)
Use, modify, deploy freely	No source-sharing obligation
Share modifications if network service	Embed in proprietary products
Your .utlx files are NOT affected	Azure/GCP/AWS Marketplace includes this
Community-driven improvements	Direct support available

Most users never need the commercial license. The AGPL is free, permissive for internal use, and doesn't touch your transformation files. The commercial license exists as a safety net for the minority of use cases where AGPL is a blocker.

2.9 Frequently Asked Questions

"Can I use UTL-X in my company for free?"

Yes. Internal use is always free under AGPL. You can run UTL-X on your servers, write any transformations, process any data. No payment needed. No source-sharing needed — internal use is not distribution.

"Do I need to open-source my .utlx files?"

No. Your transformation files are data, not part of the UTL-X software. AGPL does not apply to them. They are your intellectual property.

"Can I use UTL-X in my SaaS product?"

If you use UTL-X *unmodified*: yes, under AGPL — but you must make clear to users that UTL-X (AGPL) is used, and provide access to its source code. If you *modify* UTL-X: you must share your modifications under AGPL. If you want no obligations: buy a commercial license.

"What if I deploy from Azure Marketplace?"

Commercial license is included. No AGPL obligations. You pay $35–105/month through your Azure bill.

"Does AGPL affect my entire application?"

Only if you *link* or *embed* UTL-X directly in your application code (e.g., importing UTL-X as a Java library). If you communicate via HTTP, subprocess, or Dapr — your application is not affected. The Go and C# wrappers use subprocess communication, specifically to avoid this.

With licensing clarified, let's move on to what matters most: using UTL-X to transform data.

3 Transformation: The Cornerstone of Integration

Before we install UTL-X and write our first transformation, it's worth understanding *why* data transformation exists — and why it's the single most important capability in enterprise integration.

If you're already convinced and eager to write code, skip to Chapter 4. But if you're an architect evaluating UTL-X, or a developer wondering why a dedicated transformation language matters when you could just write Python, this chapter is for you.

3.1 The Three Pillars of Integration

Every integration platform — whether it's a $500K enterprise service bus or a shell script connecting two APIs — rests on three pillars:

1. **Messaging**: getting data from point A to point B. This is the transport layer — message queues (Service Bus, SQS, Kafka), HTTP APIs, file transfers (SFTP), event streams (Event Hub, Pub/Sub). Messaging answers: *"How does data travel?"*
2. **Transformation**: converting data from the format and structure that system A produces into the format and structure that system B expects. Transformation answers: *"How does data change shape?"*
3. **Routing and Orchestration**: deciding *where* to send data and *when*, handling sequences, parallel flows, error recovery, and retry logic. Routing answers: *"Where does data go, and in what order?"*

Traditional Enterprise Service Buses (ESBs) like Tibco BusinessWorks, MuleSoft, and IBM Integration Bus bundle all three pillars into one platform. This is convenient but expensive — you pay for the entire platform even if you only need better transformation.

UTL-X takes a different approach: it focuses exclusively on pillar two — transformation — and does it better than any integrated platform. Messaging is handled by dedicated brokers (Azure Service Bus, Kafka, GCP Pub/Sub). Routing is handled by orchestrators (Logic Apps, Open-M, Step Functions). Each component does one thing well.

3.2 Why Transformation Is the Hardest Pillar

Messaging is largely a solved problem. Whether you choose Kafka, Service Bus, or RabbitMQ, they all reliably move bytes from A to B. The differences are in guarantees (at-least-once vs exactly-once), throughput, and pricing — but the fundamental problem is solved.

Routing is complex but well-understood. BPMN, state machines, and workflow engines have decades of theory and practice behind them.

Transformation, however, is where most integration projects spend 60-80% of their effort. Why?

- **Every system has its own data model.** SAP uses IDoc segments. Dynamics 365 uses OData entities. Your partner's API uses a custom JSON schema. No two systems agree on field names, types, or structure.
- **Formats multiply the problem.** It's not just different fields — it's different *serialization formats*. XML with namespaces and attributes. JSON with nested arrays. CSV with regional number formatting. YAML with anchors and aliases. Each format has its own parsing rules, edge cases, and gotchas.

- **Business logic hides in the mapping.** A transformation isn't just moving field A to field B. It's: "If the customer is in the EU, apply reverse-charge VAT. If the quantity exceeds 100, use the bulk price. If the currency is not EUR, convert using today's exchange rate." This logic is specific to your business — no generic tool can auto-generate it.
- **Transformations change constantly.** Every API update, every new partner, every regulatory change (e-invoicing deadlines, FHIR mandates, SWIFT MX migration) requires transformation updates. The mapping is the most frequently modified component in any integration.

3.3 Transformation in the Enterprise Service Bus

In a traditional ESB architecture, transformation is embedded in the middleware:

Platform	Transformation tool	Underlying technology
Tibco BusinessWorks	Mapper activity	XSLT (generated)
SAP CPI	Message Mapping	XSLT / Groovy scripts
Azure Logic Apps	Data Mapper	XSLT (JSON→XML internally)
MuleSoft	DataWeave	Format-agnostic (like UTL-X)
IBM Integration Bus	Graphical mapper	ESQL / Java / XSLT
Boomi	Map component	Visual mapping (proprietary)

Notice a pattern? Four out of six platforms use XSLT under the hood — even for JSON data. This forces every message through an XML conversion step, regardless of the original format. MuleSoft's DataWeave is the exception — and it's the most direct competitor to UTL-X's approach.

3.4 The JSON-to-XML-to-XSLT Anti-Pattern

When JSON became the dominant API format in the 2010s, most integration platforms had a problem: their transformation engines only understood XML. Rather than rebuilding their engines, they added a conversion layer:

The message takes four conversion steps:

1. JSON input is converted to an XML representation
2. The XSLT transformation processes the XML
3. The XSLT output is XML
4. The XML is converted back to JSON

This is wrong for several reasons:

Performance: four serialization/deserialization steps instead of one. Each step allocates memory, parses text, and builds a tree. For a 100 KB JSON message, this adds 10-20ms of pure overhead — before any transformation logic runs.

Data loss: JSON and XML are not equivalent. JSON has typed values (numbers, booleans, null). XML has only strings. A JSON number 42 becomes XML text, and when converted back, the type information may be lost — is "42" a number or a string?

Attribute mismatch: JSON has no concept of XML attributes. Conventions like using @id as a property name are workarounds, not standards. Different tools handle it differently. Round-trip fidelity is not guaranteed.

Developer experience: The developer must think in XML even when the data is JSON. They write XSLT templates referencing "Order/Customer" for data that's actually a JSON object. The mental model and the actual data diverge.

Azure Logic Apps' Data Mapper does exactly this — internally converting JSON to an XML "infoset" before applying XSLT transformations. SAP CPI's Message Mapping operates on an XML DOM. Tibco BW uses XML throughout, with JSON adapters at the boundaries.

3.5 The UTL-X Approach: No Intermediate Conversion

UTL-X doesn't convert between formats before transforming. It parses the input *once* into the Universal Data Model, transforms it, and serializes the output *once* to the target format:

One parse. One transform. One serialize. No intermediate XML. No lost types. No attribute confusion.

And because the UDM is format-agnostic, the *same transformation* works regardless of input format. Change the input declaration from "input json" to "input xml", and the transformation expression stays the same — $input.Order.Customer accesses the customer whether the data came from JSON or XML.

3.6 Why UTLXe Needs Messaging Capabilities

If UTL-X only does transformation (pillar two), why does the production engine (UTLXe) integrate with message brokers?

Because in modern event-driven architectures, transformation doesn't happen in isolation. Messages arrive from brokers, need to be transformed, and the results need to go back to brokers:

Without messaging integration, UTLXe would be a passive HTTP endpoint — the calling application would need to:

1. Pull the message from the broker
2. HTTP POST it to UTLXe
3. Receive the response
4. Push the result to the output broker

That's four steps of orchestration just to transform a message. With Dapr integration, UTLXe listens directly on the broker and publishes results back — zero orchestration code needed. The transformation is a *first-class participant* in the event-driven flow, not an afterthought called via HTTP.

This is the same reason MuleSoft's Anypoint, Tibco's BWCE, and SAP CPI bundle messaging with transformation — even though they're conceptually separate pillars.

3.7 The Canonical Data Model

One of the most powerful integration patterns is the *Canonical Data Model* — and UTL-X is ideally suited for it.

The idea: instead of building point-to-point transformations between every pair of systems (N × (N-1) transformations for N systems), define a single internal format — the "canonical model" — and build only N inbound + N outbound transformations:

Without a canonical model (point-to-point):
- 6 systems = 30 transformations (each pair needs both directions)
- Adding system 7 = 12 new transformations

With a canonical model:
- 6 systems = 12 transformations (6 inbound + 6 outbound)
- Adding system 7 = 2 new transformations

UTL-X makes canonical models practical because:

- The canonical model can be in *any format* — JSON is most common, but XML or YAML work too
- Schema validation ensures all inbound transformations produce valid canonical output
- The same .utlx syntax works for every inbound and outbound transformation
- New systems are added by writing one new transformation file, not by modifying existing ones

3.8 What This Means for You

If you're evaluating UTL-X, here's the bottom line:

- **If you're replacing XSLT**: UTL-X handles XML natively, plus JSON, CSV, YAML, and OData — all in one language, with modern functional syntax instead of XML-based XSLT
- **If you're replacing custom code**: UTL-X is declarative, testable, and hot-reloadable — no redeployment for mapping changes
- **If you're building a new integration**: UTL-X is the transformation layer — pair it with your choice of message broker and orchestrator
- **If you're an architect**: UTL-X enables canonical data models with format-agnostic transformations and schema-driven development

With this context in mind, let's install UTL-X and start transforming data.

4 Getting Started

This chapter gets UTL-X running on your machine in under five minutes. By the end, you'll have installed the CLI, run your first transformation, and understood the structure of a `.utlx` file.

4.1 Installation

UTL-X is distributed as a native binary — a single file, no JVM required, instant startup. Choose the method that fits your platform.

4.1.1 macOS (Homebrew) — Recommended

```
brew tap grauwen/utlx
brew install utlx
```

Verify:

```
utlx --version
# UTL-X CLI v1.0.2
# Universal Transformation Language Extended
```

Homebrew installs the GraalVM native binary. Startup time is under 10 milliseconds — as fast as `grep` or `jq`.

To update later:

```
brew update && brew upgrade utlx
```

4.1.2 Windows (Chocolatey)

```
choco install utlx
```

Verify:

```
utlx --version
```

If Chocolatey is not available, download the binary directly (see below).

4.1.3 Linux / macOS / Windows — Direct Download

Native binaries are available from GitHub Releases for every platform:

```
# macOS (Apple Silicon)
curl -L https://github.com/grauwen/utl-x/releases/download/v1.0.2/utlx-macos-arm
64 -o utlx
chmod +x utlx
sudo mv utlx /usr/local/bin/

# Linux (x64)
curl -L https://github.com/grauwen/utl-x/releases/download/v1.0.2/utlx-linux-x64
-o utlx
```

```
chmod +x utlx
sudo mv utlx /usr/local/bin/

# Windows: download utlx-windows-x64.exe from
# https://github.com/grauwen/utl-x/releases/tag/v1.0.2
# and place in a directory on your PATH
```

4.1.4 Building from Source

If you prefer to build UTL-X yourself, or need the JVM version for development:

Prerequisites:

- JDK 17 or later (`java -version` to check)
- Git

Steps:

```
# Clone the repository
git clone https://github.com/grauwen/utl-x.git
cd utl-x

# Build the CLI JAR
./gradlew :modules:cli:jar

# Run using the wrapper script
./utlx --version
```

The wrapper script (`utlx`) automatically finds and runs the compiled JAR. On Windows, use `utlx.bat` (Command Prompt) or `utlx.ps1` (PowerShell).

Building the native binary (requires GraalVM):

```
# Install GraalVM (macOS)
brew install --cask graalvm/tap/graalvm-community-jdk22

# Build native image
./gradlew :modules:cli:nativeCompile

# Binary at: modules/cli/build/native/nativeCompile/utlx
```

The native binary starts in under 10ms and uses 40MB of memory, compared to 250ms startup and 150MB for the JVM JAR. For development, the JAR is fine. For scripting and CI/CD, use the native binary.

4.1.5 Native Binary vs JVM JAR

Aspect	Native binary (GraalVM)	JVM JAR
Startup time	< 10ms	250ms
Memory	40 MB	150 MB
File size	80 MB (single file)	38 MB JAR + JVM

Requires JVM?	No	Yes (JDK 17+)
Install method	brew, choco, download	Build from source
Best for	CLI use, scripting, CI/CD	Development, debugging

When in doubt, use the native binary. It's what `brew install utlx` gives you.

4.2 Your First Transformation

Create a file called `hello.utlx`:

```
%utlx 1.0
input json
output json
---
{
  greeting: concat("Hello, ", $input.name, "!"),
  reversed: reverse($input.name),
  length: length($input.name)
}
```

Run it:

```
echo '{"name": "World"}' | utlx transform hello.utlx
```

Output:

```
{
  "greeting": "Hello, World!",
  "reversed": "dlroW",
  "length": 5
}
```

Congratulations — you've just run your first UTL-X transformation.

4.3 The .utlx File Structure

Every UTL-X transformation file has the same structure:

```
┌──────────────────────────────────┐
│ %utlx 1.0                        │ ← Version declaration
│ input json                       │ ← Input format
│ output xml                       │ ← Output format (+ options)
│ ---                              │ ← Separator
│ {                                │
│   name: $input.customer.name,    │ ← Transformation body
│   total: sum($input.items.price)│
│ }                                │
└──────────────────────────────────┘
```

The **header** (above `---`) declares:

- `%utlx 1.0` — the UTL-X language version (currently always 1.0)
- `input <format>` — what format the input data is in (xml, json, csv, yaml, odata)
- `output <format>` — what format to produce, with optional settings

The **body** (below `---`) is the transformation expression. It's a single expression that produces the output. No `return` keyword — the entire body IS the result.

4.3.1 Input Format Options

```
input json                          // standard JSON
input xml                           // XML (auto-detects encoding)
input csv                           // CSV (comma-delimited, with headers)
input csv {delimiter: ";"}          // semicolon-delimited
input yaml                          // YAML
input odata                         // OData JSON
```

4.3.2 Output Format Options

```
output json                                // pretty-printed JSON
output json {writeAttributes: true}        // preserve XML attributes
output xml {encoding: "UTF-8"}             // XML with encoding declaration
output csv {delimiter: ";", headers: true, regionalFormat: "european"}
output yaml                                // YAML block style
```

4.3.3 Multi-Input

When your transformation needs data from multiple sources:

```
%utlx 1.0
input: orders xml, customers json
output json
---
{
  orderCount: count($orders.Order),
  customerName: $customers.name
}
```

Each input gets its own variable name (prefixed with `$`).

4.4 Trying It Out: Five Quick Examples

Here are five transformations you can run right now to see UTL-X in action:

1. XML to JSON — the most common integration task:

```
echo '<User><Name>Alice</Name><Age>30</Age></User>' | utlx --from xml --to json
```

2. JSON to YAML — configuration format conversion:

```
echo '{"server": "prod-01", "port": 8080, "debug": false}' | utlx --from json --
to yaml
```

3. Extract a field — like jq but for any format:

```
echo '{"users": [{"name": "Alice"}, {"name": "Bob"}]}' | utlx -e
'map($input.users, (u) -> u.name)'
```

4. CSV to JSON — data import:

```
echo 'name,age\nAlice,30\nBob,25' | utlx --from csv --to json
```

5. XML attribute access — the `@` syntax:

```
echo '<Product id="P-001" price="29.99">Widget</Product>' | utlx --from xml -e
'{id: $input.Product.@id, name: $input.Product, price:
toNumber($input.Product.@price)}'
```

Each of these runs in under 10 milliseconds on a native binary. That's the UTL-X developer experience: instant feedback, any format.

4.5 What's Next

You now have UTL-X installed and running. The next chapters cover:

- **Chapter 5**: UTL-X on the command line — expression mode, identity flip, shell scripting
- **Chapter 6**: The three executables — CLI, daemon, and production engine
- **Chapter 7**: The IDE — VS Code extension, live preview, UDM tree browser

5 UTL-X on the Command Line

UTL-X was designed to feel at home in a terminal. If you've used `jq` for JSON processing, UTL-X will feel familiar — but it works with XML, CSV, YAML, and OData too. This chapter covers the three CLI modes, shell scripting patterns, and when to use each.

5.1 Three Ways to Use the CLI

Mode	Syntax	Best for
Transform	`utlx transform script.utlx input.xml`	Complex transformations with a .utlx file
Expression (`-e`)	`echo data | utlx -e '.name'`	Quick one-liners, field extraction, piping
Identity (flip)	`cat data.xml | utlx`	Instant format conversion, no script needed

5.2 Expression Mode: The jq Experience

Expression mode (`-e`) is the most powerful way to use UTL-X interactively. Instead of writing a `.utlx` file, you pass the transformation as a command-line argument:

```
echo '{"name": "Alice", "age": 30}' | utlx -e '.name'
# "Alice"
```

The dot (`.`) is shorthand for `$input`. So `.name` means `$input.name`. This mirrors jq's syntax — deliberately.

5.2.1 Raw Output (-r)

By default, string output includes quotes. Use `-r` (raw) to strip them — essential for piping to other commands:

```
# With quotes (default)
echo '{"name": "Alice"}' | utlx -e '.name'
# "Alice"

# Raw (no quotes) — pipe-friendly
echo '{"name": "Alice"}' | utlx -e '.name' -r
# Alice

# Use in shell variables
NAME=$(echo '{"name": "Alice"}' | utlx -e '.name' -r)
echo "Hello, $NAME"
# Hello, Alice
```

5.2.2 Expressions with Any Format

Unlike jq, expression mode works with XML, CSV, and YAML too:

```
# Extract from XML
echo '<User><Name>Alice</Name></User>' | utlx --from xml -e '.User.Name' -r
```

```
# Alice

# Extract from XML attribute
echo '<Product id="P-001"/>' | utlx --from xml -e '.Product.@id' -r
# P-001

# Extract from YAML
echo 'server: prod-01\nport: 8080' | utlx --from yaml -e '.port' -r
# 8080

# Extract from CSV (first row)
echo 'name,age\nAlice,30\nBob,25' | utlx --from csv -e '.[0].name' -r
# Alice
```

5.2.3 Complex Expressions

Expressions can be as simple or complex as needed:

```
# Object construction
echo '{"first": "Alice", "last": "Johnson"}' | utlx -e '{fullName: concat(.first,
" ", .last)}'
# {"fullName": "Alice Johnson"}

# Array operations
echo '[3, 1, 4, 1, 5, 9]' | utlx -e 'sort($input)'
# [1, 1, 3, 4, 5, 9]

# Filter and map
echo '{"users": [{"name": "Alice", "active": true}, {"name": "Bob", "active":
false}]}' \
  | utlx -e 'map(filter(.users, (u) -> u.active), (u) -> u.name)'
# ["Alice"]

# Aggregation
echo '[{"amount": 10}, {"amount": 20}, {"amount": 30}]' | utlx -e
'sum(map($input, (x) -> x.amount))'
# 60
```

5.3 Identity Mode: The Flip

The simplest way to use UTL-X — pipe data in, get a different format out. No script, no expression, no arguments:

```
# XML → JSON (automatic)
cat order.xml | utlx
# {"Order": {"Customer": "Alice", "Total": 299.99}}

# JSON → XML (automatic)
echo '{"name": "Alice"}' | utlx
# <?xml version="1.0" encoding="UTF-8"?><name>Alice</name>
```

UTL-X auto-detects the input format and “flips” to the most useful output format:

Input detected as	Output format (default flip)
XML	JSON
JSON	XML
CSV	JSON
YAML	JSON

Override the output format with `--to`:

```
# XML → YAML (instead of default JSON)
cat order.xml | utlx --to yaml

# JSON → CSV
echo '[{"name":"Alice","age":30},{"name":"Bob","age":25}]' | utlx --to csv

# CSV → YAML
cat report.csv | utlx --to yaml
```

Override the input format with `--from` (when auto-detection fails):

```
# Force input as CSV (e.g., when piped from another command)
some-command | utlx --from csv --to json
```

5.4 Shell Scripting Patterns

5.4.1 Batch File Conversion

Convert all XML files in a directory to JSON:

```
for f in *.xml; do
  utlx --from xml --to json < "$f" > "${f%.xml}.json"
done
```

5.4.2 Pipeline Integration

UTL-X is pipe-friendly — combine it with standard Unix tools:

```
# Extract customer names from XML orders, sort, deduplicate
cat orders.xml | utlx --from xml -e 'map(.Orders.Order, (o) -> o.Customer)' \
  | utlx -e 'sort($input)' \
  | utlx -e 'unique($input)'

# Count items in a JSON array
curl -s https://api.example.com/users | utlx -e 'count($input)' -r

# Filter CSV rows and convert to JSON
cat report.csv | utlx --from csv -e 'filter($input, (row) -> toNumber(row.amount)
> 100)' --to json
```

5.4.3 Using UTL-X in CI/CD

Validate a configuration file in a GitHub Actions workflow:

```
- name: Validate config
  run: |
    cat config.yaml | utlx --from yaml -e 'if (.version != null && .database !=
null) "valid" else "invalid"' -r
```

Transform API responses in deployment scripts:

```
# Extract image tag from Kubernetes deployment
kubectl get deployment myapp -o json | utlx -e
'.spec.template.spec.containers[0].image' -r
```

5.4.4 Combining with jq

UTL-X and jq can coexist. Use jq for JSON-only tasks where its syntax is more concise, and UTL-X when you need XML, CSV, or YAML:

```
# jq for simple JSON field extraction (shorter syntax)
echo '{"name": "Alice"}' | jq -r '.name'

# UTL-X when input is XML (jq can't do this)
echo '<User><Name>Alice</Name></User>' | utlx --from xml -e '.User.Name' -r

# Combine: UTL-X converts XML to JSON, jq processes further
cat complex.xml | utlx --from xml --to json | jq '.Orders[] | select(.total >
100)'
```

5.5 The UTL-X jq Cheat Sheet

For developers coming from jq, here's how common patterns translate:

Task	jq	UTL-X
Extract field	`jq '.name'`	`utlx -e '.name'`
Raw output	`jq -r '.name'`	`utlx -e '.name' -r`
Array length	`jq '. | length'`	`utlx -e 'count($input)'`
Map array	`jq '[.[] | .name]'`	`utlx -e 'map($input, (x) -> x.name)'`
Filter array	`jq '[.[] | select(.active)]'`	`utlx -e 'filter($input, (x) -> x.active)'`
Sort	`jq 'sort_by(.name)'`	`utlx -e 'sortBy($input, (x) -> x.name)'`
Keys	`jq 'keys'`	`utlx -e 'keys($input)'`
Sum	`jq '[.[].price] | add'`	`utlx -e 'sum(map($input, (x) -> x.price))'`
Compact	`jq -c '.'`	`utlx -e '$input' --no-pretty`

The key difference: every UTL-X command above also works with `--from xml`, `--from csv`, or `--from yaml`. jq is JSON-only.

5.6 When to Use Which Mode

Situation	Use
Quick field extraction	`utlx -e '.field' -r`
Format conversion (no logic)	`cat data.xml | utlx` (identity)
Format conversion (explicit target)	`cat data.xml | utlx --to yaml`
Complex transformation with reuse	`utlx transform script.utlx input.xml`
Shell pipeline	`some-cmd | utlx -e '...' -r | next-cmd`
CI/CD validation	`utlx -e 'if (...) "ok" else "fail"' -r`
Production service	Not CLI — use `utlxe` (see Chapter 6)

The CLI (`utlx`) is for humans and scripts. The engine (`utlxe`) is for production services. They share the same language — a transformation that works in `-e` mode works identically in `utlxe`.

5.7 Performance Notes

The native binary (GraalVM) is critical for CLI usage:

- **Startup**: < 10ms (native) vs 250ms (JVM JAR)
- **For single invocations**: startup time dominates — native is 25x faster end-to-end
- **For shell loops**: `for f in *.xml; do utlx ...; done` — native processes 100 files/second, JVM 4 files/second (startup penalty per invocation)
- **For pipes**: `cat data | utlx -e '...'` — native feels instant, JVM has noticeable delay

If you're using UTL-X in scripts or CI/CD, always use the native binary. The JVM JAR is for development only (when you need debugging or hot-reload).

6 The Three Executables: utlx, utlxd, and utlxe

UTL-X is one language, but it ships as three separate executables. Each is built for a different stage of the development and deployment lifecycle. Understanding which to use — and when — is essential before diving into the language itself.

6.1 One Language, Three Runtimes

All three executables share the same core:

- The same ANTLR-based parser (identical grammar, identical AST)
- The same interpreter (identical expression evaluation)
- The same Universal Data Model (identical type system)
- The same 652 standard library functions
- The same 11 format parsers and serializers
- The same 7 schema validators

A transformation that works in one executable works identically in all three. A bug fix in the shared core fixes all three simultaneously. This is by design — the conformance suite (453+ tests) validates behavior across executables.

The difference is in what surrounds the core: how input arrives, how output is delivered, and how the process is managed.

6.2 utlx — The CLI

The command-line interface is for developers. It's the tool you install on your laptop, use in shell scripts, and run in CI/CD pipelines.

Lifecycle: single invocation. You run a command, it processes data, it exits. No persistent process, no state between runs.

Distribution: native binary (GraalVM) via Homebrew, Chocolatey, or direct download. Also available as a JVM JAR for development.

Key characteristics:

Startup time	< 10ms (native), 250ms (JVM)
Memory	40 MB (native), 150 MB (JVM)
Workers	1 (single-threaded)
Protocol	stdin / stdout / files
Strategies	TEMPLATE only
Hot reload	No — each run is independent
Metrics	None
Best for	Development, scripting, CI/CD, one-off conversions

Modes:

- **Transform:** `utlx transform script.utlx input.xml` — run a .utlx file against input data
- **Expression:** `echo data | utlx -e '.name' -r` — inline one-liner, jq-style
- **Identity (flip):** `cat data.xml | utlx` — auto-detect format, convert to the complement
- **REPL:** `utlx repl` — interactive mode for experimentation

- **Validate:** `utlx validate script.utlx` — check syntax without running
- **Functions:** `utlx functions --search date` — browse the standard library

The CLI is covered in depth in Chapter 5.

6.3 utlxd — The Daemon

The daemon is the bridge between UTL-X and your IDE. It runs as a background process on your development machine, communicating with the VS Code extension via the Language Server Protocol (LSP).

Lifecycle: long-running. Starts when VS Code opens a .utlx file, stops when VS Code closes. Keeps the parsed state of your transformations in memory for instant feedback.

Distribution: JVM JAR only. The daemon needs full Kotlin reflection for language features like autocompletion and type inference — GraalVM native image doesn't support this yet.

Key characteristics:

Startup time	1 second
Memory	200 MB
Workers	1 (serves one IDE session)
Protocol	LSP over stdio
Strategies	TEMPLATE only
Hot reload	Yes — re-parses on every keystroke
Metrics	None
Best for	VS Code extension, IDE integration

Features provided to the IDE:

- **Real-time diagnostics:** syntax errors highlighted as you type — no need to save and run
- **Autocompletion:** function names (all 652), property paths (from parsed input), keywords
- **Hover information:** function signatures, parameter types, return types
- **Live preview:** transformation result updates as you type (with sample data)
- **Go-to-definition:** navigate to function definitions and variable bindings
- **Format document:** auto-indent and style your .utlx file

The daemon is not used in production — it's a developer tool. You never deploy utlxd to a server. The IDE chapter (Chapter 7) covers it in detail.

6.4 utlxe — The Engine

The engine is for production. It's a long-running transformation service that accepts messages via HTTP, gRPC, or message brokers, transforms them at high throughput, and returns results.

Lifecycle: long-running. Starts with the container, runs indefinitely. Transformations are loaded at startup or hot-reloaded via API.

Distribution: Docker image (`ghcr.io/utlx-lang/utlxe`). Available on Azure Marketplace as a managed application.

Key characteristics:

Startup time	3 seconds (JVM)
Memory	512 MB – 8 GB (configurable)
Workers	8–128 concurrent threads
Protocol	HTTP REST (8085), gRPC, stdio-proto, stdio-json
Strategies	TEMPLATE, COPY, COMPILED, AUTO
Hot reload	Yes — load/update/unload via HTTP API
Metrics	Prometheus (port 8081)
Best for	Cloud deployment, Azure/GCP/AWS Marketplace

Transport modes:

- `--mode http` — HTTP REST API on port 8085. The primary mode for cloud deployment. Endpoints: `/api/transform`, `/api/load`, `/api/execute/\{id\}`, `/api/health`
- `--mode grpc` — gRPC service for low-latency, binary protocol communication
- `--mode stdio-proto` — protobuf over stdin/stdout, used by the C# and Go wrappers
- `--mode stdio-json` — line-delimited JSON over stdin/stdout, for backward compatibility

Execution strategies:

The engine offers four strategies that trade initialization time for runtime performance:

Strategy	Init time	Runtime	Best for
TEMPLATE	Instant	Interprets AST	Development, simple transforms
COPY	Fast (build skeleton)	Clone + fill	Schema-driven, predictable structure
COMPILED	Slow first time	JVM bytecode	Maximum throughput, complex logic
AUTO	Depends	Depends	Production default (schema → COPY, else → TEMPLATE)

COMPILED strategy compiles UTL-X expressions to JVM bytecode using the ASM library — the same technology that Java itself uses. This achieves throughput of 86,000+ messages per second on a single instance with 8 workers. Chapter 30 covers the engine lifecycle (design-time, init-time, runtime) and Chapter 34 covers performance tuning.

Cloud deployment:

UTLXe is designed for containerized environments:

- Azure Container Apps — available on Azure Marketplace (Starter $35/month, Professional $105/month)
- GCP Cloud Run — Terraform module included, $44/month
- AWS ECS/Fargate — CloudFormation template, $44/month
- Any Kubernetes cluster — Docker image from `ghcr.io`

Chapter 31 covers cloud deployment in detail.

6.5 Comparison at a Glance

Aspect	utlx (CLI)	utlxd (Daemon)	utlxe (Engine)
Purpose	Development	IDE integration	Production

Lifecycle	Single run	Long-running	Long-running
Runtime	Native or JVM	JVM only	JVM (Docker)
Startup	< 10ms	1s	3s
Workers	1	1	8–128
Protocol	stdin/stdout	LSP	HTTP / gRPC / proto
Strategies	TEMPLATE	TEMPLATE	All four
Hot reload	No	Yes (LSP)	Yes (API)
Metrics	None	None	Prometheus
Distribution	brew / choco	VS Code ext	Docker / Marketplace
Branch	main	development	development

6.6 The Wrapper Pattern

What if your application is written in C#, Go, or Python — not Kotlin/JVM? You don't need to rewrite UTL-X. Instead, you use a *wrapper library* that spawns UTLXe as a subprocess and communicates via protobuf:

The wrapper is 200–300 lines of thin client code. The engine — 100,000+ lines of transformation logic — exists once. All languages get identical behavior, verified by the same 453+ conformance tests.

Available wrappers:

Language	**Status**	**Use case**
C# / .NET	Built	Azure Functions, ASP.NET, console apps
Go	Built	Open-M controller, Go microservices
Python	Planned	Data science, Django/Flask, AWS Lambda
Java / Kotlin	Native (no wrapper)	Direct API — UTLXe IS JVM

Chapter 32 covers SDKs and wrappers in detail.

6.7 When to Use Which

I want to...	**Use**
Write a transformation	utlxd (VS Code) + utlx (to test)
Convert a file quickly	utlx (identity mode or -e)
Process files in a shell script	utlx (native binary)
Validate syntax in CI/CD	utlx validate
Deploy a transformation API	utlxe –mode http
Connect to Azure Service Bus	utlxe + Dapr sidecar
Embed in a C# application	utlxe via C# wrapper
Embed in a Go service	utlxe via Go wrapper
Run performance benchmarks	utlxe –mode http with load testing

The typical developer workflow: write in VS Code (utlxd), test on the command line (utlx), deploy to production (utlxe). Same .utlx file at every stage.

7 The UTL-X IDE

While the CLI is powerful for quick transformations and scripting, professional integration development happens in an IDE. UTL-X provides a VS Code extension backed by the utlxd daemon, giving you real-time feedback, autocompletion, and live preview as you write transformations.

This chapter describes the IDE experience and the design-time workflow. If you're using UTL-X purely from the command line, you can skip ahead to Chapter 8 — but the IDE will make you significantly more productive.

7.1 Installing the VS Code Extension

The UTL-X extension is available from the VS Code marketplace:

1. Open VS Code
2. Go to Extensions (Ctrl+Shift+X / Cmd+Shift+X)
3. Search for "UTL-X"
4. Click Install

The extension automatically starts the utlxd daemon when you open a .utlx file. No manual configuration needed — the daemon manages its own lifecycle.

7.2 The Three-Panel Layout

The UTL-X IDE experience is built around three panels, visible simultaneously:

Left panel — Input: paste or load sample input data. Select the format (XML, JSON, CSV, YAML, OData). This is your test data — what the transformation will process.

Center panel — Transformation: the .utlx editor with syntax highlighting, autocompletion, and real-time error diagnostics. This is where you write your transformation logic.

Right panel — Output: the live preview of the transformation result. Updates automatically as you type in the center panel. Shows the output in the declared format (JSON, XML, CSV, YAML).

This three-panel workflow — Input, Transform, Output — is the core of UTL-X development. You see the input, you write the logic, you see the output. No save-compile-run cycle. No switching between terminal and editor. Everything is visible at once.

7.3 Design-Time vs Runtime

Understanding the difference between design-time and runtime is crucial for UTL-X development:

Design-time is when you write the transformation:
- You're in VS Code, editing a .utlx file
- utlxd parses your code on every keystroke
- The IDE shows errors, completions, and live preview
- You work with *sample data* — representative examples, not production messages
- You can experiment freely — nothing is deployed, nothing is production

Runtime is when the transformation executes in production:
- utlxe runs the compiled transformation against real messages

- 8–128 workers process messages concurrently
- Schema validation enforces contracts
- Prometheus metrics track throughput and errors
- There is no IDE — the transformation is a deployed artifact

What you create at design-time deploys *unchanged* to runtime. The .utlx file you write in VS Code is the exact same file that utlxe loads in production. No compilation step, no packaging, no build process — the source IS the artifact.

7.4 Schema-Driven Development

The most productive way to use the IDE is *schema-driven development*: define the input and output schemas first, then write the transformation to connect them.

7.4.1 The Professional Approach

1. **Define the input schema:** what does the source data look like? (XSD, JSON Schema, Avro, etc.)
2. **Define the output schema:** what must the target data look like?
3. **Load both schemas in the IDE:** the UDM tree browser shows both structures side-by-side
4. **Write the transformation:** map fields from input to output, guided by the schemas
5. **Validate:** the IDE checks the output against the target schema in real-time

This is how Tibco BW, SAP CPI, and MuleSoft work — but they limit you to XML (XSLT). UTL-X does schema-driven development for *all* formats: JSON Schema, XSD, Avro, Protobuf, Table Schema, and OData Schema.

7.4.2 Scaffolding

When both input and output schemas are defined, the IDE can generate a transformation skeleton:

- All target fields listed with placeholder expressions
- Type conversions where source and target types differ
- Array mappings where cardinality differs
- TODO comments for fields requiring business logic

You fill in the actual mapping logic. Scaffolding accelerates development — it's not a replacement for writing transformations, but it eliminates the boilerplate of creating the output structure manually.

7.5 The UDM Tree Browser

The UDM tree browser is a panel that shows the parsed Universal Data Model as an interactive tree. Load sample data in the input panel, and the tree browser displays the complete structure:

Each node in the tree shows:

- **Property name:** the field name in the data
- **UDM type:** Scalar, Object, Array, DateTime, etc.
- **Value:** for scalar nodes, the actual value
- **Format-specific information:**
  - XML: element name, attributes (@attr), namespace declarations

- JSON: type (string, number, boolean, null, object, array)
- CSV: column index, header name, detected type
- YAML: anchor/alias references
- OData: @odata.type, navigation properties

Click a node to insert the accessor path into the transformation editor. For example, clicking the "Name" node under "Customer" inserts `\$input.Customer.Name` at the cursor position. This eliminates typos and speeds up field mapping.

Expand and collapse nodes to navigate deep structures. Search within the tree to find specific fields in large documents.

7.6 The Function Library

The function library panel lets you browse all 652 standard library functions without leaving the editor:

- **Browse by category:** 18 categories (String, Array, Math, Date, Type, Encoding, XML, JSON, CSV, YAML, Security, Binary, Financial, Geospatial, Utility, Object, Core, Other)
- **Search:** type a keyword to find functions by name or description
- **Function details:** name, signature (parameter types and return type), description, example
- **Click to insert:** click a function name to insert a call template at the cursor

For example, searching "date" shows: now(), today(), parseDate(), formatDate(), addDays(), dateDiff(), and more — with signatures and examples for each.

7.7 Real-Time Diagnostics

The IDE shows errors as you type, without saving or running:

- **Syntax errors:** missing brackets, invalid keywords, unterminated strings — highlighted with red underlines and error messages
- **Unknown functions:** calling a function that doesn't exist in the stdlib — immediately flagged
- **Type warnings:** passing a string where a number is expected (when detectable from context)
- **Schema violations:** when output doesn't match the declared output schema

Errors appear in the Problems panel (Ctrl+Shift+M) with line numbers and descriptions. Click an error to jump to the offending line.

7.8 Autocompletion

The IDE provides context-aware autocompletion:

- **After $input. :** property names from the parsed input data (if sample data is loaded)
- **After a function name and parenthesis:** parameter hints showing expected types
- **At the start of an expression:** function names, keywords (if, let, def, match, try)
- **Inside an object literal:** available property names from the output schema (if loaded)

Autocompletion uses the utlxd daemon's real-time understanding of your transformation — it's not a static word list but a live analysis of the AST and available data.

7.9 The Header Editor

The .utlx header (the part above the separator) can be edited visually:

- **Input format:** dropdown to select XML, JSON, CSV, YAML, OData
- **Output format:** dropdown with format-specific options
- **Output options:** checkboxes and fields for options like writeAttributes, encoding, delimiter
- **Multi-input:** add or remove named inputs with their formats
- **Schema references:** attach input and output validation schemas

The header editor generates the header block — you can always switch to text editing if you prefer manual control.

7.10 Keyboard Shortcuts

Shortcut	Action
Ctrl+Space	Trigger autocompletion
F12	Go to definition
Shift+F12	Find all references
Ctrl+Shift+F	Format document
Ctrl+Shift+M	Show Problems panel
Ctrl+K Ctrl+I	Show hover information
Ctrl+Shift+P then "UTL-X"	All UTL-X commands

7.11 Tips for Productive IDE Usage

Start with sample data. Before writing a single line of transformation code, paste representative sample data in the input panel. The UDM tree browser and autocompletion become dramatically more useful when they have real data to analyze.

Use the live preview. Don't run transformations from the terminal during development. The live preview updates on every keystroke — you see the output change as you type. This is faster than any save-compile-run cycle.

Work schema-first. If you have schemas for both input and output, load them. The IDE validates your output in real-time, catching contract violations before you even save the file.

Browse functions, don't memorize. With 652 functions, nobody remembers them all. Use the function library search. Type what you want to do — "convert date", "filter array", "encrypt" — and the library shows relevant functions.

Trust the red underlines. If the IDE shows an error, fix it before moving on. Real-time diagnostics catch 90% of issues that would otherwise only surface at runtime.

8 Language Fundamentals

This chapter teaches the building blocks of UTL-X: the types you work with, how you access data, how you bind variables, and how you make decisions. Everything that follows — functions, patterns, pipelines — builds on these fundamentals.

If you've written JavaScript, Python, or any functional language, most of this will feel familiar. UTL-X deliberately borrows syntax from widely-known languages so you can be productive immediately.

8.1 Scalar Types

Scalars are the atomic values — the leaves of any data tree.

Strings: enclosed in double quotes.

```
"Hello, World"
"Line one\nLine two"
"She said \"hello\""
```

Numbers: integers and decimals. UTL-X uses 64-bit floating point internally (like JavaScript), but preserves integer representation when possible.

```
42
3.14
-17
1.5e10
```

Booleans:

```
true
false
```

Null: represents the absence of a value.

```
null
```

Type coercion happens automatically in some contexts — for example, concatenating a number with a string converts the number to a string. For explicit conversion, use the stdlib functions: `toString()`, `toNumber()`, `toBoolean()`.

8.2 Objects

Objects are collections of named properties — the workhorses of data transformation.

8.2.1 Object Literals

Create objects with curly braces and key-value pairs:

```
{
  name: "Alice",
```

```
  age: 30,
  active: true
}
```

Keys don't need quotes unless they contain special characters. Values can be any expression — including other objects, arrays, or function calls.

8.2.2 Property Access

Access properties with dot notation:

```
$input.customer.name
$input.order.items[0].price
```

Or bracket notation for dynamic keys:

```
$input["customer"]["name"]
let field = "name"
$input.customer[field]
```

8.2.3 Nested Access

Dot notation chains naturally for deep structures:

```
$input.Order.Customer.Address.City
```

This works identically whether the data came from XML, JSON, CSV, or YAML — the UDM normalizes all formats into the same property access pattern.

8.2.4 XML Attribute Access

XML attributes use the `@` prefix:

```
$input.Order.@id              // the "id" attribute on <Order>
$input.Product.@price          // the "price" attribute on <Product>
```

This is specific to XML input — JSON, CSV, and YAML don't have attributes. See Chapter 20 for XML transformation details and Chapter 21 for the attribute design decisions.

8.2.5 Computed Property Names

When the key is dynamic, use square brackets:

```
let key = "fullName"
{ [key]: concat($input.first, " ", $input.last) }
// produces: {"fullName": "Alice Johnson"}
```

8.2.6 The Spread Operator

Merge objects with `...`:

```
let base = {name: "Alice", age: 30}
let override = {age: 31, active: true}
{...base, ...override}
// produces: {name: "Alice", age: 31, active: true}
```

Later spreads override earlier ones — `age: 31` wins over `age: 30`.

8.3 Arrays

Arrays are ordered collections — essential for processing lists of items, rows, or repeated elements.

8.3.1 Array Literals

```
[1, 2, 3]
["Alice", "Bob", "Charlie"]
[{name: "Alice"}, {name: "Bob"}]
```

8.3.2 Index Access

Zero-based indexing:

```
$input.items[0]       // first item
$input.items[2]       // third item
```

Negative indexing is not supported — use `last()` instead:

```
last($input.items)    // last item
```

8.3.3 Array Operations

Arrays are processed with higher-order functions (Chapter 14 covers these in depth):

```
// Transform each element
map($input.items, (item) -> item.name)

// Keep only matching elements
filter($input.items, (item) -> item.price > 100)

// Accumulate a result
reduce($input.items, 0, (sum, item) -> sum + item.price)

// Sort
sortBy($input.items, (item) -> item.name)
```

8.4 The $input Variable

Every UTL-X transformation has access to `\$input` — the parsed input data as a UDM tree.

```
$input                        // the entire input document
$input.Order                  // a child property
```

```
$input.Order.Items[0]         // array element
$input.Order.@id              // XML attribute
```

For multi-input transformations, each input has its own named variable:

```
// Header: input: orders xml, customers json
$orders.Order[0].Total        // from the XML input
$customers[0].name            // from the JSON input
```

8.4.1 Safe Navigation

What if a property doesn't exist? Without safe navigation, you get a null reference. With `?.`, you get `null` gracefully:

```
$input.order?.discount         // null if "order" or "discount" is missing
$input.customer?.address?.city // null if any step is missing
```

8.4.2 Recursive Descent

Search for a property name anywhere in the tree with `..`:

```
$input..ProductCode     // finds "ProductCode" at any depth
```

This returns an array of all matching values — useful when the exact path varies or the structure is deeply nested.

8.5 Variable Binding (let)

Use `let` to bind a value to a name. Variables are immutable — once bound, they cannot be changed.

```
let name = $input.customer.name
let total = sum(map($input.items, (i) -> i.price * i.qty))
{
  customerName: name,
  orderTotal: total,
  averagePrice: total / count($input.items)
}
```

Variables make transformations readable by naming intermediate results. Without `let`, you'd nest everything:

```
// Without let — hard to read
{
  orderTotal: sum(map($input.items, (i) -> i.price * i.qty)),
  averagePrice: sum(map($input.items, (i) -> i.price * i.qty)) /
count($input.items)
}
```

```
// With let — clear and no duplication
let total = sum(map($input.items, (i) -> i.price * i.qty))
{
  orderTotal: total,
  averagePrice: total / count($input.items)
}
```

Multiple `let` bindings chain naturally — each sees the previous bindings. But the separator between `let` bindings depends on context. This is one of the few places where UTL-X syntax requires attention:

8.5.1 Top-Level: No Separator Needed

At the top level of a .utlx body (outside any braces), `let` bindings are separated by newlines:

```
let items = $input.Order.Items.Item
let prices = map(items, (i) -> toNumber(i.@price) * toNumber(i.@qty))
let total = sum(prices)
let tax = total * 0.21
{
  subtotal: total,
  tax: tax,
  grandTotal: total + tax
}
```

No commas, no semicolons — just newlines. The final expression (the object literal) is the return value.

8.5.2 Inside Object Literals: Commas Required

When `let` bindings appear *inside* an object literal (between `{` and `}`), they need commas — just like properties:

```
{
  let subtotal = 100,
  let tax = subtotal * 0.08,
  let total = subtotal + tax,

  subtotal: subtotal,
  tax: tax,
  total: total
}
```

The commas separate `let` bindings from each other and from the property definitions. Think of `let` inside braces as "object members" — they follow the same comma rules as properties.

8.5.3 Inside Lambdas Returning Arrays: Semicolons Required

When `let` bindings are followed by an array (not an object), the parser needs semicolons to distinguish "array return" from "array indexing":

```
// Inside a map lambda that returns an array:
items |> map(item => {
  let price = toNumber(item.Price);
  let tax = price * 0.08;

  [item.Name, price + tax]
})
```

Without semicolons, the parser would read `let tax = price * 0.08[item.Name, ...]` — interpreting the array as an index operation on the let value.

8.5.4 Summary of Let Separators

Context	Separator	Example
Top-level (.utlx body)	Newline	let x = 1 (newline) let y = 2
Inside object { }	Comma	{let x = 1, let y = 2, result: x + y}
Lambda returning array	Semicolon	let x = 1; let y = 2; [x, y]

When in doubt, use commas inside braces and semicolons inside lambdas. At the top level, newlines are always sufficient.

Design Note (F02): The three-separator rule for `let` bindings is a known inconsistency inherited from the parser implementation, not a deliberate language design choice. A future UTL-X version will simplify this to: *newlines are always sufficient* (the Kotlin model). Commas and semicolons will remain accepted for backward compatibility but will no longer be required. See the F02 design document for details. This section of the book will be updated when the improvement is implemented.

8.6 Conditional Expressions

8.6.1 if / else

UTL-X's `if` is an *expression*, not a statement — it always produces a value:

```
if ($input.age >= 18) "adult" else "minor"
```

Use parentheses around the condition. Both branches are required — `if` without `else` is not allowed (every expression must produce a value).

Nest for complex logic:

```
if ($input.score >= 90) "A"
else if ($input.score >= 80) "B"
else if ($input.score >= 70) "C"
else "F"
```

8.6.2 Ternary Operator

For inline conditions, the ternary operator is more concise:

```
$input.active ? "yes" : "no"
```

8.6.3 Nullish Coalescing (??)

Provide a default value when something is null:

```
$input.nickname ?? $input.name ?? "Unknown"
```

This evaluates left to right — the first non-null value wins.

8.6.4 Combining Conditionals

Real-world transformations combine these freely:

```
{
  status: if ($input.paid) "PAID" else "PENDING",
  discount: $input.loyaltyLevel ?? "NONE",
  shipping: $input.express ? "OVERNIGHT" : "STANDARD",
  greeting: concat("Dear ", $input.title ?? "Customer")
}
```

8.7 Comments

Single-line comments start with `//`:

```
// Calculate the order total including tax
let total = sum(prices)  // sum all line items
```

There are no multi-line comments in UTL-X. This is deliberate — it keeps transformations scannable. Every line is either code or a comment, never ambiguous.

8.8 Type Annotations (Optional)

UTL-X supports optional type annotations for documentation:

```
let name: String = $input.customer.name
let total: Number = sum(prices)
```

Type annotations are *not enforced at runtime* in the current version. They serve as documentation — making your intent clear to other developers (and to the IDE, which uses them for better autocompletion).

8.9 Putting It Together

Here's a complete transformation that uses all the fundamentals covered in this chapter:

```
%utlx 1.0
input xml
output json
---
// Extract order data
```

```
let order = $input.Order
let items = order.Items.Item
let currency = order.@currency ?? "EUR"

// Calculate financial summary
let lineAmounts = map(items, (item) ->
  toNumber(item.@price) * toNumber(item.@qty)
)
let subtotal = sum(lineAmounts)
let taxRate = if (order.Customer.@country == "NL") 0.21 else 0
let tax = subtotal * taxRate

// Build output
{
  orderId: order.@id,
  customer: order.Customer,
  currency: currency,
  items: map(items, (item) -> {
    sku: item.@sku,
    description: item,
    quantity: toNumber(item.@qty),
    unitPrice: toNumber(item.@price),
    lineTotal: toNumber(item.@price) * toNumber(item.@qty)
  }),
  subtotal: subtotal,
  tax: tax,
  total: subtotal + tax,
  status: order.@status ?? "NEW"
}
```

This transformation demonstrates: property access, attribute access (@), safe navigation (??), let bindings, map with lambdas, conditional logic (if/else), arithmetic, and object construction. All in 25 lines of readable, declarative code.

The next chapter explores the Universal Data Model — the internal representation that makes all of this work across formats.

9 The Universal Data Model (UDM)

The Universal Data Model is the single most important concept in UTL-X. It's the abstraction that makes format-agnosticism possible — the internal representation that lets your transformation work identically whether the input is XML, JSON, CSV, YAML, or OData.

This chapter explains what UDM is, how each format maps to it, and why certain design decisions were made. If you want to understand *why* UTL-X works the way it does — not just *how* — this is the chapter.

9.1 Why UDM Exists

Consider the problem: XML has elements, attributes, namespaces, and mixed content. JSON has objects, arrays, strings, numbers, booleans, and null. CSV has rows and columns. YAML has all of JSON's types plus anchors, aliases, and tags.

Without an intermediate representation, a transformation engine would need separate logic for every format combination:

- XML → JSON: handle attributes, namespaces, text nodes
- JSON → XML: handle types (number → string), arrays (→ repeated elements)
- CSV → JSON: handle headers, type detection
- XML → CSV: handle hierarchy flattening
- … and so on for every pair

With N formats, that's N × (N-1) conversion paths — each with its own edge cases and bugs.

UDM eliminates this by providing one common tree structure:

Every input format is parsed into UDM. Every transformation operates on UDM. Every output format is serialized from UDM. The transformation never touches raw XML tags or JSON braces — it works with the *meaning* of the data.

9.2 UDM Types

UDM has eight types. Every piece of data from any format maps to one of these:

UDM Type	What it holds	Examples
Scalar	A single value: string, number, boolean, or null	"Alice", 42, 3.14, true, null
Object	Named properties + optional XML attributes + metadata	{name: "Alice", age: 30}
Array	Ordered list of UDM values	[1, 2, 3], [{…}, {…}]
DateTime	Timestamp with timezone (ISO 8601)	2026-04-28T14:30:00Z
Date	Date only, no time	2026-04-28
LocalDateTime	Date and time without timezone	2026-04-28T14:30:00
Time	Time only, no date	14:30:00
Binary	Raw byte array	Binary file content

There's also a Lambda type for function values, but you rarely encounter it directly — it's used internally for `map`, `filter`, and other higher-order functions.

The first three types — Scalar, Object, and Array — account for 99% of real-world data. The temporal types (DateTime, Date, LocalDateTime, Time) provide first-class date handling without string parsing at every access.

9.3 How Formats Map to UDM

9.3.1 JSON → UDM

JSON maps to UDM almost directly — JSON was the inspiration for UDM's structure:

JSON	UDM
String: "Alice"	Scalar(string: "Alice")
Number: 42	Scalar(number: 42)
Boolean: true	Scalar(boolean: true)
Null: null	Scalar(null)
Object: {"name": "Alice"}	Object(properties: {name → Scalar("Alice")})
Array: [1, 2, 3]	Array(elements: [Scalar(1), Scalar(2), Scalar(3)])

JSON-to-UDM is lossless. UDM-to-JSON is lossless. Round-trip fidelity is guaranteed.

9.3.2 XML → UDM

XML is more complex because it has features that JSON lacks: attributes, namespaces, mixed content, and the distinction between elements and text.

XML	UDM
Element with children: <Order><Customer>…</Customer></Order>	Object(properties: {Customer → …})
Element with text: <Name>Alice</Name>	Object(properties: {_text → Scalar("Alice")})
Attribute: id="123"	Stored in Object.attributes map
Repeated elements: <Item/><Item/>	Array of Objects
Namespace: xmlns="…"	Stored in Object.metadata

The key design decisions for XML:

Text content uses _text. When an XML element contains only text — like `<Name>Alice</Name>` — the text is stored as a property called `\_text` inside the UDM Object. This is an internal convention that you never see in output (the serializers unwrap it automatically). It exists because UDM Objects store properties as a map, and the text content needs a key.

When you write `\$input.Order.Customer`, UTL-X automatically unwraps the `\_text` and returns "Alice" — not the internal `\{\_text: "Alice"\}` wrapper. This unwrapping was refined through bugs B13 and B14 (see Chapter 21 for the full story).

Attributes are separate. XML attributes are stored in a separate `attributes` map on the UDM Object, not mixed with child element properties. This prevents name collisions — an element could theoretically have both a child element and an attribute with the same name. Access attributes with the `\@` prefix: `\$input.Order.\@id`.

Repeated elements become arrays. When an XML element appears multiple times with the same name, they're automatically grouped into a UDM Array. `<Item/><Item/><Item/>` becomes an Array of three Objects. Single elements stay as Objects (not wrapped in an array) unless array hints are provided.

9.3.3 CSV → UDM

CSV maps to an Array of Objects, where each row becomes an Object with column headers as property names:

```
name,age,city
Alice,30,Amsterdam
Bob,25,Rotterdam
```

Becomes:

```
Array([
  Object({name: "Alice", age: 30, city: "Amsterdam"}),
  Object({name: "Bob", age: 25, city: "Rotterdam"})
])
```

Numbers are auto-detected: "30" becomes Scalar(number: 30), not Scalar(string: "30"). Booleans too: "true" becomes Scalar(boolean: true).

9.3.4 YAML → UDM

YAML is a superset of JSON, so the mapping is the same as JSON. YAML-specific features (anchors, aliases, tags) are resolved during parsing — the resulting UDM is identical to what JSON would produce for the same data.

9.3.5 OData → UDM

OData JSON is standard JSON with metadata conventions (@odata.context, @odata.type). These metadata properties are parsed as regular Object properties — accessible via `$input["\@odata.context"]`.

9.4 UDM Navigation

9.4.1 Dot Notation

The primary way to access data:

```
$input.Order.Customer.Name          // nested property access
$input.Order.Items.Item[0].Price    // array element access
$input.Order.@id                     // XML attribute access
```

9.4.2 Safe Navigation

Handle missing properties gracefully with `?.`:

```
$input.Order?.Customer?.Name        // returns null if any step is missing
```

Without `?.`, a missing intermediate property causes an error. With `?.`, you get `null` — which you can then handle with `??` (nullish coalescing):

```
$input.Order?.Customer?.Name ?? "Unknown Customer"
```

9.4.3 Recursive Descent

Find a property name at any depth with `..`:

```
$input..ProductCode                 // finds ProductCode anywhere in the tree
```

Returns an array of all matches. Useful for deeply nested or variable-depth structures.

9.5 UDM and Type Coercion

UTL-X provides both automatic and explicit type coercion.

9.5.1 Automatic Coercion

In certain contexts, UTL-X converts types automatically:

- String concatenation: `concat("Total: ", 42)` — number 42 becomes string "42"
- XML text unwrapping: `\$input.Customer` — the internal _text Object unwraps to the scalar value

9.5.2 Explicit Coercion

Use stdlib functions for explicit conversion:

```
toString(42)           // "42"
toNumber("42")         // 42
toBoolean("true")      // true
toNumber("not-a-num")  // 0 (returns 0 for unparseable input)
```

9.5.3 XML Text Node Unwrapping

This is the most important coercion in UTL-X. When XML is parsed:

```
<Name>Alice</Name>
```

The UDM contains: `Object(properties: \{\_text: Scalar("Alice")\})`.

But when you write `\$input.Name`, you get `"Alice"` — not the wrapper object. UTL-X automatically unwraps the _text node during property access. This is handled by the interpreter (for TEMPLATE strategy) and RuntimeOps (for COMPILED strategy).

The unwrapping rules:

- If an Object has only a `\_text` property and no real attributes → unwrap to the scalar
- If an Object has `\_text` AND attributes → keep as Object (attributes would be lost)
- If an Object has child elements → no unwrapping (it's a real object)

These rules were stabilized in bugs B13 (property access unwrapping) and B14 (serializer output unwrapping). Chapter 21 covers the design decisions in detail.

9.6 UDM is Format-Agnostic, Transformations are Format-Agnostic

The key insight: because UDM normalizes all formats into one tree, your transformation expression is *identical* regardless of input format.

The expression `\$input.Order.Customer` works whether the input is:

- XML: `<Order><Customer>Alice</Customer></Order>`
- JSON: `\{"Order": \{"Customer": "Alice"\}\}`
- YAML: `Order:\n  Customer: Alice`

Change `input xml` to `input json` in the header — the body stays the same. This is what "format-agnostic" means in practice: the transformation logic is decoupled from the serialization format.

9.7 UDM and Flat/Relational Data

UDM is inherently hierarchical — Objects contain Objects which contain Objects. This mirrors XML and JSON naturally. But not all data is hierarchical.

9.7.1 The Flat Data Challenge

Consider SAP IDoc segments exported as XML. Orders and order lines are *siblings*, not nested:

```
<IDOC>
  <E1EDK01><BELNR>ORD-001</BELNR><CURRENCY>EUR</CURRENCY></E1EDK01>
  <E1EDP01><BELNR>ORD-001</BELNR><MATNR>Widget</MATNR><MENGE>2</MENGE></E1EDP01>
  <E1EDP01><BELNR>ORD-001</BELNR><MATNR>Gadget</MATNR><MENGE>1</MENGE></E1EDP01>
  <E1EDK01><BELNR>ORD-002</BELNR><CURRENCY>USD</CURRENCY></E1EDK01>
  <E1EDP01><BELNR>ORD-002</BELNR><MATNR>Gizmo</MATNR><MENGE>5</MENGE></E1EDP01>
</IDOC>
```

The order lines belong to orders — but the relationship is in the `BELNR` field value, not in the XML structure. UDM parses this as siblings in the same array, with no awareness that `BELNR` is a join key.

This pattern appears in SAP IDocs, EDI/EDIFACT segments, database exports, and any flat file where parent-child relationships are expressed through reference keys rather than nesting.

9.7.2 How to Handle It in UTL-X

Use `groupBy` to build a lookup, then `map` to construct the hierarchy:

```
let headers = $input.IDOC.E1EDK01
let linesByOrder = groupBy($input.IDOC.E1EDP01, (line) -> line.BELNR)

map(headers, (header) -> {
  orderId: header.BELNR,
  currency: header.CURRENCY,
  lines: map(linesByOrder[header.BELNR] ?? [], (line) -> {
    product: line.MATNR,
    quantity: toNumber(line.MENGE)
  })
})
```

The `groupBy` creates an indexed map (O(N) to build), and each header does an O(1) key lookup — efficient even with thousands of records.

For multi-level nesting (header → line → schedule), chain multiple groupBy calls:

```
let linesByOrder = groupBy($input.lines, (l) -> l.orderId)
let schedulesByLine = groupBy($input.schedules, (s) -> s.lineId)

map($input.headers, (h) -> {
  ...h,
  lines: map(linesByOrder[h.orderId] ?? [], (l) -> {
    ...l,
    schedules: schedulesByLine[l.lineId] ?? []
  })
})
```

9.7.3 Hierarchical vs Relational: A Design Trade-Off

Tools like IBM's Mercator (later WTX) solved this with an "intermediate card" — a declarative mapping that specifies join keys and cardinalities. The engine then automatically restructures flat data into hierarchies.

UTL-X takes a different approach: the `groupBy` + `map` pattern uses existing language constructs instead of a separate mapping artifact. This is more verbose but more flexible — you can add conditions, transformations, and error handling at each level.

A future `join()` stdlib function could simplify the common case:

```
let enrichedOrders = join(
  \$input.IDOC.E1EDK01,           // parent records
  \$input.IDOC.E1EDP01,           // child records
  (header) -> header.BELNR,       // parent key extractor
  (line) -> line.BELNR,           // child key extractor
  "lines"                         // name for the new property on each parent
)
```

The 5th parameter (`"lines"`) is a string that becomes a *new property name* on each parent object. After `join()`, every E1EDK01 gains a `.lines` property containing its matched E1EDP01 records — you access it with dot notation like any other property:

```
// After join(), .lines is a regular property:
map(enrichedOrders, (order) -> {
  orderId: order.BELNR,
  lineCount: count(order.lines),
  total: sum(map(order.lines, (l) -> toNumber(l.MENGE) * toNumber(l.PRICE)))
})
```

See the F03 design document and the architecture document on N-to-M mapping for the full analysis, performance characteristics, and multi-level nesting patterns.

9.8 When UDM Matters

Most of the time, you don't think about UDM — you just write `\$input.name` and it works. UDM matters when:

- You're debugging unexpected behavior (especially XML attributes and _text)
- You're working with flat/relational data (IDoc segments, CSV with foreign keys)
- You're optimizing memory usage (UDM expansion factors — see Chapter 35)
- You're working with the COMPILED strategy (which generates bytecode against UDM types)
- You're writing a wrapper library (which serializes UDM to/from protobuf)
- You're contributing to UTL-X itself (all core logic operates on UDM)

For day-to-day transformation development, UDM is invisible — which is exactly the point. It does its job and gets out of the way.

10 Schema-to-Schema Mapping

This chapter is about *thinking in schemas* — defining your input and output contracts before writing transformation code. It covers how traditional tools approach schema mapping (and why their XSLT-centric approach is broken for JSON), how UTL-X handles it differently, and the full matrix of schema-to-schema conversions.

If you just want to transform data and don't care about schemas, you can skip this chapter. But if you're building production integrations — especially in regulated industries like finance, healthcare, or government — schema-driven development is non-negotiable.

10.1 Why Schema Mapping Matters

A schema defines the *contract* between two systems: what fields exist, what types they are, which are required, and what constraints apply. Without schemas, integration is ad-hoc — you discover missing fields, wrong types, and structural mismatches at runtime, often in production.

Professional integration starts with schemas:

1. Define the input schema: "What does the source system send?"
2. Define the output schema: "What does the target system expect?"
3. Write the transformation: "How do I map from one to the other?"
4. Validate: "Does my output actually match the contract?"

This is how every mature integration platform works — Tibco BW, SAP CPI, MuleSoft, and now UTL-X. The difference is *which schema formats* they support and *how* they do the mapping.

10.2 How Traditional Tools Do It

10.2.1 The XSLT Approach

In Tibco BW, SAP CPI, and Azure Logic Apps, the workflow is:

1. Load source XSD (XML Schema)
2. Load target XSD
3. Draw mapping lines between elements in a visual mapper
4. The tool generates an XSLT stylesheet
5. Runtime: XML input → XSLT processor → XML output

This works well for XML-to-XML transformations. The problem is: what about JSON?

10.2.2 The JSON Problem

Most modern APIs use JSON. But the mapping tools above only understand XSD (XML schemas). So they do this:

1. JSON arrives
2. Convert JSON to XML (using an internal XML "infoset" representation)
3. Load the source XSD (generated from JSON Schema or inferred)
4. Apply XSLT transformation
5. XML output
6. Convert XML back to JSON

Four conversion steps. Two of them lossy:

JSON → XML is lossy: JSON has typed values (numbers, booleans). XML has only text. JSON arrays have no element names. JSON has no attributes. These semantic gaps mean the conversion is an approximation, not an equivalence.

XML → JSON is lossy: XML attributes have no JSON equivalent. Single vs repeated elements (object vs array) is ambiguous. Namespace prefixes are stripped or mangled.

This anti-pattern is not theoretical — it's how Azure Logic Apps' Data Mapper, SAP CPI's Message Mapping, and Tibco BW work today. MuleSoft's DataWeave is the notable exception — it's format-agnostic, like UTL-X.

10.3 The UTL-X Approach

UTL-X doesn't convert between formats before mapping. The transformation operates on the UDM (Universal Data Model), which represents all formats natively:

- JSON input: parsed directly to UDM (lossless)
- XML input: parsed directly to UDM (attributes and namespaces preserved)
- CSV input: parsed directly to UDM (rows become objects)
- No intermediate XML. No XSLT. No lost types.

The schema mapping is format-agnostic too. You can map from a JSON Schema to an XSD-defined structure — UTL-X handles the format differences in parsing and serialization, not in the mapping logic.

10.4 Tier 1: Instance Data Mapping

Instance data mapping is what most people think of when they hear "transformation": converting actual data from one structure to another.

10.4.1 Every Data Format Has an Accompanying Schema Format

Data Format (Tier 1)	Schema Format (Tier 2)	Standard
JSON	JSON Schema	IETF drafts (04 through 2020-12)
XML	XSD	W3C XML Schema Definition
CSV	Table Schema (TSCH)	Frictionless Data
YAML	JSON Schema	Shared with JSON
OData	EDMX / CSDL (OSCH)	OASIS OData

The schema describes the *structure* of the data. UTL-X can validate input and output against these schemas. The transformation maps between instances (actual data), guided by the schemas.

10.4.2 Common Instance Mapping Scenarios

- JSON → JSON: REST API v1 response → REST API v2 request
- XML → JSON: SOAP response → REST API request
- CSV → XML: bank statement → SEPA payment instruction (pain.001)
- JSON → CSV: API data → Excel-compatible report
- XML → XML: UBL invoice → country-specific tax format
- OData → JSON: Dynamics 365 data → internal canonical model

In every case, UTL-X handles the format conversion automatically. You focus on the field mapping — which field goes where, what calculations are needed, what conditions apply.

10.5 Tier 2: Metadata Mapping (Schema-to-Schema)

This is where UTL-X is unique: it can transform *schemas themselves*, not just data instances.

10.5.1 Why Metadata Mapping Is Powerful

Imagine you have an XSD for your XML API, but your new consumer needs a JSON Schema for their OpenAPI specification. Traditionally, you'd manually recreate the schema — tedious, error-prone, and impossible to keep synchronized.

With UTL-X, you transform the schema:

```
%utlx 1.0
input xsd
output jsch
---
$input
```

The XSD is parsed as structured data (it's XML, after all). The JSON Schema is produced by the JSCH serializer. Types, constraints, and structure are mapped automatically.

10.5.2 The Full Schema-to-Schema Matrix

UTL-X supports conversion between all Tier 2 schema formats:

From / To	XSD	JSCH	Avro	Proto	OSCH	TSCH
XSD	–	Yes	Yes	Yes	Yes	Yes
JSCH	Yes	–	Yes	Yes	Yes	Yes
Avro	Yes	Yes	–	Yes	Partial	Yes
Proto	Yes	Yes	Yes	–	Partial	Yes
OSCH	Yes	Yes	Partial	Partial	–	Yes
TSCH	Yes	Yes	Yes	Yes	Yes	–

"Partial" means some semantic concepts don't map perfectly between formats (e.g., OData navigation properties have no direct Avro equivalent). The conversion preserves what it can and documents what it can't.

10.5.3 Practical Use Cases for Schema Mapping

API gateway migration: Your organization is moving from SOAP (WSDL/XSD) to REST (OpenAPI/JSON Schema). Transform the XSD types to JSON Schema — automatically, not manually.

Kafka pipeline: Your producer uses JSON Schema but your schema registry requires Avro. Transform the schema format without changing the data format.

Data contract generation: You have an XSD for your XML feed. Generate a Table Schema (TSCH) for CSV data quality validation of the same data in flat-file form.

Documentation: Transform any schema into a human-readable table of field names, types, and constraints.

Code generation: Transform a schema into TypeScript interfaces, Java classes, or database DDL.

10.6 Nested Schemas: Include and Import

Real-world schemas are rarely single files. They reference other schemas:

- **XSD:** `<xs:include>` (same namespace) and `<xs:import>` (different namespace)
- **JSON Schema:** `\$ref` (local and remote references)
- **Avro:** Named types referenced by name across schemas
- **Protobuf:** `import "other.proto"`
- **OpenAPI:** `\$ref` to components/schemas, external files
- **RAML:** `!include` fragments, libraries, overlays

UTL-X handles nested schemas by:

1. Resolving references during parsing (following includes/imports)
2. Building a complete type graph (all types visible)
3. Making all types available for mapping
4. Preserving reference structure in output (for round-trip fidelity)

This means you can transform a complex, multi-file XSD with imports into a single JSON Schema with $ref definitions — or vice versa.

10.7 JSON Schema to JSON Schema Mapping

A special but common case: transforming between JSON Schema versions.

JSON Schema has evolved through multiple drafts — draft-04, draft-06, draft-07, 2019-09, and 2020-12. Each version introduced new keywords, changed semantics, or deprecated features.

UTL-X can migrate schemas between versions:

- draft-04 → 2020-12: update `\$schema` URI, replace `definitions` with `\$defs`, update `\$ref` syntax
- Add or remove fields while preserving constraints
- Rename properties while maintaining validation rules

This is valuable for API versioning — evolve the schema while maintaining backward compatibility.

10.8 Schema-Driven Validation in UTL-X

Schemas aren't just for documentation — UTL-X uses them for runtime validation:

10.8.1 Pre-Validation

Validate the input *before* transformation:

```
%utlx 1.0
input json {schema: "order-input.json"}
output xml
```

```
---
// transformation body
```

If the input doesn't match `order-input.json` (a JSON Schema), the transformation fails with a validation error — before any mapping code runs. This catches data quality issues at the source.

10.8.2 Post-Validation

Validate the output *after* transformation:

```
%utlx 1.0
input json
output xml {schema: "invoice-output.xsd"}
---
// transformation body
```

If the output doesn't match `invoice-output.xsd`, the transformation fails — catching mapping bugs before they reach the target system.

10.8.3 The Validation Orchestrator

UTLXe's validation orchestrator runs validation in a sandwich pattern:

1. Pre-validate input against input schema
2. Execute transformation
3. Post-validate output against output schema

If any step fails, the message is rejected with a detailed error — field name, constraint violated, expected vs actual value. No partial results, no corrupted output.

10.9 The Bridge: USDL

The next chapter introduces USDL — UTL-X's own Universal Schema Definition Language. USDL is to schemas what UDM is to data: a format-agnostic representation that bridges all schema formats.

Where UDM asks "what does the data look like?", USDL asks "what *should* the data look like?" — types, constraints, cardinality, and documentation in one portable format.

11 Format Support Overview

UTL-X supports 11 formats organized in two tiers. Tier 1 formats are *data formats* — you transform instances of these daily. Tier 2 formats are *schema formats* — you work with these when defining contracts, validating data, or converting between schema standards.

This chapter provides an overview of all supported formats, their key characteristics, and the conversion matrix. Detailed treatment of each format is in Part III (Chapters 20–28).

11.1 Tier 1: Data Formats

Data formats carry the actual business data — orders, invoices, patient records, sensor readings, configurations.

11.1.1 JSON

The lingua franca of modern APIs. Every REST service, every frontend framework, every NoSQL database speaks JSON.

- **Parser:** recursive descent, handles all JSON types including nested structures
- **Number handling:** preserves integer vs float distinction (42 vs 42.0)
- **Encoding:** UTF-8 (standard), auto-detects BOM
- **Options:** `{writeAttributes: true}` for XML attribute preservation
- **Round-trip:** JSON → UDM → JSON is lossless

11.1.2 XML

The enterprise workhorse. SOAP services, EDI, UBL invoices, HL7 healthcare, government data — XML is everywhere in enterprise integration.

- **Parser:** custom recursive descent with namespace support
- **Attributes:** stored separately in UDM, accessed via `\@` prefix
- **Namespaces:** full support including prefixed elements and inheritance
- **Text content:** internal _text convention (auto-unwrapped, invisible in output)
- **Repeated elements:** auto-detected as arrays, configurable via array hints
- **Encoding:** auto-detects from XML declaration (UTF-8, ISO-8859-1, Windows-1252)
- **Options:** `{encoding: "UTF-8"}` for output encoding

11.1.3 CSV

The universal import/export format. Every spreadsheet, every database, every legacy system can produce and consume CSV.

- **Parser:** handles quoted fields, escaped delimiters, multi-line values
- **Headers:** auto-detected from first row (configurable)
- **Type detection:** numbers, booleans, dates auto-detected from string content
- **Delimiters:** comma (default), semicolon, pipe, tab
- **Regional formats:** USA (1,234.56), European (1.234,56), French (1 234,56), Swiss (1′234.56)
- **BOM:** detects and handles UTF-8 BOM
- **Options:** `{delimiter: ";", headers: true, regionalFormat: "european", decimals: 2}`

11.1.4 YAML

The configuration standard. Kubernetes manifests, CI/CD pipelines, Docker Compose, OpenAPI specs — YAML dominates DevOps.

- **Parser:** SnakeYAML-based, handles all YAML 1.1 features
- **Anchors and aliases:** resolved during parsing (transparent to transformation)
- **Multi-document:** supported (separated by `---`)
- **Output styles:** block (default, human-readable) or flow (compact, JSON-like)
- **Options:** `{writeAttributes: true}`, same as JSON
- **Round-trip:** YAML is a superset of JSON — same UDM mapping

11.1.5 OData

The Microsoft ecosystem format. Dynamics 365, SharePoint, SAP Gateway, Power Platform — OData JSON is standard JSON with metadata conventions.

- **Parser:** standard JSON parser with OData metadata awareness
- **Metadata:** @odata.context, @odata.type, @odata.id accessible as properties
- **Entity sets:** standard `value` array pattern
- **Navigation properties:** nested entity access
- **Options:** `{metadata: "...", context: "...", wrapCollection: true}`

11.2 Tier 2: Schema Formats

Schema formats describe the *structure* of data — types, constraints, cardinality. They're used for validation, documentation, code generation, and contract enforcement.

11.2.1 XSD (XML Schema Definition)

The W3C standard for XML validation. Every enterprise XML standard (UBL, HL7, ISO 20022, EDIFACT) ships with XSD.

- **Versions:** 1.0 and 1.1
- **Patterns:** Russian Doll, Venetian Blind, Salami Slice, Garden of Eden, Swiss Army Knife (Chapter 27)
- **Features:** complex types, simple types, restrictions, extensions, imports

11.2.2 JSCH (JSON Schema)

The schema language for JSON. Used in OpenAPI, AsyncAPI, data contracts.

- **Drafts:** draft-04, draft-06, draft-07, 2019-09, 2020-12
- **Features:** type, properties, required, pattern, enum, $ref, allOf/oneOf/anyOf

11.2.3 Avro

Apache Avro schema for data pipelines. The standard schema format for Kafka and Hadoop ecosystems.

- **Features:** records, enums, arrays, maps, unions, logical types (date, timestamp, decimal, UUID)

11.2.4 Protobuf (Protocol Buffers)

Google's schema format for gRPC and efficient serialization.

- **Version:** proto3

- **Features:** messages, enums, repeated fields, oneof, maps, packages

11.2.5 OSCH (OData Schema / EDMX)

The schema format for OData services. Describes entity types, navigation properties, and service endpoints.

- **Format:** XML-based CSDL (Common Schema Definition Language)

11.2.6 TSCH (Table Schema)

The Frictionless Data standard for CSV metadata. Describes column names, types, and constraints for tabular data.

- **Format:** JSON-based field definitions

11.3 Format Detection

UTL-X auto-detects the input format in identity mode and expression mode:

Detection rule	Format
Starts with `<` or `<?xml`	XML
Starts with `\{` or `[`	JSON
Contains commas/semicolons with consistent column count	CSV
Contains `:` patterns without braces	YAML
Explicit `--from` flag	Overrides auto-detection

Auto-detection works for 95% of cases. Use `--from` when the input is ambiguous (e.g., a YAML file that looks like JSON, or a CSV with only one column).

11.4 Format Conversion Matrix

UTL-X can convert between any pair of Tier 1 formats:

From / To	JSON	XML	CSV	YAML	OData
JSON	–	Yes	Yes	Yes	Yes
XML	Yes	–	Yes	Yes	Yes
CSV	Yes	Yes	–	Yes	Yes
YAML	Yes	Yes	Yes	–	Yes
OData	Yes	Yes	Yes	Yes	–

Every conversion is supported. Some are more natural than others:

- **JSON ↔ YAML:** lossless (YAML is a superset of JSON)
- **JSON ↔ XML:** mostly lossless (XML attributes need `writeAttributes` for full fidelity)
- **CSV → JSON/XML:** lossless (rows become objects/elements)
- **JSON/XML → CSV:** requires flattening (hierarchical → tabular, may lose nesting)
- **OData → JSON:** strip metadata for clean JSON

11.5 Identity Mode Defaults

When using identity mode (no script, just format conversion), UTL-X picks sensible defaults:

Input	Default output	Override with
XML	JSON	`--to yaml`, `--to csv`
JSON	XML	`--to yaml`, `--to csv`
CSV	JSON	`--to xml`, `--to yaml`
YAML	JSON	`--to xml`, `--to csv`

The default "flip" is always to JSON (except JSON input, which flips to XML). This reflects the most common use case: "I have XML/CSV/YAML and I need JSON."

11.6 Format Options Summary

All format options use the `{key: value}` syntax in the output declaration:

Format	Option	Example
JSON	writeAttributes	`{writeAttributes: true}`
XML	encoding	`{encoding: "UTF-8"}`
CSV	delimiter	`{delimiter: ";"}`
CSV	headers	`{headers: true}`
CSV	regionalFormat	`{regionalFormat: "european"}`
CSV	decimals	`{decimals: 2}`
CSV	bom	`{bom: true}`
YAML	writeAttributes	`{writeAttributes: true}`
Any	schema (planned)	`{schema: "contract.json"}`

Options are optional — sensible defaults apply when not specified. Multiple options combine: `output csv {delimiter: ";", headers: true, regionalFormat: "european"}`.

11.7 What's Next

Part I is complete. You now understand:

- What UTL-X is and why it exists (Chapters 1–3)
- How to install and use it (Chapters 4–5)
- The three executables and the IDE (Chapters 6–7)
- The language fundamentals (Chapter 8)
- The Universal Data Model (Chapter 9)
- Schema mapping and USDL (Chapters 10–11)
- All supported formats (this chapter)

Part II dives into the language itself: expressions, functions, pattern matching, validation, and pipeline chaining. By the end of Part II, you'll be able to write any transformation UTL-X is capable of.

Part II

The Language

Mastering expressions, functions, patterns, and advanced features

12 Expressions and Operators

UTL-X is an expression-based language — everything produces a value. There are no statements, no void functions, no side effects. A transformation body is one expression (possibly with `let` bindings) that evaluates to the output.

This chapter covers all operators, their precedence, and practical patterns. If you've used JavaScript, Kotlin, or any C-family language, most operators will be immediately familiar.

12.1 Arithmetic Operators

Standard arithmetic on numbers:

```
10 + 3       // 13       addition
10 - 3       // 7        subtraction
10 * 3       // 30       multiplication
10 / 3       // 3.333... division (always floating point)
10 % 3       // 1        modulo (remainder)
2 ** 10      // 1024     exponentiation
-42          // -42      unary minus
```

Division always produces a floating-point result — `10 / 3` is `3.333...`, not `3`. There is no integer division operator. Use `floor(10 / 3)` if you need integer results.

Division by zero throws a runtime error. Use a guard if the divisor might be zero:

```
if (count > 0) total / count else 0
```

12.2 Comparison Operators

Compare values and produce booleans:

```
10 == 10     // true     equality
10 != 5      // true     inequality
10 > 5       // true     greater than
10 < 5       // false    less than
10 >= 10     // true     greater or equal
10 <= 5      // false    less or equal
```

String comparison is lexicographic (alphabetical):

```
"apple" < "banana"    // true
"Alice" == "alice"    // false (case-sensitive)
```

Null comparisons:

```
null == null   // true
null != "abc"  // true
null == 0      // false (null is not zero)
```

12.3 Logical Operators

Combine boolean expressions:

```
true && false    // false    logical AND
true || false    // true     logical OR
!true            // false    logical NOT
```

Short-circuit evaluation: `&&` stops at the first `false`, `||` stops at the first `true`. This is useful for guarding expressions:

```
// Safe: second expression only evaluates if items is not null
$input.items != null && count($input.items) > 0
```

12.3.1 Truthy and Falsy Values

In boolean contexts, UTL-X treats these as *falsy*:

- `false`
- `null`
- `0`
- `""` (empty string)

Everything else is *truthy*, including empty arrays `[]` and empty objects `{}`.

12.4 String Concatenation

UTL-X does not use `+` for string concatenation — use the `concat()` function:

```
concat("Hello", " ", "World")     // "Hello World"
concat($input.first, " ", $input.last)
```

This is deliberate. `+` is arithmetic only. Mixing `+` for both numbers and strings leads to ambiguity (`"3" + 4` — is it `"34"` or `7`?). UTL-X avoids this by keeping arithmetic and string operations separate.

For complex string building, chain multiple `concat()` calls or use `join()`:

```
join([first, middle, last], " ")   // "Alice M. Johnson"
```

12.5 Object Construction

Objects are created with curly braces. This is both the most common expression and the most powerful:

```
// Simple object
{name: "Alice", age: 30}

// Nested objects
{
  customer: {name: "Alice", city: "Amsterdam"},
```

```
  order: {total: 299.99, currency: "EUR"}
}

// Computed keys
let field = "fullName"
{[field]: concat($input.first, " ", $input.last)}
// produces: {"fullName": "Alice Johnson"}

// Spread: merge objects
let defaults = {currency: "EUR", country: "NL"}
{...defaults, name: $input.name, total: $input.total}
// produces: {currency: "EUR", country: "NL", name: "...", total: ...}
```

12.5.1 Conditional Properties

Include properties only when a condition is true by spreading a conditional object:

```
{
  name: $input.name,
  ...if ($input.discount != null) {discount: $input.discount} else {}
}
```

Or use the nullish coalescing pattern to provide defaults:

```
{
  name: $input.name,
  status: $input.status ?? "NEW",
  priority: $input.priority ?? "NORMAL"
}
```

12.6 Array Construction

Arrays are created with square brackets and processed with higher-order functions:

```
// Literal arrays
[1, 2, 3]
["Alice", "Bob", "Charlie"]

// Array from transformation
map($input.items, (item) -> item.name)
// produces: ["Widget", "Gadget", "Gizmo"]

// Combining arrays with spread
let a = [1, 2, 3]
let b = [4, 5, 6]
[...a, ...b]
// produces: [1, 2, 3, 4, 5, 6]

// Filtering
filter($input.items, (item) -> item.price > 100)
```

12.7 The Pipe Operator (|>)

The pipe operator passes the result of the left expression as the first argument to the right function. It makes data flow readable — left to right instead of nested inside-out:

```
// Without pipe — read from inside out
sort(unique(map($input.items, (item) -> item.category)))

// With pipe — read left to right
$input.items
  |> map((item) -> item.category)
  |> unique()
  |> sort()
```

Both produce the same result: a sorted list of unique categories. But the pipe version reads like a recipe: "take items, map to categories, remove duplicates, sort."

Pipes chain naturally for multi-step processing:

```
$input.orders
  |> filter((o) -> o.status == "ACTIVE")
  |> map((o) -> {id: o.id, total: o.total})
  |> sortBy((o) -> o.total)
  |> map((o) -> concat(o.id, ": ", toString(o.total)))
```

12.8 Nullish Coalescing (??)

Provide a default value when an expression is null:

```
$input.nickname ?? $input.name ?? "Anonymous"
```

Evaluates left to right — the first non-null value wins. Essential for handling optional fields:

```
{
  currency: $input.currency ?? "EUR",
  country: $input.address?.country ?? "NL",
  discount: $input.promoCode ?? "NONE"
}
```

12.9 Safe Navigation (?.)

Access properties on potentially null values without throwing errors:

```
$input.order?.customer?.address?.city
// Returns null if any step is null — no error
```

Without `?.`, a null intermediate value causes a runtime error. With `?.`, null propagates silently.

Combine with `??` for defaults:

```
$input.order?.customer?.name ?? "Unknown Customer"
```

12.10 Operator Precedence

From highest to lowest precedence:

Precedence	Operators	Associativity
1 (highest)	Unary: `!`, `-`	Right
2	Exponentiation: `**`	Right
3	Multiplicative: `*`, `/`, `%`	Left
4	Additive: `+`, `-`	Left
5	Comparison: `<`, `>`, `<=`, `>=`	Left
6	Equality: `==`, `!=`	Left
7	Logical AND: `&&`	Left
8	Logical OR: `||`	Left
9	Nullish coalescing: `??`	Left
10	Ternary: `? :`	Right
11	Pipe: `|>`	Left
12 (lowest)	Conditional: `if/else`	Right

When in doubt, use parentheses. They make intent explicit and prevent precedence surprises:

```
// Ambiguous: does && bind tighter than ||?
a || b && c        // means: a || (b && c) — && is higher

// Clear: parentheses make intent obvious
(a || b) && c      // different meaning, explicit
```

12.11 Practical Expression Patterns

12.11.1 Price Calculation

```
let qty = toNumber($input.quantity)
let unit = toNumber($input.unitPrice)
let discount = toNumber($input.discount ?? "0") / 100
let net = qty * unit * (1 - discount)
let vat = net * 0.21
{
  net: round(net * 100) / 100,
  vat: round(vat * 100) / 100,
  gross: round((net + vat) * 100) / 100
}
```

12.11.2 Conditional Formatting

```
{
  status: if ($input.paid && $input.shipped) "COMPLETE"
          else if ($input.paid) "AWAITING_SHIPMENT"
          else "PENDING_PAYMENT",

  urgency: if ($input.daysOverdue > 30) "CRITICAL"
           else if ($input.daysOverdue > 7) "WARNING"
           else "NORMAL",

  label: concat(
    $input.firstName ?? "",
    if ($input.middleName != null) concat(" ", $input.middleName) else "",
    " ",
    $input.lastName ?? ""
  )
}
```

12.11.3 Data Normalization

```
{
  email: lowerCase(trim($input.email ?? "")),
  phone: replace(replace($input.phone ?? "", " ", ""), "-", ""),
  country: upperCase($input.countryCode ?? "NL"),
  amount: abs(toNumber($input.amount ?? "0"))
}
```

These patterns compose naturally — every sub-expression produces a value, and values flow through operators and functions without intermediate variables (unless you want readability via `let`).

13 Functions and Lambdas

Functions are the building blocks of UTL-X transformations. Lambdas — anonymous inline functions — are what make `map`, `filter`, `reduce`, and the entire functional programming model work. This chapter covers both, from basic syntax to advanced composition patterns.

13.1 Lambda Expressions

A lambda is an anonymous function — a piece of logic you pass to another function. If you've used arrow functions in JavaScript, closures in Kotlin, or lambdas in Python, you'll recognize the syntax immediately.

13.1.1 Basic Syntax

```
(parameter) -> expression
```

The parameter is on the left, the arrow `->` separates it from the body, and the body is a single expression that produces the result. No `return` keyword — the expression IS the return value.

```
// Double a number
(x) -> x * 2

// Extract a name
(user) -> user.name

// Format a greeting
(name) -> concat("Hello, ", name, "!")
```

13.1.2 Multiple Parameters

```
(x, y) -> x + y
(item, index) -> {position: index, value: item}
(accumulator, current) -> accumulator + current.price
```

13.1.3 No Parameters

```
() -> "constant value"
() -> now()
```

13.1.4 Lambdas in Action

Lambdas are rarely written on their own — they're passed to higher-order functions:

```
// map: transform each element
map([1, 2, 3], (x) -> x * 2)
// [2, 4, 6]

// filter: keep elements that match
filter($input.users, (u) -> u.active == true)
// [{name: "Alice", active: true}, ...]
```

```
// reduce: accumulate a result
reduce([10, 20, 30], 0, (sum, x) -> sum + x)
// 60

// sortBy: order by a computed value
sortBy($input.products, (p) -> p.price)
// [{price: 9.99, ...}, {price: 29.99, ...}, ...]
```

13.2 User-Defined Functions

For logic you want to name and reuse within a transformation, use `def`:

```
def calculateTax(amount, rate) = amount * rate / 100

def formatCurrency(amount, currency) =
  concat(currency, " ", toString(round(amount * 100) / 100))

{
  subtotal: $input.total,
  tax: calculateTax($input.total, 21),
  formatted: formatCurrency($input.total * 1.21, "EUR")
}
```

13.2.1 Function Definitions Are Expressions

A `def` binds a name to a lambda. These are equivalent:

```
// Named function
def double(x) = x * 2

// Lambda bound to a variable
let double = (x) -> x * 2
```

Both can be called as `double(5)` → `10`. The `def` form is more readable for named functions. The `let` form is useful when passing functions as arguments.

13.2.2 Recursive Functions

Functions can call themselves:

```
def factorial(n) =
  if (n <= 1) 1
  else n * factorial(n - 1)

factorial(5)   // 120
```

Use recursion sparingly — deep recursion can exhaust the stack. For most data processing, `map`, `filter`, and `reduce` are better choices.

13.3 Higher-Order Functions

Higher-order functions take other functions as arguments. UTL-X's standard library is built on this pattern. Here are the essential ones:

13.3.1 map — Transform Each Element

```
map(array, (element) -> newValue)
```

Produces a new array where each element is transformed:

```
map([1, 2, 3], (x) -> x * 2)
// [2, 4, 6]

map($input.employees, (emp) -> {
  fullName: concat(emp.firstName, " ", emp.lastName),
  salary: emp.salary
})
```

13.3.2 filter — Select Matching Elements

```
filter(array, (element) -> booleanCondition)
```

Keeps only elements where the lambda returns true:

```
filter($input.orders, (o) -> o.total > 100)

filter($input.users, (u) -> u.role == "admin" && u.active)
```

13.3.3 reduce — Accumulate a Result

```
reduce(array, initialValue, (accumulator, element) -> newAccumulator)
```

Combines all elements into a single value:

```
// Sum
reduce([10, 20, 30], 0, (sum, x) -> sum + x)
// 60

// Build a comma-separated string
reduce(["Alice", "Bob", "Charlie"], "", (acc, name) ->
  if (acc == "") name else concat(acc, ", ", name)
)
// "Alice, Bob, Charlie"

// Find maximum
reduce($input.scores, 0, (max, s) -> if (s > max) s else max)
```

13.3.4 find — First Match

```
find(array, (element) -> condition)
```

Returns the first element where the lambda returns true, or null if no match:

```
find($input.users, (u) -> u.email == "alice@example.com")
```

13.3.5 sortBy — Order by Computed Value

```
sortBy(array, (element) -> sortKey)
```

Sorts by the value returned by the lambda:

```
sortBy($input.employees, (e) -> e.lastName)     // alphabetical
sortBy($input.products, (p) -> p.price)          // cheapest first
sortBy($input.orders, (p) -> -p.total)           // most expensive first (negate)
```

13.3.6 groupBy — Group by Key

```
groupBy(array, (element) -> groupKey)
```

Creates a map where keys are the lambda results and values are arrays of matching elements:

```
groupBy($input.employees, (e) -> e.department)
// {"Engineering": [...], "Sales": [...], "Marketing": [...]}
```

See Chapter 9 for how `groupBy` is used for flat-to-hierarchical transformation.

13.4 Function Composition

13.4.1 Pipe Chains

The pipe operator (`|>`) chains functions into readable data flows:

```
$input.transactions
  |> filter((t) -> t.amount > 0)
  |> map((t) -> {
    date: t.date,
    amount: t.amount,
    category: t.category ?? "Uncategorized"
  })
  |> sortBy((t) -> t.date)
  |> map((t) -> concat(t.date, ": ", toString(t.amount), " (", t.category, ")"))
```

Read top to bottom: filter positive transactions, map to a clean structure, sort by date, format as strings.

13.4.2 Nested Functions

When pipes aren't suitable, functions nest naturally:

```
join(
  sort(unique(map($input.items, (i) -> i.category))),
  ", "
)
// "Electronics, Furniture, Kitchen"
```

Read inside out: map to categories, remove duplicates, sort, join with commas.

Note: this is the *string* `join()` function (2 parameters: array + separator). A different `join()` function for *data restructuring* (5 parameters: parents, children, parentKey, childKey, propertyName) is proposed as a future addition for flat-to-hierarchical transformation — see Chapter 9 and the F03 design document. The two functions are distinguished by their parameter count.

13.4.3 Combining map and filter

A very common pattern — filter first, then transform:

```
$input.employees
  |> filter((e) -> e.active && e.department == "Engineering")
  |> map((e) -> {
    name: concat(e.firstName, " ", e.lastName),
    level: e.seniorityLevel,
    yearsOfService: dateDiff(now(), parseDate(e.startDate, "yyyy-MM-dd"),
"years")
  })
```

Or transform first, then filter (when the filter condition depends on the transformation):

```
$input.orders
  |> map((o) -> {
    ...o,
    lineTotal: sum(map(o.lines, (l) -> l.qty * l.price))
  })
  |> filter((o) -> o.lineTotal > 1000)
```

13.5 Closures

Lambdas capture variables from their surrounding scope:

```
let threshold = 100
let currency = $input.settings.currency

map($input.items, (item) -> {
  name: item.name,
  price: item.price,
  currency: currency,                          // captured from outer scope
  expensive: item.price > threshold             // captured from outer scope
})
```

The lambda "closes over" `threshold` and `currency` — it remembers their values even though they were defined outside the lambda. This is standard closure behavior, familiar from JavaScript, Kotlin, Python, and every functional language.

13.6 Practical Patterns

13.6.1 Flatten and Restructure

```
// Input: array of objects with nested arrays
// Output: flat array of all nested items with parent context

$input.departments
  |> map((dept) ->
    map(dept.employees, (emp) -> {
      department: dept.name,
      employee: emp.name,
      salary: emp.salary
    })
  )
  |> flatten()
```

13.6.2 Aggregate with Multiple Results

```
let items = $input.order.items
{
  count: count(items),
  total: sum(map(items, (i) -> i.price * i.qty)),
  average: sum(map(items, (i) -> i.price)) / count(items),
  cheapest: sortBy(items, (i) -> i.price) |> first(),
  mostExpensive: sortBy(items, (i) -> -i.price) |> first(),
  categories: unique(map(items, (i) -> i.category))
}
```

13.6.3 Lookup Table Pattern

```
let countryNames = {
  "NL": "Netherlands",
  "DE": "Germany",
  "FR": "France",
  "BE": "Belgium"
}

map($input.addresses, (addr) -> {
  ...addr,
  countryName: countryNames[addr.countryCode] ?? addr.countryCode
})
```

13.6.4 Conditional Transformation

```
map($input.items, (item) ->
  if (item.type == "PRODUCT") {
```

```
    kind: "physical",
    name: item.name,
    weight: item.weight ?? 0,
    price: item.price
  } else if (item.type == "SERVICE") {
    kind: "digital",
    name: item.name,
    duration: item.hours ?? 1,
    rate: item.price
  } else {
    kind: "unknown",
    name: item.name,
    raw: item
  }
)
```

13.7 The 652 Standard Library Functions

The functions shown in this chapter — `map`, `filter`, `reduce`, `find`, `sortBy`, `groupBy`, `concat`, `sum`, `count`, `unique`, `flatten` — are just the beginning. UTL-X includes 652 functions across 18 categories.

The next chapter provides an overview organized by category. The complete reference with signatures and examples is in Part VIII (Chapter 48).

14 Standard Library Overview

UTL-X ships with 652 built-in functions organized in 18 categories. This chapter provides a guided tour — what each category offers, when you'd use it, and the most important functions in each. The complete reference with signatures and examples for every function is in Part VIII (Chapter 48).

You don't need to memorize 652 functions. You need to know *what categories exist* so you can search for the right function when you need it. The IDE's function library and `utlx functions --search` command make discovery easy.

14.1 String Functions (83 functions)

The largest category. Text manipulation is central to data transformation — field formatting, concatenation, parsing, validation.

14.1.1 Most-Used String Functions

Function	What it does	Example
concat(…)	Join strings	concat("Hello", " ", name)
upperCase(s)	UPPERCASE	upperCase("hello") → "HELLO"
lowerCase(s)	lowercase	lowerCase("HELLO") → "hello"
trim(s)	Remove whitespace	trim(" hello ") → "hello"
contains(s, sub)	Check substring	contains("hello", "ell") → true
startsWith(s, prefix)	Check prefix	startsWith("NL123", "NL") → true
replace(s, old, new)	Replace text	replace("2026-04-30", "-", "/")
split(s, delim)	Split to array	split("a,b,c", ",") → ["a","b","c"]
substring(s, start, len)	Extract part	substring("hello", 0, 3) → "hel"
length(s)	Character count	length("hello") → 5
matches(s, regex)	Regex test	matches("abc123", "[a-z]+[0-9]+") → true

14.1.2 Case Conversion

Beyond simple upper/lower, UTL-X supports naming convention conversions:

```
camelCase("order_line_item")     // "orderLineItem"
snakeCase("orderLineItem")      // "order_line_item"
kebabCase("orderLineItem")      // "order-line-item"
capitalize("hello world")       // "Hello world"
```

These are essential for API mapping — converting between Java-style camelCase, Python-style snake_case, and URL-style kebab-case.

14.1.3 Padding and Formatting

```
leftPad("42", 5, "0")           // "00042"  (invoice numbers)
rightPad("EUR", 5, " ")         // "EUR  "  (fixed-width output)
truncate("Long description...", 50)  // first 50 chars
```

14.2 Array Functions (67 functions)

The second-largest category. Arrays are everywhere — order lines, user lists, CSV rows, XML repeated elements.

14.2.1 Transform and Filter

```
map(items, (i) -> i.name)                    // transform each
filter(items, (i) -> i.active)               // keep matching
flatMap(items, (i) -> i.tags)                // map + flatten
```

14.2.2 Search and Access

```
find(items, (i) -> i.id == "X")              // first match or null
first(items)                                  // first element
last(items)                                   // last element
count(items)                                  // length
contains(items, "value")                      // is value in array?
```

14.2.3 Aggregate

```
sum(numbers)                                  // add all
avg(numbers)                                  // average
min(numbers)                                  // smallest
max(numbers)                                  // largest
reduce(items, init, (acc, x) -> ...)          // custom aggregation
```

14.2.4 Reshape

```
sort(items)                                   // natural order
sortBy(items, (i) -> i.name)                 // sort by field
reverse(items)                                // reverse order
unique(items)                                 // remove duplicates
flatten([[1,2],[3,4]])                        // [1,2,3,4]
groupBy(items, (i) -> i.category)            // group into map
take(items, 5)                                // first 5
drop(items, 5)                                // skip first 5
slice(items, 2, 7)                            // elements 2-6
chunk(items, 3)                               // groups of 3
zip(array1, array2)                           // pair elements
```

14.3 Math Functions (37 functions)

Numeric operations for financial calculations, statistics, and data processing.

```
abs(-42)              // 42
ceil(3.2)             // 4
floor(3.8)            // 3
round(3.456)          // 3
round(3.456 * 100) / 100  // 3.46 (2 decimal places)
sqrt(144)             // 12
```

```
pow(2, 10)            // 1024
min(10, 20)           // 10
max(10, 20)           // 20
```

For financial precision, use `round()` explicitly — UTL-X uses floating-point arithmetic, which can produce rounding artifacts (0.1 + 0.2 = 0.30000000000000004). Always round currency amounts: `round(amount * 100) / 100`.

14.4 Date/Time Functions (68 functions)

The third-largest category. Date handling is critical in integration — every message has timestamps, due dates, and validity periods.

14.4.1 Current Time

```
now()                 // 2026-04-30T10:30:00Z (with timezone)
today()               // 2026-04-30 (date only)
```

14.4.2 Parse and Format

```
parseDate("2026-04-30", "yyyy-MM-dd")          // parse string to date
formatDate(date, "dd/MM/yyyy")                  // format date to string
parseDate("30-04-2026", "dd-MM-yyyy")           // European format
formatDate(date, "MMMM d, yyyy")                // "April 30, 2026"
```

14.4.3 Arithmetic

```
addDays(date, 30)                               // 30 days later
addMonths(date, 3)                              // 3 months later
addYears(date, 1)                               // next year
dateDiff(date1, date2, "days")                  // days between dates
```

14.4.4 Extract Components

```
year(date)            // 2026
month(date)           // 4
day(date)             // 30
hour(datetime)        // 10
minute(datetime)      // 30
```

14.5 Type Functions (27 functions)

Check types and convert between them.

14.5.1 Type Checking

```
isString("hello")     // true
isNumber(42)          // true
isBoolean(true)       // true
isArray([1,2])        // true
```

```
isObject({a: 1})      // true
isNull(null)          // true
getType("hello")      // "string"
```

14.5.2 Type Conversion

```
toString(42)          // "42"
toNumber("42")        // 42
toBoolean("true")     // true
toNumber("not-a-num") // 0 (safe — returns 0 for unparseable input)
```

14.6 Encoding Functions (30 functions)

Encode and decode data for transport and storage.

```
base64Encode("hello")                          // "aGVsbG8="
base64Decode("aGVsbG8=")                       // "hello"
urlEncode("hello world")                       // "hello%20world"
urlDecode("hello%20world")                     // "hello world"
urlEncodeComponent("a=b&c=d")                  // "a%3Db%26c%3Dd"
hexEncode("hello")                             // "68656c6c6f"
hexDecode("68656c6c6f")                        // "hello"
```

14.7 Security Functions (16 functions)

Hashing and UUID generation. Part of the separate `stdlib-security` module.

```
md5("hello")                                   // "5d41402abc4b2a76..."
sha256("hello")                                // "2cf24dba5fb0a30e..."
hmacSha256("message", "secret-key")            // HMAC signature
uuid()                                         // random UUID v4
```

See Chapter 16 for security library details including encryption, digital signatures, and compliance considerations.

14.8 XML Functions (60 functions)

XML-specific operations for encoding, namespace handling, and encoding detection.

```
detectXMLEncoding(xmlString)                    // "UTF-8" or "ISO-8859-1"
convertXMLEncoding(xmlString, "UTF-8")          // re-encode
stripBOM(data)                                  // remove byte order mark
```

14.9 Other Categories

14.9.1 JSON Functions (6)

Parse and render JSON within transformations:

```
parseJson("{\"name\": \"Alice\"}")              // parse string to UDM
renderJson(object)                              // UDM to JSON string
```

14.9.2 CSV Functions (12)

Parse and render CSV:

```
parseCsv("name,age\nAlice,30")                 // parse to array of objects
renderCsv(arrayOfObjects)                       // to CSV string
```

14.9.3 YAML Functions (22)

Parse and render YAML:

```
parseYaml("name: Alice\nage: 30")              // parse to UDM
renderYaml(object)                              // to YAML string
```

14.9.4 Financial Functions (16)

Currency formatting and financial calculations:

```
formatCurrency(1234.56, "EUR")                 // "EUR 1,234.56"
parseAmount("1.234,56", "european")            // 1234.56
```

14.9.5 Geospatial Functions (8)

Distance calculations:

```
distance(52.37, 4.90, 48.86, 2.35)            // Amsterdam to Paris in km
```

14.9.6 Binary Functions (47)

Compression, byte manipulation:

```
compress(data, "gzip")                         // compress
decompress(data, "gzip")                       // decompress
```

14.9.7 Utility Functions (27)

General-purpose helpers:

```
range(1, 10)                                   // [1, 2, 3, ..., 9]
repeat("ab", 3)                                // "ababab"
coalesce(null, null, "found")                  // "found" (first non-null)
```

14.9.8 Object Functions (1)

```
keys(object)                                   // ["name", "age"]
values(object)                                 // ["Alice", 30]
```

14.10 Finding the Right Function

With 652 functions, discovery matters more than memorization.

14.10.1 In the CLI

```
utlx functions --search "date"        # find date-related functions
utlx functions --module string         # list all string functions
utlx functions                         # list all categories
```

14.10.2 In the IDE

The function library panel in VS Code lets you browse by category and search by name or description. Click a function to see its signature and an example.

14.10.3 In This Book

Chapter 48 (Part VIII) contains the complete reference — every function with its signature, parameter types, return type, description, and a runnable example. Use it as a desk reference.

14.11 A Note on Naming

UTL-X uses camelCase for all function names: `upperCase`, `lowerCase`, `parseDate`, `formatDate`, `toNumber`, `groupBy`, `sortBy`. This is consistent and predictable — if you know the concept, you can guess the name.

Common mistake: `lowercase` (wrong) → `lowerCase` (correct). The IDE catches this immediately with a red underline.

15 Security Library and Message Security

Security in data transformation is often an afterthought — until an audit finds credit card numbers in log files or patient data crossing a network boundary unencrypted. This chapter covers UTL-X's security functions, the practical decisions around message encryption in integration flows, and what should (and absolutely should NOT) happen with sensitive data during transformation.

15.1 The Security Library (stdlib-security)

UTL-X's security functions live in a separate module (`stdlib-security`) from the main standard library. This separation is deliberate — not every deployment needs cryptographic functions, and keeping them separate reduces the attack surface for environments where crypto is restricted.

15.1.1 Available Functions

Category	Functions	Use case
Hash	md5, sha1, sha256, sha512	Data integrity, checksums, fingerprinting
HMAC	hmacSha256, hmacSha512	Message authentication, API signing
UUID	uuid, uuidV4, uuidV5	Unique identifiers, correlation IDs
Base64	base64Encode, base64Decode	Binary-to-text encoding
Hex	hexEncode, hexDecode	Byte representation
URL	urlEncode, urlDecode, urlEncodeComponent	URL-safe encoding

15.1.2 Hashing

Hash functions produce a fixed-size fingerprint from arbitrary input. They're one-way — you can't reverse a hash to get the original data.

```
sha256("sensitive-data")
// "6f2c7b22f1c4e3a8d9f..."  (64 hex characters, always)

md5($input.document)
// "5d41402abc4b2a76..."     (32 hex characters)
```

Common uses in integration:

- **Deduplication:** hash the message content, compare to previously seen hashes
- **Integrity verification:** hash before sending, hash after receiving, compare
- **Anonymization:** hash a customer ID to create a pseudonymous identifier
- **Cache keys:** hash the transformation source + input to cache compiled results

15.1.3 HMAC (Hash-based Message Authentication)

HMAC combines a hash with a secret key — proving both integrity AND authenticity:

```
hmacSha256("message-body", "shared-secret-key")
// "a1b2c3d4..."
```

```
// Verify: receiver computes the same HMAC with the same key
// If they match, the message hasn't been tampered with
```

Common uses:
- **API authentication:** sign requests for webhook verification (Stripe, GitHub, Shopify)
- **Message integrity:** ensure messages haven't been modified in transit
- **Token generation:** create short-lived authentication tokens

15.1.4 UUID Generation

```
uuid()        // "f47ac10b-58cc-4372-a567-0e02b2c3d479" (random v4)
uuidV4()      // same as uuid()
```

Common uses:
- **Correlation IDs:** trace a message through a multi-step pipeline
- **Idempotency keys:** ensure a message is processed exactly once
- **Unique identifiers:** generate IDs for new records created by transformation

15.2 Transport Security: mTLS

Before discussing message-level security, let's be clear about what's already handled:

Transport Layer Security (TLS) encrypts the connection between two endpoints. Every message UTL-X receives over HTTPS is encrypted in transit. The message content is decrypted at the TLS termination point (the container's ingress) and is in cleartext inside the container.

Mutual TLS (mTLS) goes further — both client AND server present certificates, proving identity in both directions. Azure Container Apps supports mTLS. Dapr sidecars use mTLS automatically between services in Kubernetes.

For most integration scenarios, TLS/mTLS is sufficient:
- Messages are encrypted between endpoints
- Identity is verified (who sent this message?)
- No additional encryption needed inside the trusted network

15.3 Message-Level Encryption

15.3.1 When It's Required

Some regulations require encryption of the message *payload* itself — not just the connection:

- **PCI DSS:** cardholder data must be encrypted at rest and in transit
- **HIPAA:** protected health information (PHI) must be encrypted
- **NEN 7510:** Dutch healthcare information security standard
- **Government classified data:** often requires end-to-end encryption

15.3.2 The Encrypt-Transform-Re-encrypt Pattern

When message-level encryption is required, the transformation flow becomes:

```
Encrypted message arrives
    ↓ decrypt (UTLXe needs the decryption key)
    ↓ transform (cleartext in memory, briefly)
    ↓ encrypt (UTLXe needs the encryption key for the target)
Encrypted message sent
```

This works but adds complexity:

- UTLXe needs access to decryption AND encryption keys
- Key management: rotation, storage, access control
- Performance: decrypt + encrypt adds 1-5ms per message
- Debugging: encrypted messages cannot be inspected in logs or queues
- The message IS in cleartext in UTLXe's memory during transformation — this is unavoidable

15.3.3 Why This Is a Maintenance Burden

```
Keys to manage:           per source × per target = N × M keys
Certificate lifecycle:    each expires, must be renewed
Key rotation:             planned rotations, emergency rotations
Access control:           who can access which keys?
Audit:                    prove keys are handled correctly
Testing:                  tests need test keys (not production keys)
```

For 5 source systems and 3 target systems: 15 key pairs to manage. Each with its own rotation schedule. This is why most integration platforms rely on transport encryption (TLS) and access controls — message-level encryption is reserved for regulated data that truly requires it.

15.3.4 Recommendation

Use **transport encryption (TLS/mTLS)** for all communication. Add **message-level encryption** only when regulation specifically requires it (PCI DSS, HIPAA, classified data). Don't encrypt "just in case" — the operational cost is significant and usually unnecessary.

15.4 Digital Signatures and Integrity

15.4.1 When You Need Signatures

Digital signatures prove two things: **integrity** (the message hasn't been modified) and **authenticity** (the message came from a known sender). They don't encrypt — they verify.

Common in:

- **Peppol e-invoicing:** UBL invoices are digitally signed with XAdES
- **Banking (ISO 20022):** payment messages signed for non-repudiation
- **API webhooks:** HMAC signature in HTTP header (simpler than PKI)
- **JWT tokens:** signed claims for API authentication

15.4.2 Impact on Transformation

Critical rule: if you transform a digitally signed message, the signature becomes invalid. The signature covers the exact bytes of the original — any change (even whitespace) breaks it.

The correct flow:

1. Verify the signature (before transformation)
2. Transform the content (signature is now invalid)

3. Re-sign the output (with the sender's key for the target)

This means UTLXe must:
- Verify signatures on input (pre-validation step)
- Sign output (post-transformation step)
- Have access to the appropriate keys

15.4.3 What UTL-X Can Do Today

```
// HMAC verification (simple API webhooks)
let expectedSig = hmacSha256($input.body, "webhook-secret")
if (expectedSig != $input.headers.signature)
  error("Invalid webhook signature")

// Hash for integrity check
let contentHash = sha256(renderJson($input))
{...$input, integrityHash: contentHash}
```

15.4.4 What UTL-X Cannot Do Yet (Future)
- JWS (JSON Web Signature): sign and verify JWT tokens
- XMLDSig: verify XML digital signatures (XAdES for Peppol)
- AES encryption/decryption: symmetric encryption
- RSA sign/verify: asymmetric cryptography
- Key Vault integration: retrieve keys from Azure Key Vault, GCP Secret Manager

These are documented as future additions to the security library.

15.5 Large Files and Sensitive Data

15.5.1 The Claim Check Pattern
Sensitive large files (contracts, medical images, financial documents) should NOT travel through the message body. Use the **claim check pattern**:

1. Store the file in secure object storage (Azure Blob, S3, GCS) with encryption at rest
2. Put a **reference** in the message (URL, blob name, CMIS object ID)
3. UTL-X transforms the message metadata — not the file content
4. The consuming application retrieves the file from storage using the reference

```
// Transform the reference, not the file:
{
  invoiceId: $input.invoiceId,
  document: {
    storageType: "azure-blob",
    container: "invoices",
    blobName: concat($input.invoiceId, ".pdf"),
    url: concat("https://storage.blob.core.windows.net/invoices/",
$input.invoiceId, ".pdf")
  }
}
```

15.5.2 Avoid Base64 Embedding

Embedding binary content as Base64 in messages is an anti-pattern for sensitive data:

- 33% size increase (Base64 overhead)
- The entire file is in UTL-X memory during transformation
- The file content may appear in logs, error messages, queue storage
- No access control — anyone with the message has the file

For files under 50 KB (small images, digital signatures), Base64 is acceptable. For anything larger or sensitive, use the claim check pattern.

15.6 What NOT to Log

This topic is covered in depth in Chapter 36 (Logging and Compliance). The short version:

Never log: credit card numbers (PCI DSS), patient data (HIPAA/GDPR), passwords, API keys, session tokens, private keys.

Always log: transformation ID, timestamp, duration, success/failure, error field names (not values).

UTL-X's default logging (INFO level) logs no message content — only operational metadata. DEBUG and TRACE levels can expose sensitive data and must NEVER be enabled in production with sensitive workloads.

16 Pattern Matching and Control Flow

Most transformation logic is straightforward: map fields, apply functions, construct objects. But real-world integration always has conditional paths — different output structures based on message type, different calculations based on country, different handling for errors.

This chapter covers UTL-X's control flow mechanisms: `match` expressions for multi-way branching, `try/catch` for error recovery, and practical patterns for routing and polymorphic data.

16.1 Match Expressions

`match` is UTL-X's multi-way branch — like `switch` in Java or `when` in Kotlin, but more powerful because it supports value matching, guard clauses, and variable binding.

16.1.1 Basic Value Matching

```
match ($input.status) {
  "NEW" -> "Order received",
  "PROCESSING" -> "In progress",
  "SHIPPED" -> "On the way",
  "DELIVERED" -> "Completed",
  _ -> "Unknown status"
}
```

The `_` is the wildcard — it matches anything. Always include it as the last case to handle unexpected values. Without a wildcard, an unmatched value produces null.

16.1.2 Matching with Guards

Add conditions after the pattern with `if`:

```
match ($input.amount) {
  n if n > 10000 -> "high-value",
  n if n > 1000 -> "medium-value",
  n if n > 0 -> "standard",
  0 -> "zero",
  _ -> "negative"
}
```

The variable `n` is bound to the matched value — you can use it in the guard condition and in the result expression.

16.1.3 Complex Matching

Match can branch on any expression, not just simple values:

```
match (getType($input.data)) {
  "string" -> {type: "text", value: $input.data},
  "number" -> {type: "numeric", value: $input.data},
  "array" -> {type: "list", count: count($input.data)},
  "object" -> {type: "record", keys: keys($input.data)},
  _ -> {type: "unknown"}
}
```

16.1.4 Match vs If/Else Chains

For two or three conditions, `if/else` is fine:

```
if ($input.country == "NL") "domestic"
else if ($input.country == "BE" || $input.country == "DE") "neighboring"
else "international"
```

For more than three conditions, `match` is clearer:

```
match ($input.country) {
  "NL" -> "domestic",
  "BE" -> "neighboring",
  "DE" -> "neighboring",
  "FR" -> "EU-west",
  "IT" -> "EU-south",
  "PL" -> "EU-east",
  _ -> "international"
}
```

The rule of thumb: **two branches → if/else, three or more → match**.

16.2 Try/Catch — Error Recovery

Not every input is clean. Dates might be unparseable. Numbers might be text. Fields might be missing. `try/catch` lets you handle errors gracefully instead of failing the entire transformation.

16.2.1 Basic Try/Catch

```
try {
  parseDate($input.dateString, "yyyy-MM-dd")
} catch (e) {
  today()    // fallback: use today's date if parsing fails
}
```

The `try` block contains the expression that might fail. If it succeeds, the result is used. If it throws an error, the `catch` block runs instead.

16.2.2 Using the Error Variable

The `catch` variable `e` contains the error message:

```
{
  result: try {
    toNumber($input.price) * toNumber($input.quantity)
  } catch (e) {
    0    // fallback to zero
  },
  error: try {
    toNumber($input.price) * toNumber($input.quantity)
    null    // no error
  } catch (e) {
    e    // capture the error message
```

```
  }
}
```

16.2.3 Try/Catch in Map

A common pattern — transform an array, handling errors per element instead of failing the entire batch:

```
map($input.records, (record) -> {
  id: record.id,
  amount: try { toNumber(record.amount) } catch (e) { 0 },
  date: try { parseDate(record.date, "yyyy-MM-dd") } catch (e) { null },
  valid: try { toNumber(record.amount); true } catch (e) { false }
})
```

Each record is processed independently. A bad date in record 5 doesn't prevent records 1-4 and 6+ from being transformed correctly.

16.2.4 When to Use Try/Catch vs Null Coalescing

```
// Use ?? when the value might be null/missing:
$input.discount ?? 0

// Use try/catch when the operation might FAIL:
try { parseDate($input.date, "yyyy-MM-dd") } catch (e) { null }
```

`??` handles absence (null). `try/catch` handles failure (exceptions). Different problems, different tools.

16.3 Practical Control Flow Patterns

16.3.1 Message Type Routing

Integration messages often carry a type indicator. Route to different transformation logic based on type:

```
match ($input.messageType) {
  "ORDER" -> {
    type: "order",
    orderId: $input.data.orderId,
    customer: $input.data.customer,
    lines: $input.data.lines
  },
  "INVOICE" -> {
    type: "invoice",
    invoiceId: $input.data.invoiceId,
    total: $input.data.total,
    dueDate: $input.data.dueDate
  },
  "CREDIT_NOTE" -> {
    type: "credit",
```

```
    creditId: $input.data.creditId,
    amount: -abs($input.data.amount)
  },
  _ -> error(concat("Unknown message type: ", $input.messageType))
}
```

16.3.2 API Version Handling

Different API versions send different field names. Handle both:

```
{
  // v2 uses "fullName", v1 uses separate "firstName" + "lastName"
  name: if ($input.fullName != null) $input.fullName
        else concat($input.firstName ?? "", " ", $input.lastName ?? ""),

  // v2 uses ISO date, v1 uses US format
  date: try { parseDate($input.date, "yyyy-MM-dd") }
        catch (e) { try { parseDate($input.date, "MM/dd/yyyy") }
        catch (e2) { null } },

  // v2 includes "status", v1 doesn't
  status: $input.status ?? "ACTIVE"
}
```

16.3.3 Country-Specific Business Rules

```
let vatRate = match ($input.customer.country) {
  "NL" -> 21,
  "DE" -> 19,
  "FR" -> 20,
  "BE" -> 21,
  "LU" -> 17,
  "IE" -> 23,
  _ -> 0    // non-EU: reverse charge, 0% VAT
}

let isReverseCharge = vatRate == 0 && $input.customer.vatId != null

{
  subtotal: $input.total,
  vatRate: vatRate,
  vatAmount: if (isReverseCharge) 0 else $input.total * vatRate / 100,
  reverseCharge: isReverseCharge,
  total: if (isReverseCharge) $input.total else $input.total * (1 + vatRate /
100)
}
```

16.3.4 Data Cleansing with Fallbacks

```
{
  email: try {
```

```
    let e = lowerCase(trim($input.email))
    if (matches(e, "^[\\w.-]+@[\\w.-]+\\.[a-z]{2,}$")) e else null
  } catch (err) { null },

  phone: try {
    replace(replace(replace($input.phone ?? "", " ", ""), "-", ""), "+", "00")
  } catch (err) { null },

  amount: try {
    abs(toNumber(replace($input.amount ?? "0", ",", ".")))
  } catch (err) { 0 },

  country: upperCase(trim($input.countryCode ?? "")) ?? "XX"
}
```

16.3.5 Polymorphic Output (Different Structure Based on Input)

```
if (count($input.items) == 1) {
  // Single-item order: flat structure
  orderId: $input.orderId,
  product: $input.items[0].name,
  quantity: $input.items[0].qty,
  price: $input.items[0].price
} else {
  // Multi-item order: nested structure with summary
  orderId: $input.orderId,
  itemCount: count($input.items),
  items: map($input.items, (i) -> {name: i.name, qty: i.qty, price: i.price}),
  total: sum(map($input.items, (i) -> i.qty * i.price))
}
```

16.4 Control Flow Summary

Mechanism	Use when	Example
if/else	2-3 simple conditions	if (active) “yes” else “no”
ternary ? :	Inline condition	active ? “yes” : “no”
?? (nullish)	Default for missing value	name ?? “Unknown”
?. (safe nav)	Property might not exist	order?.customer?.name
match	3+ conditions or complex routing	match (type) { … }
try/catch	Operation might fail	try { parseDate(…) } catch { null }
error()	Deliberately fail	error(“Invalid input”)

The key insight: in UTL-X, every control flow construct is an *expression* that produces a value. There are no statements. This means you can use `if/else`, `match`, and `try/catch` anywhere a value is expected — inside objects, as function arguments, in array elements, nested in other expressions.

17 Schema Validation

A transformation that produces wrong output is worse than one that fails — because wrong output is sent downstream, corrupts data in the target system, and the error is discovered hours or days later. Schema validation catches these errors at transformation time, before they leave UTL-X.

This chapter covers UTL-X's validation capabilities: what schemas are supported, how the validation orchestrator works, and the difference between syntactical and semantic validation.

17.1 Why Validate?

Consider a Peppol e-invoicing flow. Your transformation maps Dynamics 365 JSON to UBL 2.1 XML. Without validation:

- A missing `cbc:InvoiceTypeCode` element → Peppol Access Point rejects the invoice hours later
- A wrong VAT calculation → the tax authority flags the invoice weeks later
- An invalid IBAN → the payment fails days later

With validation:

- Pre-validation catches input data issues before the transformation runs
- Post-validation catches mapping bugs before the output leaves UTL-X
- The error message tells you exactly which field, which constraint, what's wrong

Validation is the difference between "it compiled and ran" and "it produced correct output."

17.2 Supported Schema Formats

UTL-X includes seven schema validators — one for each schema format:

Validator	Schema format	Validates	Standard
JsonSchemaValidator	JSON Schema	JSON data	Draft-04 through 2020-12
XsdValidator	XSD	XML data	W3C XSD 1.0 / 1.1
AvroSchemaValidator	Avro Schema	Avro-compatible data	Apache Avro 1.11
ProtobufValidator	Protobuf	Proto3-compatible data	Protocol Buffers 3
TableSchemaValidator	Table Schema (TSCH)	Tabular / CSV data	Frictionless Data
ODataSchemaValidator	OData Schema (OSCH)	OData entities	OASIS OData v4
YamlSchemaValidator	YAML via JSON Schema	YAML data	YAML validated as JSON

Each validator understands the schema format's native constraint language. JSON Schema's `required`, `pattern`, `minimum`. XSD's `minOccurs`, `xs:pattern`, `xs:restriction`. Avro's field defaults and unions. They're not wrappers — they validate against the real schema semantics.

17.3 The Validation Orchestrator

UTLXe's `ValidationOrchestrator` runs validation in a sandwich pattern around the transformation:

```
┌──────────────────────────────────────────────────────┐
│ 1. PRE-VALIDATION                                    │
│    Validate input data against input schema          │
│    → Fail fast if input is invalid                   │
├──────────────────────────────────────────────────────┤
│ 2. TRANSFORMATION                                    │
│    Execute the .utlx transformation                  │
├──────────────────────────────────────────────────────┤
│ 3. POST-VALIDATION                                   │
│    Validate output data against output schema        │
│    → Catch mapping bugs before output leaves         │
└──────────────────────────────────────────────────────┘
```

If pre-validation fails, the transformation never runs — saving compute time and preventing cascading errors from bad input. If post-validation fails, the output is rejected — the mapping has a bug that needs fixing.

17.3.1 Configuring Validation

Validation is configured per transformation. Today this is done via the engine's `TransformConfig`:

```yaml
# transform.yaml
strategy: TEMPLATE
validationPolicy: STRICT
inputs:
  - name: input
    schema: "order-input.json"
output:
  schema: "invoice-output.xsd"
```

A future enhancement (F01) will allow inline schema declaration in the .utlx header:

```
%utlx 1.0
input json {schema: "order-input.json"}
output xml {schema: "invoice-output.xsd"}
---
// transformation body
```

Both approaches use the same validators and orchestrator — the difference is where the schema reference lives.

17.4 Validation Policies

Not every validation failure should stop the transformation. The policy controls what happens:

Policy	On failure	Use when

STRICT	Transformation fails with error	Production — contract enforcement
WARN	Warning logged, transformation continues	Migration — monitor before enforcing
SKIP	Schema loaded but not validated	Documentation only

Start with `WARN` during development (see what would fail). Switch to `STRICT` in production (enforce the contract).

17.5 JSON Schema Validation

JSON Schema is the most commonly used validator — it validates JSON, YAML, and OData data.

17.5.1 What JSON Schema Checks

```
{
  "type": "object",
  "required": ["orderId", "customer", "total"],
  "properties": {
    "orderId": {"type": "string", "pattern": "^ORD-[0-9]{3,}$"},
    "customer": {"type": "string", "minLength": 1},
    "total": {"type": "number", "minimum": 0},
    "currency": {"type": "string", "enum": ["EUR", "USD", "GBP"]},
    "lines": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "object",
        "required": ["product", "qty"],
        "properties": {
          "product": {"type": "string"},
          "qty": {"type": "integer", "minimum": 1}
        }
      }
    }
  }
}
```

This schema enforces: orderId is required and matches a pattern, customer is a non-empty string, total is a non-negative number, currency is one of three values, lines is a non-empty array of objects with required product and qty fields.

17.5.2 Validation Errors

When validation fails, the error message pinpoints the problem:

```
Validation error at $.customer: required field missing
Validation error at $.total: -50 is less than minimum 0
Validation error at $.orderId: "INV-001" does not match pattern "^ORD-[0-9]{3,}$"
Validation error at $.lines[2].qty: "five" is not of type integer
```

Each error includes the JSON Path to the failing field, the constraint that was violated, and the actual value. This makes debugging fast — you know exactly what's wrong and where.

17.6 XSD Validation

XSD validation is essential for enterprise XML — UBL invoices, HL7 messages, ISO 20022 payments all ship with XSD schemas.

```
%utlx 1.0
input json
output xml {schema: "UBL-Invoice-2.1.xsd"}
---
// If the output doesn't match the UBL XSD, transformation fails
```

XSD checks: element names, types, cardinality (minOccurs/maxOccurs), patterns, enumerations, namespace compliance. The JDK's built-in `javax.xml.validation` handles XSD 1.0; XSD 1.1 support uses the same API.

17.7 Syntactical vs Semantic Validation

Everything described so far is **syntactical validation** — checking the structure, types, and constraints defined in a schema.

17.7.1 What Syntactical Validation Catches

- Is this field a string? A number? A date?
- Is this field present (required)?
- Does this string match a pattern (regex)?
- Is this number within range (min/max)?
- Is this array the right length?
- Does this element have the right children (XSD structure)?

17.7.2 What Syntactical Validation Cannot Catch

- Does the invoice total equal the sum of line items?
- If the country is NL, does the VAT ID start with "NL"?
- Is the end date after the start date?
- Is this IBAN valid (checksum)?
- Can an order move from SHIPPED back to PENDING?

These are **semantic** rules — they involve business logic, cross-field relationships, and domain knowledge. No structural schema can express them.

17.7.3 The Gap

UTL-X currently supports syntactical validation (7 schema validators). Semantic validation — business rules expressed in languages like Schematron, FEEL, or custom assertions — is a planned future capability. Chapter 38 covers the semantic validation vision in detail, including the proposed `assert` syntax:

```
// Future syntax (not yet implemented):
validate {
  assert sum($output.lines.total) == $output.grandTotal
```

```
  assert $output.issueDate < $output.dueDate
  assert if ($output.country == "NL") matches($output.vatId, "NL[0-9]{9}B[0-9]
{2}")
}
```

For now, semantic rules must be expressed as part of the transformation logic — using `if/else`, `match`, or `try/catch` to check conditions and produce errors or warnings.

17.8 Validation in Practice

17.8.1 Development Workflow

1. Write your transformation
2. Add an output schema (JSON Schema or XSD)
3. Set policy to WARN
4. Run with sample data — check which fields fail validation
5. Fix the transformation until all warnings clear
6. Switch to STRICT for production

17.8.2 Production Workflow

1. Message arrives
2. Pre-validate against input schema → reject bad input immediately
3. Transform
4. Post-validate against output schema → catch mapping bugs
5. Send output — guaranteed to match the contract

17.8.3 When to Skip Validation

- **Performance-critical pipelines:** validation adds 1-5ms per message. At 86K msg/s, this matters. Skip validation when the input source is trusted and the transformation is well-tested.
- **Pass-through transformations:** when `\$input` is the entire body (no field mapping), validation of the output is the same as validation of the input — redundant.
- **Development/exploration:** when experimenting with data, validation errors slow you down. Use SKIP until the mapping stabilizes.

17.9 Validation and the Conformance Suite

The UTL-X conformance suite (453+ tests) validates transformation correctness — not schema compliance. These are different concerns:

- **Conformance tests:** "Does this transformation produce the expected output?" (functional correctness)
- **Schema validation:** "Does this output match the contract?" (structural compliance)

Both are important. Conformance tests catch regressions. Schema validation catches contract violations. Use both in production.

18 Pipeline Chaining and Multi-Hop Transformations

18.1 What Is Pipeline Chaining?

18.2 Why Pipelines?

18.3 Single-Hop vs Multi-Hop

18.3.1 Single-Hop (Simple Transformation)

18.3.2 Multi-Hop (Pipeline)

18.4 Multi-Hop with Additional Inputs per Stage

18.4.1 The Problem

18.4.2 The Solution: Additional Inputs per Hop

18.4.3 When Is Multi-Hop with Additional Inputs Handy?

18.4.3.1 1. Data Enrichment Pipelines

18.4.3.2 2. Multi-System Integration (Hub-and-Spoke)

18.4.3.3 3. Validation with External Reference Data

18.4.3.4 4. Template-Based Output Generation

18.4.3.5 5. Configuration-Driven Transformations

18.5 Defining Pipelines

18.5.1 Pipeline via UTLXe HTTP API

18.5.2 Pipeline via CLI

18.6 Multi-Input Transformations (Single Step)

18.7 Pipeline Patterns

18.7.1 Enrichment Pipeline

18.7.2 Validation Sandwich

18.7.3 Fan-Out

18.7.4 Fan-In (Aggregation)

18.7.5 Conditional Routing

18.8 Performance Characteristics

18.9 Status: Multi-Hop Additional Inputs

19 Data Restructuring: Flat, Hierarchical, and Back

Real-world integration is rarely a simple field-to-field mapping. The source data is structured one way, the target expects it another way. Orders arrive flat, the target wants them nested. Nested data needs to become CSV. Sequential segments need grouping by header. A customer ID needs to become a customer name.

This chapter covers the complete toolkit for restructuring data in UTL-X: from flat to hierarchical, from hierarchical to flat, and everything in between.

19.1 The Six Restructuring Patterns

Pattern	Direction	Function	Status
Parent-child nesting	Flat → Hierarchical	join()	Proposed (F03)
Record enrichment	Flat → Enriched	lookup()	Proposed (F04)
Sequential grouping	Flat → Grouped	chunkBy()	Proposed (F05)
Denormalization	Hierarchical → Flat	unnest()	Proposed (F06)
Key-based grouping	Flat → Map	groupBy()	Exists
Array flattening	Nested → Flat	flatten()	Exists

19.2 Pattern 1: Flat → Hierarchical (join)

The most common integration challenge: you receive flat records with key references, and the target expects nested parent-child structures.

19.2.1 The Problem

Orders and order lines arrive as separate flat arrays. Each line references its parent order by key:

```
{
  "orders":     [{"orderId": "A", "customer": "Acme"}, {"orderId": "B",
"customer": "Globex"}],
  "orderLines": [{"orderId": "A", "product": "Widget"}, {"orderId": "A",
"product": "Gadget"}, {"orderId": "B", "product": "Gizmo"}]
}
```

Target expects nested:

```
[{"orderId": "A", "customer": "Acme", "lines": [{"product": "Widget"},
{"product": "Gadget"}]},
 {"orderId": "B", "customer": "Globex", "lines": [{"product": "Gizmo"}]}]
```

19.2.2 The Solution: join()

```
join(
  \$input.orders,                    // parent array
  \$input.orderLines,                // child array
  (order) -> order.orderId,          // parent key
  (line) -> line.orderId,            // child key
```

```
  "lines"                           // property name for nested children
)
```

The 5th parameter `"lines"` is a string that tells `join()` what to name the new property it creates on each parent. After this call, each order has a `.lines` property containing its matched order lines:

```
let enrichedOrders = join(...)

// Now use order.lines like any property:
map(enrichedOrders, (order) -> {
  orderId: order.orderId,
  lineCount: count(order.lines),
  total: sum(map(order.lines, (l) -> l.qty * l.price))
})
```

19.2.3 Today's Workaround (without join)

Until `join()` is implemented, use `filter()` per parent:

```
map(\$input.orders, (order) -> {
  orderId: order.orderId,
  customer: order.customer,
  lines: filter(\$input.orderLines, (l) -> l.orderId == order.orderId)
})
```

This works but is O(N times M) — for 500 orders and 5,000 lines, that's 2.5 million comparisons. `join()` does it in 5,500 operations.

19.3 Pattern 2: Record Enrichment (lookup)

You have a record with a reference key and need to add fields from a lookup table — not nest the entire matching record, just extract specific fields.

19.3.1 The Problem

An order has `customerId` but needs `customerName`:

```
{"orderId": "ORD-001", "customerId": "C-42", "total": 299.99}
```

Customer table:

```
[{"id": "C-42", "name": "Acme Corp", "country": "NL"}]
```

You want: `{orderId: "ORD-001", customerName: "Acme Corp", country: "NL"}` — not a nested customer object.

19.3.2 The Solution: lookup()

```
let customer = lookup(order.customerId, \$input.customers, (c) -> c.id)

{
  orderId: order.orderId,
  total: order.total,
  customerName: customer?.name ?? "Unknown",
  country: customer?.country
}
```

`lookup()` finds the first matching record and returns it. You then pick the fields you need with dot notation. Use `?.` for safety in case no match is found.

19.3.3 Today's Workaround (without lookup)

Use `find()`:

```
let customer = find(\$input.customers, (c) -> c.id == order.customerId)
```

Identical result. `lookup()` adds the potential for internal caching when used repeatedly inside `map()`.

19.4 Pattern 3: Sequential Grouping (chunkBy)

For data where parent-child is determined by position, not by key fields.

19.4.1 The Problem

SAP IDoc segments arrive in sequence — headers and lines interleaved:

```
E1EDK01  (header)    ← order 1 starts
E1EDP01  (line)      ← belongs to order 1
E1EDP01  (line)      ← belongs to order 1
E1EDK01  (header)    ← order 2 starts
E1EDP01  (line)      ← belongs to order 2
```

No key connects lines to headers. Position is the only relationship: "everything between one header and the next belongs together."

19.4.2 The Solution: chunkBy()

```
let groups = chunkBy(\$input.segments, (seg) -> seg.type == "E1EDK01")

map(groups, (chunk) -> {
  header: chunk[0],
  lines: filter(drop(chunk, 1), (seg) -> seg.type == "E1EDP01")
})
```

`chunkBy()` splits the flat sequence into groups. A new group starts whenever the lambda returns true. Each group is an array where the first element is the header and the rest are its children.

19.4.3 Today's Workaround (without chunkBy)

Use `reduce()` with manual state tracking — possible but extremely verbose and error-prone. `chunkBy()` turns 15 lines of reduce logic into a single function call.

19.5 Pattern 4: Denormalization (unnest)

The reverse of `join()` — take nested data and flatten it. Essential for producing CSV, database rows, or flat file output.

19.5.1 The Problem

Hierarchical order data:

```
[{"orderId": "A", "customer": "Acme", "lines": [
    {"product": "Widget", "qty": 10},
    {"product": "Gadget", "qty": 5}
]}]
```

Target is flat CSV:

```
orderId,customer,product,qty
A,Acme,Widget,10
A,Acme,Gadget,5
```

Each parent field must be repeated for every child row.

19.5.2 The Solution: unnest()

```
%utlx 1.0
input json
output csv
---
unnest(\$input.orders, "lines")
```

One function call. The `"lines"` parameter tells `unnest()` which nested array to expand. Parent fields (orderId, customer) are automatically repeated for each child. The `lines` property itself is removed from the output.

For multi-level flattening, chain:

```
\$input.orders |> unnest("lines") |> unnest("schedules")
```

19.5.3 Today's Workaround (without unnest)

```
flatten(map(\$input.orders, (order) ->
  map(order.lines, (line) -> {
    orderId: order.orderId,
    customer: order.customer,
    ...line
  })
))
```

Works but requires manually listing parent fields or using spread (which includes the `lines` array itself, causing unwanted nesting in the output).

19.6 Pattern 5: Key-Based Grouping (groupBy — exists)

Group flat records into a map by key. Already available in UTL-X.

```
groupBy(\$input.employees, (e) -> e.department)
// {"Engineering": [{...}, {...}], "Sales": [{...}], "Marketing": [{...}]}
```

Use `groupBy` when you want a map (key → array) rather than a nested hierarchy. Common for: aggregation per group, partitioning data by category, building lookup indexes.

> **Known Issue (B15):** `groupBy()` currently cannot be indexed with a UDM value — `groups[order.orderId]` fails with "Index must be a number or string." This bug blocks the efficient groupBy + index pattern. Use `filter()` as a workaround until B15 is fixed.

19.7 Pattern 6: Array Flattening (flatten — exists)

Unwrap one level of nested arrays. Already available in UTL-X.

```
flatten([[1, 2], [3, 4], [5, 6]])
// [1, 2, 3, 4, 5, 6]
```

Use `flatten` when `map()` produces arrays and you want a single flat array instead of an array of arrays. Common pattern: `map` + `flatten` = `flatMap`.

19.8 Combining Patterns

Real integrations often combine multiple patterns:

19.8.1 IDoc → Enriched Hierarchical JSON

```
// 1. Group sequential segments (chunkBy)
let orderGroups = chunkBy(\$input.segments, (seg) -> seg.type == "E1EDK01")

// 2. Structure into parent-child (map)
let orders = map(orderGroups, (chunk) -> {
  ...chunk[0],
  lines: filter(drop(chunk, 1), (seg) -> seg.type == "E1EDP01")
})

// 3. Enrich with customer data (lookup)
map(orders, (order) -> {
  let customer = lookup(order.KUNNR, \$input.customers, (c) -> c.id)

  orderId: order.BELNR,
  customer: customer?.name ?? "Unknown",
  country: customer?.country,
  lines: map(order.lines, (line) -> {
    product: line.MATNR,
```

```
    quantity: toNumber(line.MENGE)
  })
})
```

Three patterns combined: `chunkBy` (sequential → grouped), `map` (grouped → hierarchical), `lookup` (enrichment).

19.8.2 Hierarchical → Flat CSV Report

```
// 1. Unnest orders → lines
let flat = unnest(\$input.orders, "lines")

// 2. Enrich with product descriptions (lookup)
map(flat, (row) -> {
  let product = lookup(row.productCode, \$input.catalog, (p) -> p.code)

  ...row,
  productDescription: product?.description ?? row.productCode,
  weight: product?.weight ?? 0
})

// Output as CSV
```

Two patterns: `unnest` (hierarchical → flat), `lookup` (enrichment).

19.9 Choosing the Right Pattern

I have / I need	Use
Flat records with keys → nested parent-child	join()
Record with ID → add fields from reference table	lookup() or find()
Sequential segments → groups by header	chunkBy()
Nested data → flat rows for CSV/database	unnest()
Flat records → map of key → array	groupBy()
Array of arrays → single flat array	flatten()
Conditional children → nest only matching	filter() then join()
Many-to-many via bridge table	Two join() calls through bridge
Composite key matching	concat() key then join() or lookup()

Part III

Formats Deep Dive

Working with XML, JSON, CSV, YAML, OData, and schema formats

20 XML Transformations

XML is the most feature-rich format UTL-X handles — and the most common in enterprise integration. SOAP services, UBL invoices, HL7 healthcare messages, SAP IDocs, SWIFT payments, government data — XML dominates the enterprise world.

This chapter covers everything specific to XML: how to read elements and attributes, how to write XML output with namespaces, how the parser handles edge cases, and real-world patterns from Peppol, FHIR, and SWIFT.

20.1 Reading XML

20.1.1 Element Access

XML elements map to UDM Object properties. Access them with dot notation:

```
<Order>
  <Customer>Alice Johnson</Customer>
  <Total>299.99</Total>
</Order>
```

```
\$input.Order.Customer       // "Alice Johnson"
\$input.Order.Total          // 299.99 (auto-detected as number)
```

UTL-X auto-unwraps text content — you get the value directly, not a wrapper object. This is the B13/B14 behavior described in Chapter 9.

20.1.2 Attribute Access

XML attributes use the `\@` prefix:

```
<Product id="P-001" price="29.99" currency="EUR">Widget</Product>
```

```
\$input.Product.@id          // "P-001"
\$input.Product.@price       // "29.99" (string — attributes are always strings)
\$input.Product.@currency    // "EUR"
\$input.Product              // "Widget" (text content, auto-unwrapped)
```

Note: attribute values are always strings. Use `toNumber()` for numeric attributes:

```
toNumber(\$input.Product.@price)    // 29.99 (number)
```

20.1.3 Repeated Elements

When an XML element appears multiple times, UTL-X automatically creates an array:

```
<Items>
  <Item>Widget</Item>
  <Item>Gadget</Item>
```

```
  <Item>Gizmo</Item>
</Items>
```

```
\$input.Items.Item           // ["Widget", "Gadget", "Gizmo"] (array)
\$input.Items.Item[0]        // "Widget"
count(\$input.Items.Item)    // 3
```

20.1.4 The Single-Element Problem

If there's only ONE `Item`, it's parsed as a single object — not an array of one:

```
<Items>
  <Item>Widget</Item>
</Items>
```

```
\$input.Items.Item           // "Widget" (NOT ["Widget"])
```

This can break code that expects an array. Solution: use array hints or always wrap with `flatten([...])`:

```
// Safe: always treat as array
let items = if (isArray(\$input.Items.Item)) \$input.Items.Item else [\
$input.Items.Item]
map(items, (i) -> ...)
```

The XML parser supports array hints — element names that should always be treated as arrays, even with a single occurrence. This is configured at the engine level.

20.1.5 Nested Access

Deep access chains naturally:

```
\$input.Invoice.AccountingSupplierParty.Party.PartyName.Name
\$input.Bundle.entry[0].resource.Patient.name[0].family
```

Use safe navigation for optional elements:

```
\$input.Invoice?.AccountingCustomerParty?.Party?.Contact?.ElectronicMail
```

20.1.6 Namespace-Prefixed Elements

XML with namespace prefixes:

```
<cac:AccountingSupplierParty>
  <cac:Party>
    <cbc:Name>Windmill Trading B.V.</cbc:Name>
  </cac:Party>
</cac:AccountingSupplierParty>
```

Access with the full prefixed name:

```
\$input["cac:AccountingSupplierParty"]["cac:Party"]["cbc:Name"]
```

Bracket notation is required because `:` is not valid in dot notation. This is common in UBL, FHIR, and XBRL documents.

20.2 Writing XML

20.2.1 Basic Element Output

When `output xml` is declared, objects become elements:

```
%utlx 1.0
input json
output xml
---
{
  Order: {
    Customer: \$input.name,
    Total: \$input.amount
  }
}
```

Produces:

```
<?xml version="1.0" encoding="UTF-8"?>
<Order>
  <Customer>Alice Johnson</Customer>
  <Total>299.99</Total>
</Order>
```

20.2.2 Creating Attributes

Use `\@` prefix in property names:

```
{
  Product: {
    @id: "P-001",
    @currency: "EUR",
    _text: \$input.description
  }
}
```

Produces:

```
<Product id="P-001" currency="EUR">Widget</Product>
```

The `_text` property sets the text content when the element also has attributes. Without attributes, text content is set directly: `{Product: "Widget"}`.

20.2.3 Self-Closing Elements

Empty objects with attributes produce self-closing elements:

```
{Item: {@sku: "W-01", @price: "50.00", @qty: "2"}}
```

Produces:

```
<Item sku="W-01" price="50.00" qty="2"/>
```

20.2.4 Namespace Declarations

Add XML namespace declarations as attributes:

```
{
  Invoice: {
    "@xmlns": "urn:oasis:names:specification:ubl:schema:xsd:Invoice-2",
    "@xmlns:cbc":
"urn:oasis:names:specification:ubl:schema:xsd:CommonBasicComponents-2",
    "cbc:ID": \$input.invoiceNumber,
    "cbc:IssueDate": \$input.date
  }
}
```

Produces:

```
<Invoice xmlns="urn:oasis:names:specification:ubl:schema:xsd:Invoice-2"
xmlns:cbc="urn:oasis:names:specification:ubl:schema:xsd:CommonBasicComponents-2">
  <cbc:ID>INV-2026-001</cbc:ID>
  <cbc:IssueDate>2026-04-30</cbc:IssueDate>
</Invoice>
```

20.2.5 Encoding

Control output encoding:

```
output xml {encoding: "UTF-8"}      // default
output xml {encoding: "ISO-8859-1"} // legacy systems
```

20.3 XML Encoding Detection and Conversion

UTL-X can detect and convert between XML encodings:

```
detectXMLEncoding(\$input)                        // "UTF-8" or "ISO-8859-1"
convertXMLEncoding(\$input, "UTF-8")              // re-encode to UTF-8
```

This is essential when integrating with legacy systems that use ISO-8859-1 or Windows-1252. UTL-X reads any encoding and outputs in the declared encoding.

20.4 XML Attributes in JSON/YAML Output

When XML is transformed to JSON or YAML, attributes need special handling because JSON and YAML have no concept of attributes. Chapter 21 covers this in detail. The short version:

- Non-leaf element attributes: preserved as `\@key` properties (always)
- Leaf element attributes: dropped by default, preserved with `{writeAttributes: true}`
- Self-closing element attributes: preserved (no text to unwrap)

20.5 Common XML Patterns

20.5.1 SOAP Envelope Parsing

```
// Strip the SOAP envelope, extract the business payload:
let body = \$input["soap:Envelope"]["soap:Body"]
let payload = body[keys(body)[0]]    // first child of Body = the operation
result
{
  ...payload
}
```

20.5.2 UBL Invoice (Peppol BIS 3.0)

```
let inv = \$input.Invoice
{
  invoiceId: inv["cbc:ID"],
  issueDate: inv["cbc:IssueDate"],
  currency: inv["cbc:DocumentCurrencyCode"],
  supplier: inv["cac:AccountingSupplierParty"]["cac:Party"]["cac:PartyName"]
["cbc:Name"],
  customer: inv["cac:AccountingCustomerParty"]["cac:Party"]["cac:PartyName"]
["cbc:Name"],
  total: toNumber(inv["cac:LegalMonetaryTotal"]["cbc:PayableAmount"]),
  lines: map(inv["cac:InvoiceLine"], (line) -> {
    lineId: line["cbc:ID"],
    description: line["cac:Item"]["cbc:Name"],
    quantity: toNumber(line["cbc:InvoicedQuantity"]),
    unitPrice: toNumber(line["cac:Price"]["cbc:PriceAmount"])
  })
}
```

20.5.3 HL7 FHIR XML

FHIR uses `value` attributes instead of text content (see FHIR analysis in the architecture docs):

```
let patient = \$input.Patient
{
  resourceType: "Patient",
  id: patient.id.@value,
  name: patient.name.family.@value,
  birthDate: patient.birthDate.@value,
  active: toBoolean(patient.active.@value)
}
```

The `\@value` accessor handles FHIR's convention naturally — no special FHIR mode needed.

20.5.4 SAP IDoc XML

SAP exports IDocs as flat XML segments (see Chapter 20 on data restructuring for the grouping pattern):

```
// Simple field extraction from IDoc:
{
  orderNumber: \$input.IDOC.E1EDK01.BELNR,
  customer: \$input.IDOC.E1EDK01.KUNNR,
  currency: \$input.IDOC.E1EDK01.WAERK,
  lines: map(\$input.IDOC.E1EDP01, (line) -> {
    material: line.MATNR,
    quantity: toNumber(line.MENGE),
    price: toNumber(line.NETPR)
  })
}
```

20.6 XML Round-Trip

XML → UDM → XML preserves structure, attributes, namespaces, and text content. The round-trip is not byte-for-byte identical (whitespace and formatting may differ) but is semantically equivalent.

What IS preserved: element names, element order, attributes, attribute values, text content, namespaces, CDATA content.

What is NOT preserved: insignificant whitespace, comment nodes, processing instructions, DTD declarations, entity references (resolved during parsing).

For most integration use cases, this is correct behavior — you care about the data, not the formatting.

21 XML Attribute Handling: Design Decisions

21.1 The Fundamental Problem

21.2 How UTL-X Handles Attributes Internally (UDM)

21.3 The _text Convention

21.4 The writeAttributes Option

21.5 What Gets Kept vs Dropped (Default)

21.6 Why Drop by Default? (DataWeave-Compatible)

21.7 JSON vs YAML Consistency

21.8 Industry Conventions Reference

21.9 FHIR XML: The value Attribute Convention

22 JSON Transformations

22.1 JSON in UTL-X

22.2 Reading JSON

22.3 Writing JSON

22.4 JSON-Specific Functions

22.5 Common JSON Patterns

22.6 JSON Lines (JSONL)

23 CSV Transformations

23.1 CSV in UTL-X

23.2 CSV Options

23.3 Reading CSV

23.4 Writing CSV

23.5 CSV-Specific Functions

23.6 Common CSV Patterns

23.7 Regional Formats

24 YAML Transformations

24.1 YAML in UTL-X

24.2 Reading YAML

24.3 Writing YAML

24.4 YAML Output Options

24.5 YAML-Specific Functions

24.6 Common YAML Patterns

24.7 YAML ↔ JSON Conversion

25 OData Transformations

25.1 What Is OData?

25.2 OData in UTL-X

25.3 Reading OData

25.4 Writing OData

25.5 OData-Specific Functions

25.6 Common OData Patterns

25.6.1 Dynamics 365 Integration

25.6.2 SharePoint Integration

25.6.3 SAP OData Integration

25.6.4 Power Platform

25.7 OData Schema (EDMX/CSDL)

25.8 OData ↔ Other Formats

26 Schema Formats

26.1 XSD (XML Schema Definition)

26.2 JSON Schema

26.3 Avro Schema

26.4 Protobuf (proto3)

26.5 OSCH (OData Schema / EDMX)

26.6 TSCH (Table Schema)

26.7 USDL (Universal Schema Definition Language)

27 XSD Design Patterns Deep Dive

27.1 The Eight XML Schema Design Patterns

XML Schema (XSD) can be structured in fundamentally different ways. Understanding these patterns is essential for working with enterprise XML — and for understanding why schema-to-schema mapping is challenging.

27.1.1 1. Russian Doll

27.1.2 2. Salami Slice

27.1.3 3. Venetian Blind

27.1.4 4. Garden of Eden

27.1.5 5. Swiss Army Knife (Hybrid)

27.1.6 6-8. Chameleon, Abstract, Adapter

27.2 Pattern Detection in UTL-X

27.3 Pattern Conversion in UTL-X

27.4 XSD Patterns in Cross-Format Mapping

28 Cross-Format Patterns

28.1 The Power of Format Agnosticism

28.2 XML ↔ JSON Conversion

28.3 CSV ↔ JSON Conversion

28.4 XML → CSV (Flattening)

28.5 JSON → XML (Structuring)

28.6 Multi-Format Pipelines

28.7 Data Normalization Patterns

Part IV

Real-World Applications

Enterprise integration, cloud deployment, and production patterns

29 Enterprise Integration

29.1 UTL-X in Integration Architecture

29.2 Use Case: European E-Invoicing (Peppol/UBL)

29.3 Use Case: Healthcare (HL7 FHIR)

29.4 Use Case: Financial Services (SWIFT/ISO 20022)

29.5 Use Case: Retail/E-Commerce

29.6 Message Broker Integration

29.7 Error Handling in Production

30 Engine Lifecycle: Design-Time, Init-Time, Runtime

30.1 The Three Phases

Every UTL-X transformation goes through three distinct phases. Understanding them is key to performance tuning, debugging, and choosing the right execution strategy.

30.1.1 Phase 1: Design-Time

30.1.2 Phase 2: Init-Time (Load / Compile)

30.1.3 Phase 3: Runtime (Execute)

30.2 Phase Diagram

30.3 How Strategies Affect Each Phase

30.4 Worker Threading Model

31 Cloud Deployment

31.1 UTLXe: The Production Engine

31.2 Azure Deployment

31.3 GCP Deployment

31.4 AWS Deployment

31.5 Docker and Kubernetes

31.6 Health and Monitoring

31.7 Hot Reload

32 SDKs and Wrapper Libraries

32.1 The Wrapper Pattern

32.2 Available SDKs

32.2.1 C# / .NET Wrapper (Built)

32.2.2 Go Wrapper (Built)

32.2.3 Python Wrapper (Planned)

32.2.4 Java / Kotlin Wrapper (Direct)

32.2.5 Node.js / TypeScript Wrapper (Planned)

32.3 Alternative: HTTP API (No SDK Needed)

32.4 Alternative: Dapr Sidecar (Cloud-Native)

32.5 SDK Comparison

33 Migration Guides

33.1 Migrating from XSLT

33.2 Migrating from DataWeave

33.3 Migrating from jq

33.4 Migrating from Custom Code

33.5 Migrating from Tibco BW / BWCE

34 Performance and Optimization

34.1 Execution Strategies

34.2 Performance Characteristics

34.3 Memory Optimization

34.4 GraalVM Native Image

34.5 Benchmarking

34.6 Production Tuning

35 Message Parsing, Memory, and Size Boundaries

35.1 How UTL-X Parses Messages

35.1.1 DOM-Style Parsing (What UTL-X Uses)

35.1.2 SAX-Style Parsing (Not Used by UTL-X)

35.1.3 StAX-Style Parsing (Not Used by UTL-X)

35.1.4 Summary: Parsing Approaches

35.2 Memory Expansion Factor

35.2.1 The Problem: Why 100KB of XML Becomes 5-10MB in Memory

35.2.2 Expansion Factors by Format

35.2.3 Real-World Examples

35.3 Practical Message Size Boundaries

35.3.1 Sweet Spot: 1 KB - 1 MB

35.3.2 Feasible: 1 MB - 10 MB

35.3.3 Not Recommended: 10 MB - 50 MB

35.3.4 Not Feasible: 50 MB+

35.3.5 Size Boundary Summary

35.4 Memory Sizing Guide for Production

35.4.1 Formula

35.4.2 The 75% Rule

35.5 Future: Streaming Support?

35.5.1 Could UTL-X Support Streaming?

35.5.2 Split-Before-Transform Pattern

35.6 Comparison: UTL-X vs Streaming Tools

36 Logging, Observability, and Compliance

36.1 Logging in UTL-X

36.1.1 What UTL-X Logs

36.1.2 Log Levels

36.1.3 Configuring Logging

36.2 When NOT to Log: Compliance and Data Protection

36.2.1 PCI DSS (Payment Card Industry Data Security Standard)

36.2.2 GDPR (General Data Protection Regulation)

36.2.3 HIPAA (Health Insurance Portability and Accountability Act)

36.2.4 NEN 7510 (Dutch Healthcare Information Security)

36.2.5 SOX (Sarbanes-Oxley Act — Financial)

36.3 Designing for Compliance

36.3.1 The Principle: Log Metadata, Not Data

36.3.2 Audit Logging Pattern

36.3.3 Masking in Error Messages

36.4 Prometheus Metrics (Safe by Design)

37 Quality Assurance and Testing

37.1 UTL-X's Layered Testing Strategy

37.2 Layer 1: Unit Tests (Kotlin)

37.3 Layer 2: Format Tests (Kotlin)

37.4 Layer 3: Module Tests (Kotlin)

37.5 Layer 4: Integration Tests (Kotlin)

37.6 Layer 5: Conformance Suite (Python)

37.7 Auto-Capture: Recording Tests from CLI Usage

37.8 UTLXe Engine Tests

37.9 Test-Driven Transformation Development

37.10 Continuous Integration

Part V

Future Outlook

Semantic validation, API contracts, and the road ahead

38 Semantic Validation: Beyond Syntax

38.1 Syntactical vs Semantic Validation

38.1.1 What UTL-X Does Today: Syntactical Validation

38.1.2 What UTL-X Does NOT Do: Semantic Validation

38.1.3 The Gap

38.2 Semantic Validation Languages

38.2.1 Schematron (XML world)

38.2.2 JSONTron

38.2.3 FEEL (Friendly Enough Expression Language)

38.2.4 Other Approaches

38.3 How Semantic Validation Could Work in UTL-X

38.3.1 Option 1: Built-in assertion syntax

38.3.2 Option 2: Schematron support for XML output

38.3.3 Option 3: FEEL integration for cross-format validation

38.3.4 Option 4: Validation DSL (UTL-X native)

38.3.5 Recommended Approach

39 API Contracts and Data Contracts

39.1 What Is an API Contract?

39.2 Two Dimensions of Contracts

39.2.1 Hierarchical Contracts (API-Centric)

39.2.2 Relational Contracts (Data-Centric)

39.2.3 Key Difference

39.3 Hierarchical API Contract Standards

39.3.1 OpenAPI (REST APIs)

39.3.2 RAML (REST APIs — MuleSoft)

39.3.3 SOAP / WSDL (XML Web Services)

39.3.4 AsyncAPI (Event-Driven APIs)

39.3.5 gRPC / Protocol Buffers

39.4 Abstract vs Concrete API Contracts

39.4.1 Abstract Contracts (Schema-Only)

39.4.2 Concrete Contracts (Full API Specification)

39.4.3 The Relationship

39.5 Relational Data Contract Standards

39.5.1 Data Contract (datacontract.com)

39.5.2 ODCS (Open Data Contract Standard — PayPal)

39.5.3 Comparison: datacontract.com vs ODCS

39.6 How UTL-X Relates to API and Data Contracts

39.6.1 UTL-X as the Transformation Layer

39.6.2 Schema Formats in UTL-X

39.6.3 Future: Data Contract Integration

39.6.4 Future: Contract-Driven Transformation

40 Formats Not Yet Covered (and Why)

40.1 Formats on the Horizon

UTL-X currently supports 5 data formats (Tier 1) and 6 schema formats (Tier 2). Several important formats are NOT yet supported. This chapter explains each, why they matter, and the path to support.

40.2 TOML (Tom's Obvious Minimal Language)

40.3 JSON-LD (JSON for Linked Data)

40.4 HL7 FHIR (Fast Healthcare Interoperability Resources)

40.5 EDI / EDIFACT (Electronic Data Interchange)

40.6 HL7 v2 (Health Level Seven Version 2)

40.7 Parquet / Arrow (Columnar Data)

40.8 MessagePack / CBOR (Binary JSON)

40.9 GraphQL

40.10 Apache Thrift

40.11 FlatBuffers

40.12 Summary: Format Priority Matrix

41 Competitive Landscape: Mapping and Integration Tools

41.1 Command-Line Transformation Tools

41.1.1 jq — The JSON Swiss Knife

41.1.2 yq — YAML/JSON/XML/CSV/TOML Processor

41.1.3 JSONata — JSON Query and Transformation

41.1.4 XSLT — The XML Veteran

41.2 Enterprise Integration Platforms (iPaaS)

41.2.1 MuleSoft (Salesforce)

41.2.2 Tibco BusinessWorks (BWCE)

41.2.3 Boomi (formerly Dell Boomi)

41.2.4 Workato

41.2.5 Informatica / IICS

41.3 Hyperscaler Native Tools

41.3.1 Azure

41.3.2 AWS

41.3.3 GCP

41.3.4 Hyperscaler Comparison Table

41.4 ERP Vendors

41.4.1 SAP

41.4.2 Oracle

41.4.3 Microsoft Dynamics 365

41.4.4 Odoo

41.4.5 Summary: ERP + UTL-X Integration Pattern

41.5 Open Source Alternatives

41.5.1 Apache Camel

41.5.2 Smooks

41.5.3 Ballerina

41.6 Positioning Matrix

42 Open-M: The Integration Controller (Teaser)

42.1 What Is Open-M?

42.2 How UTL-X Fits in Open-M

42.3 Why Open-M + UTL-X?

42.4 Open-M Architecture (Preview)

42.5 MPPM: Message Processing Pipeline Model

42.5.1 What Is MPPM?

42.5.2 MPPM Stages

42.5.3 MPPM Message Envelope

42.5.4 How MPPM Maps to UTL-X

42.5.5 Why MPPM Matters for UTL-X Users

42.6 What This Means for UTL-X Users

43 AI and UTL-X: The Future of Automated Transformation

43.1 The Current State: AI-Assisted UTL-X Development

43.1.1 What Works Today

43.1.2 The Problem: Token Cost and Context

43.2 The Vision: A UTL-X Fine-Tuned LLM

43.2.1 What Fine-Tuning Would Enable

43.2.2 Token Savings Calculation

43.3 Training Data: What Would Be Needed

43.3.1 Available Training Corpus

43.3.2 Additional Training Data Needed

43.4 Fine-Tuning Approaches

43.4.1 Option A: LoRA Fine-Tune on Open Model

43.4.2 Option B: Fine-Tune on Frontier Model

43.4.3 Option C: RAG (Retrieval-Augmented Generation) — No Fine-Tuning

43.4.4 Recommended Path

43.5 Investment Estimate

43.5.1 Development Costs

43.5.2 Ongoing Costs

43.5.3 Revenue Potential

43.6 What an Investor Would Want to Know

43.6.1 The Opportunity

43.6.2 The Moat

43.6.3 The Ask

43.6.4 Why NOW

43.7 Ethical Considerations

44 Why Kotlin and GraalVM (and Why Not Other Languages)

44.1 Why Kotlin Was the Right Choice

44.1.1 Kotlin's Strengths for UTL-X

44.1.2 Kotlin vs Alternatives Considered

44.2 Why GraalVM on Top of Kotlin

44.2.1 GraalVM Native Image

44.2.2 GraalVM for Cloud (UTLXe)

44.3 Why NOT a JavaScript/TypeScript Version

44.3.1 The Temptation

44.3.2 Why It Would Fail

44.4 Why NOT WebAssembly (WASM)

44.4.1 The Promise

44.4.2 Why It Doesn't Work (Today)

44.5 Why Multiple Language Implementations Lead to Deviation

44.5.1 The N-Implementation Problem

44.5.2 The Wrapper Pattern Avoids Deviation

Part VI

Case Studies

Real-world use cases, step-by-step walkthroughs, and recipes

45 Case Studies and Real-World Use Cases

45.1 Case Study 1: European E-Invoicing (Peppol / UBL)

45.2 Case Study 2: Healthcare HL7 FHIR Integration

45.3 Case Study 3: Financial Services — SWIFT ISO 20022 Migration

45.4 Case Study 4: Retail — Multi-Channel Order Processing

45.5 Case Study 5: IoT / Manufacturing — Sensor Data Normalization

45.6 Case Study 6: Government — Tax Filing Integration

45.7 Azure Marketplace: Getting Started Guide

45.8 Recipes: Common Transformation Patterns

Part VII

Reference

Grammar, appendices, and general reference

46 Grammar Reference

46.1 UTLX 1.0 Grammar (EBNF)

46.2 Header Syntax

46.3 Output Options

46.4 Expression Syntax

46.5 Reserved Words

46.6 Identifier Rules

47 Appendices

47.1 Appendix A: Format Conversion Matrix

47.2 Appendix B: Error Messages Reference

47.3 Appendix C: Comparison Tables

47.4 Appendix D: Industry Standards Reference

47.5 Appendix E: Glossary

47.6 Appendix F: Version History

47.7 Appendix G: License

Part VIII

Standard Library Encyclopedia

Complete reference for all 652 functions — organized by category with examples

48 Standard Library Reference

48.1 How to Read This Reference

48.2 Core Functions (49)

48.3 String Functions (83)

48.4 Array Functions (67)

48.5 Math Functions (37)

48.6 Date/Time Functions (68)

48.7 Type Functions (27)

48.8 Object Functions (1)

48.9 Encoding Functions (30)

48.10 XML Functions (60)

48.11 Security Functions (16)

48.12 JSON Functions (6)

48.13 Binary Functions (47)

48.14 Utility Functions (27)

48.15 Financial Functions (16)

48.16 Geospatial Functions (8)

48.17 CSV Functions (12)

48.18 YAML Functions (22)